

</> fn main()

Proyecto personal · Sistemas de bajo nivel

Construir un JDK desde cero en Rust

Roadmap técnico por hitos

Autor: **Esteban Nicolás Larena**

Fecha: **23 de agosto de 2026**

Contenido

Introducción y alcance	3
Los tres pilares	3
El problema del bootstrap	3
Orden de construcción	3
Fase A — La JVM (el runtime)	4
Concurrencia — ruta al multithread profesional	6
Fase B — El compilador (javac, en Rust)	8
Compilador — pasadas y estructuras internas	16
Fase C — Las bibliotecas	17
Fase D — Herramientas (la “DK”)	18
Fase E — Cerrar el círculo	18
Más allá del JDK — la plataforma	19
Fase F — burst (optimizador)	19
Fase G — plain_data / value types	20
Fase H — KajiLibrary (biblioteca dogfooded)	21
Fase I — Depurador (JVMTI / jdb)	26
Fase J — jlink (imagen de runtime)	28
Estado actual	30

Introducción y alcance

Este documento traza la ruta para construir un **JDK educativo desde cero en Rust**: no para competir con HotSpot, sino para **cargar y ejecutar bytecode real** y, eventualmente, compilar y correr código Java escrito y compilado con herramientas propias. El objetivo es doble: aprender sistemas de bajo nivel a fondo y completar un reto de ingeniería de gran calibre.

Un JDK no es una sola pieza. Son **tres pilares** que se sostienen entre sí:

Los tres pilares

Pilar	Qué hace	Lenguaje
JVM (el runtime)	Carga y ejecuta bytecode. Parser de <code>.class</code> → intérprete → objetos → heap → GC.	Rust
Compilador (<code>javac</code>)	Convierte <code>.java</code> en bytecode <code>.class</code> : lexer → AST → análisis semántico → emisión.	Rust
Bibliotecas (<code>java.base</code>)	<code>Object</code> , <code>String</code> , <code>System</code> , colecciones... escritas <i>en Java</i> , compiladas por el compilador propio.	Java

El problema del bootstrap

Las bibliotecas necesitan al **compilador** (para compilarse) y a la **JVM** (para ejecutarse). Si el compilador se escribiera en Java, se necesitaría a sí mismo para compilarse — el clásico problema del huevo y la gallina.

*Decisión de diseño: el compilador se escribe **en Rust**, igual que la JVM. Así Rust compila al compilador, el compilador compila las bibliotecas, y la JVM las ejecuta. El ciclo queda roto.*

Orden de construcción

La **JVM va primero**, sin excepción: tanto la salida del compilador como las bibliotecas son inertes sin un motor que las ejecute, y no se puede ni *probar* que el compilador genera bytecode correcto sin algo que lo corra.

A · JVM → B · Compilador → C · Bibliotecas → E · Cerrar el círculo
(motor) (.java → .class) (en Java) (todo junto)
└─ D · Herramientas se completan en el camino (javap, java, jar...)

Horizontes: Base el núcleo, por aquí empezamos Avanzado más duro, segunda pasada Cumbre lo más difícil, pero vamos a llegar

*Cómo leer el roadmap: es una **ruta ordenada por hitos**. Cada hito tiene un criterio de éxito medible y no se cierra hasta cumplirlo. El orden respeta las dependencias: no se puede el hito N sin el N-1.*

Fase A — La JVM (el motor que ejecuta bytecode)

A0 Parsear el .class (≡ javap) núcleo logrado Base

Base Lector de bytes (cursor big-endian) — el `Reader` (`u1` / `u2` / `u4`)

Base Constant pool (17 clases de entrada; 1-indexed, Long/Double = 2 slots)

Base Header: magic, versiones, flags, this/super/interfaces

Base fields, methods, attributes: `Code`, `LineNumberTable`, `SourceFile`, `StackMapTable` (los 7 frame types)

Base Desensamblado de bytecode (tabla de opcodes completa) con comentarios `// ...` resueltos

Base Volcado `javap` brief y `-v` byte-idéntico; flags de visibilidad (`-public` / `-protected` / `-package` / `-p`)

Avanzado *Pendiente (no esencial):* `Signature`, `BootstrapMethods`, `InnerClasses`, `Exceptions`, `ConstantValue`, anotaciones, `Record`, `NestHost/Members`, `LocalVariableTable`

✓ **Éxito: logrado** — el volcado coincide **byte a byte** con `javap -v` (y brief) sobre 12 fixtures.

A1 Intérprete mínimo logrado Base

Base *Frame*: pila de operandos + variables locales

Base Contador de programa (PC) y *loop* de despacho de opcodes

Base Opcodes: `iconst`, `iload`, `istore`, `iadd`, `ireturn`

Base Parseo de descriptores de método (`(II)I`)

✓ **Éxito: logrado** — ejecutar un método que suma dos enteros (`Add.add`).

A2 Control de flujo y métodos logrado Base

Base Saltos: `if_icmpgt`, `goto`, comparaciones

Base *Method area* (metadatos de clases cargadas)

Base `invokestatic` + pila de frames (llamadas anidadas)

✓ **Éxito: logrado** — corren `factorial` recursivo (120) y `fibonacci` (55).

A3 Objetos y heap logrado Base

Base Heap + allocator simple; arena de bytes (`Vec<u8>`) + `malloc` bump

Base *Layout* de objetos (campos, con herencia) y de clases (con *vtable*)

Base `new`, `getfield` / `putfield`

Base `invokevirtual` / `invokespecial` / `invokeinterface` + dispatch dinámico (*vtable* + *itable*)

Base Arrays (objetos y primitivos int-category, con ancho fiel)

✓ **Éxito: logrado** — herencia (`Dog extends Animal`) y dispatch dinámico verificados: `a.sound()` sobre una referencia `Animal / Speaker` corre `Dog.sound`.

A4 Robustez del runtime logrado Base

Base Excepciones: `athrow` + tablas de excepción + *stack unwinding*; `finally`; excepciones **implícitas** (NPE, índice, cast, tamaño negativo)

Base *Linking*: resolución bajo demanda + **verificador** (type-checking con *StackMapTable*); errores de linkage (`NoClassDefFound` / `NoSuchMethod`)

Base Inicialización de clase (`<<clinit>`), perezosa, super-primero)

Base Jerarquía de class loaders (bootstrap/app + delegación)

✓ **Éxito: logrado** — un `try` / `catch` atrapa una `Boom` (con *unwinding* entre frames y `finally`), y `Base` / `Derived` inicializan en orden super-primero.

A5 Nativos + GC logrado Avanzado

Avanzado Puente a métodos nativos (I/O real) — `System.out.println` imprime de verdad

Avanzado *Intrinsics* — `getClass`, `hashCode`, `Math`, `arraycopy`, `String` / `Class`...

Avanzado GC mark & sweep — y más: **compactante**, free list con *coalescing*, política de fragmentación, 4 disparadores sobre *safepoint*, y **marcado transitivo** correcto

✓ **Éxito: logrado** — `System.out.println` imprime de verdad y el GC recolecta (y **compacta**) basura.

A6 Cumbre del runtime **en progreso** **Cumbre**

Cumbre  **Verificador de bytecode JVMs-estricto** — verifica **con o sin** `StackMapTable` (inferencia por punto fijo como fallback, y *cross-check* de la tabla contra la inferencia); gate **estructural** (§4.9: targets, no caerse del final, `jsr / ret`, tabla de excepciones); tipos de locales estrictos + cota de `max_stack`; reglas de **objetos sin inicializar** (`<init>` una sola vez); **handlers de excepción** tipados; y **acceso/linkage** (regla `protected`, `count` de `invokeinterface`)

Cumbre  **Sistema de tipos completo** — `int / long / double / float` ejecutados y verificados: cómputo, conversiones, comparaciones, división con excepción, categoría-2 (params/campos/estáticos/arrays/frames), y el **lattice de referencias** (covarianza de arrays + `join`/LUB)

Cumbre  **GC generacional** — young (Eden + 2 *survivors*) con colector por **copia** (minor: evacúa/promueve por edad, estilo Cheney con *forwarding*), **write barrier + remembered set** para las raíces `old→young`, y Old con **mark-sweep-compact**; *minor* disparado al llenarse Eden, *major*/full por occupancy o explícito

Cumbre  **Referencias débiles** (`java.lang.ref`) — `WeakReference` + `ReferenceQueue`: el major trata `referent` como débil y, al morir el referente, lo limpia (`get()→null`) y **encola** la referencia. Base para `PhantomReference` / `Cleaner` (post-mortem)

Cumbre **Hilos / concurrencia** (*en progreso*) — objetivo: **multithread profesional**, por fases.  **Fase 0 — green threads**: `java.lang.Thread` + `start / join / sleep`, scheduler cooperativo round-robin en el safepoint, varios call stacks por-thread, GC sobre las raíces de **todos** los threads, terminación; verificado (`Threads.run → 100`) y visible en el step-visualizer.  **Monitores**: `synchronized` de **bloques y métodos** (`ACC_SYNCHRONIZED`, sin opcode — el VM toma/suelta el monitor al entrar/salir del frame), reentrancia, señalización `wait / notify / notifyAll` sobre *wait-set/blocked-set*, `IllegalMonitorStateException`, `wait(timeout)` y monitor **GC-safe** (las claves se remapean por el *forward* del GC, así que se compacta con monitores tomados).  **Substrato de SO + GIL (E1+E2)**: `JVM_THREADS=os-gil` corre cada `Thread.start()` en un `std::thread` real tras un **GIL** (`Arc<Mutex<SharedVm>>`, un opcode por lock), con `park / unpark` reales y `sleep` en tiempo real — correcto pero **serializado** (referencia/oráculo).  **API de Thread (H1)**: `currentThread / yield / nombre/id / isAlive, getState()` con los seis estados de `Thread.State`, `new Thread(runnable)`, `start` dos veces → `IllegalThreadStateException`, e `interrupt()` que despierta `sleep / join / wait`.  **Sacar el GIL (E3/H3 — paralelismo real, subconjunto conservador)**: `JVM_THREADS=os` corre los opcodes **frame-local** (aritmética int/stack/ramas) **lock-free en paralelo** sobre un `RunningCtx` por-hilo con code cache; solo `SharedVm` se lockea; el GC que mueve objetos frena a todos con un **safepoint stop-the-world cooperativo**. El `Arc<Mutex<JVM>>` de toda-la-VM desapareció.  **Ensanchado**: (W1) frame-local completo (int/long/float/double, shifts, conversiones, refs, ramas); (W3) lecturas compartidas concurrentes bajo `RwLock` (`getField / arraylength / array-loads`); (W2) **asignación lock-free** (`new / newarray`, Eden en un arena `UnsafeCell` **verificado con Miri**).  **JMM (H4 — modelo de memoria relajado)**: campos no-volatile **lock-free** (`Relaxed` atómico), `volatile` con `Acquire / Release` (`AtomicU64` sin *tearing* para `long / double`), Eden sobre un substrato atómico 8-alineado **Miri-verificado**; test de publicación `volatile` que coincide `green=os-gil=os`.  **CAS (H5)**: `compareAndSwap` intrínseco (`cas_u32 / cas_u64` sobre el `AtomicRegion`) + `AtomicInteger / AtomicLong / AtomicReference` (la CAS de referencia pasa por el write barrier).  **Núcleo + primeras utilidades de java.util.concurrent (H6)**: **AQS** (`AbstractQueuedSynchronizer`) en modo **exclusivo y compartido**, con `ReentrantLock / Condition`, `ReentrantReadWriteLock`, `Semaphore`, `CountDownLatch`, `CyclicBarrier`, `ArrayBlockingQueue / LinkedBlockingQueue / PriorityBlockingQueue / DelayQueue`, el **framework Executor** (`ThreadPoolExecutor` + `ScheduledThreadPoolExecutor` + `submit / Future / FutureTask / Callable`), un `ConcurrentHashMap` (encadenamiento + *lock striping*) y un `CompletableFuture` (`supplyAsync / thenApply / thenCompose`, completación excepcional con `exceptionally`, y `thenCombine` que fusiona dos futuros con un `BiFunction`) —todo en Java sobre AQS (o el monitor), validado `green=os-gil=os`; motivaron `Object.equals`, `java.util.function / Objects`, `Comparator / Comparable / Delayed` y `System.nanoTime`.  **Periféricos de Thread**: `ThreadLocal` (aislamiento por-hilo real — lista de asociación colgada del `Thread` actual, keyed por identidad GC-safe; verificado con 4 hilos OS), **prioridad** (`get/set` + rango `[1,10]`) y **daemon** (atributo + regla de ciclo de vida).  **Falta**: el **volumen** restante de j.u.c (deques, *resize*/lecturas lock-free del `ConcurrentHashMap`), locks por-estructura finos y `UncaughtExceptionHandler`; y un **bug residual** conocido — una referencia *stale* puede alcanzar un frame bajo **GC intenso concurrente** en `JVM_THREADS=os` (guard parcial landeado; próximo paso `ThreadSanitizer`; `green / os-gil`, el oráculo, **no** afectados). El paralelismo de cómputo del LLM vive en **kernels off-heap** (rayon/CUDA), no en el heap Java. La hoja de ruta completa (JMM, CAS, `java.util.concurrent`) tiene su propia sección («Concurrencia»).

Cumbre  **Cobertura del set de opcodes — 199/199 alcanzables: completo** — cerrados `nop` (0x00), `goto_w` (0xc8), el prefijo `wide` (0xc4: índice de local de 16 bits, la única forma de direccionar los slots pasados el 255; `wide iinc` mide 6 bytes, el resto 4 — salió barato porque los handlers de locales ya reciben `slot: usize` y nunca se enteran del ancho, así que ensanchar es puro *decoding*) y `multianewarray` (0xc5: aloca recursivamente **sólo** los `dimensions` niveles indicados — `new int[3][]` deja los internos en `null` —, validando todos los conteos *antes* de alocar nada; los niveles superiores guardan referencias y sólo el más interno usa el ancho real del elemento, y las referencias hijo pasan por `store_reference` para que el *remembered set* vea los punteros `old→young`. Hubo que **modelarlo también en el verificador**, sin lo cual la clase corría con el verificador claudicando en silencio). Los 3 de `jsr / ret / jsr_w` quedan **excluidos por diseño** —**JVMS §4.9.1** los prohíbe en class files de versión 50.0+ (Java 6 en adelante) — con la postura **leer sí, ejecutar no**: el desensamblador los soporta completo (requisito de A0, que se mide contra `javap` sobre `java.base`) y el gate estructural del verificador los rechaza. Nota de diseño: `subrutinas-jsr-ret.md`. El número honesto es entonces **199 de 199 alcanzables: completo**

Cumbre  **invokedynamic** (0xba) — **no era un opcode, era un subsistema**, y corre: **5 de las 6 fábricas** que emite `javac`. `StringConcatFactory` (todo "a" + b desde Java 9), `SwitchBootstraps.typeSwitch` (`switch` sobre patrones de tipo y de enum), `ObjectMethods` (`equals / hashCode / toString` de un `record`, los tres desde una sola entrada de `BootstrapMethods`, distinguidos por el nombre del call site), `LambdaMetafactory.metafactory` (lambdas y method references) y `ConstantBootstraps.invoke` (constantes dinámicas). La sexta, `altMetafactory`, necesita serialización y queda **fuera de alcance**. `LambdaMetafactory` **genera una clase real** al vuelo (con el escritor de `.class`): implementa la interfaz funcional, con campos para las capturas y un método SAM que reenvía al handle — igual que el JDK (que spinnea con ASM), ya sin el atajo del objeto sintético. Y con el **modelo de objetos de java.lang.invoke** cerrado (abajo), `ConstantBootstraps.invoke` **se movió a Java** (es sólo `handle.invokeWithArguments(args)`); los otros *bootstrap methods* siguen como intrínsecos, en `java.lang.constant` en Java. Ruta completa, correcciones y mediciones: `invokedynamic-ruta.md`

Avanzado  **ldc de literales de clase** (`Foo.class`, `int[].class`) — empuja el mirror `Class<...>`, cacheado por Class ID (así `Foo.class == Foo.class`) y **sin inicializar** la clase: un literal no es *uso activo* (§5.5). Los de array reusan el mirror sintético que arma `anewarray`, ya que una clase de array no tiene `.class` que cargar

Cumbre  **La VM puede invocar Java** (`call_java`) — empuja un frame propio y lo corre con un bucle de `run_one` anidado, devolviendo el resultado. Resultó ser el **caso general de lo que la inicialización de clases ya hacía**: `<clinit>` era la variante sin argumentos ni resultado. Importa porque **los intrínsecos dejan de ser terminales**: un nativo que sólo puede calcular y devolver tiene que reimplementar lo que necesite de la biblioteca. Con esto, `String.valueOf(Object)` llama al `toString()` del objeto, un `record` pregunta el `equals / hashCode` de sus componentes, y un `condy` ejecuta su bootstrap

Cumbre ✓ **Modelo de objetos de** `java.lang.invoke` (`MethodHandle` / `MethodType` / `Lookup`) — **cierra 0xba**. Las clases viven en **Java** (`bootstrap/java/lang/invoke/`), con `MethodHandle.invoke` / `invokeWithArguments` **nativos** — polimorfía de firma (§2.9.3), interceptados en `invokevirtual` antes de la resolución normal: el único intrínseco que de verdad corresponde. **El premio se cobró:** `ConstantBootstraps.invoke` es ahora **Java** (`return handle.invokeWithArguments(args)`), lo que exigió hacer el **cache de condy raíz de GC** (se remapea con el `forward` del colector). `ldc` de `MethodHandle` / `MethodType` —que `javac` **nunca** emite— se prueba con **class files hechos a mano por el escritor de .class** (su estreno). Kinds: `static/virtual/special/constructor` + **mirrors de primitivos** (`int.class` → `Integer.TYPE`, sin boxing). Y **LambdaMetafactory** genera clases reales al vuelo (vía el escritor de `.class`), implementando la interfaz funcional y reenviando al `handle` — como el JDK, ya sin el atajo del objeto sintético

Cumbre JIT (bytecode → código nativo)

✓ **Éxito: parcial** — **verificador JVMs-estricto**, **GC generacional**, el **set de opcodes completo** (`invokedynamic` incluido) e **hilos de SO con paralelismo real ensanchado** (GIL removido; cómputo + lecturas + asignación lock-free en `JVM_THREADS=os`, con el `unsafe` Miri-verificado) y el **JMM relajado** (campos no-volatile lock-free con `Relaxed`, `volatile` con `Acquire` / `Release`, Eden atómico) logrados; falta locks por-estructura finos y/o el JIT.

A7 Conformidad JVMs ✓ auditoría 2026-08-12 — 13/13 cerrados **Cumbre**

Cumbre ✓ **Default methods de interfaces (JSR 335, Java 8)** — estaba **roto** (`NoSuchMethodError`: la vtable heredaba sólo de la superclase). Ahora fusiona los defaults de las superinterfaces (BFS desde las directas = regla *maximally-specific* para todo lo que emite `javac`; la clase siempre gana; un método abstracto —sin `Code`— nunca toma slot). Cubre `invokeinterface` e `invokevirtual`. Los static de interface ya funcionaban. Test → 115 (`green=os-gil=os`)

Cumbre ✓ **Throwable completo** — `getMessage` / `toString` (nativo: lee el nombre de clase runtime) + **stack traces**: la VM captura la traza (`\tat pkg.Class.method`, innermost-first) en el `unwind` —punto único de faults implícitos y `throw`— y la guarda en el objeto (interning GC-safe); `printStackTrace()` la imprime. Constructores (`String`) en toda la jerarquía

Cumbre ✓ **ExceptionInInitializerError** (JVMs §5.5) — un `<clinit>` que lanza **propagaba... nada**: se tragaba el fallo y el `getstatic` devolvía 0. Ahora propaga al código que disparó la init (un `floor` de unwinding por `call_java` + `pending_exception`), envuelve si no es `Error`, y deja la clase **erroneous** → `NoClassDefFoundError` al reuso

Cumbre ✓ **Uncaught exception** (§2.10) — estaba **roto**: una excepción sin catch hacía `panic!` de **toda la VM**. Ahora imprime el formato exacto del JDK (`Exception in thread "Thread-1" java.lang.RuntimeException: ...` + la traza capturada en el `throw`) y termina **solo ese thread** — los joiners despiertan, `main` sigue; si es `main`, termina el programa. Test → 42 (`green=os-gil=os`). Falta la API `setUncaughtExceptionHandler`

Cumbre ✓ **ArrayStoreException** (§6.5) — `aastore` hace el chequeo dinámico (clase runtime del valor vs tipo de elemento, `is_subtype` con covarianza; null siempre válido). Test → 42. **Bonus: 2 bugs latentes expuestos y arreglados** — los mirrors de clases array se alocaban en **Eden** (el minor GC los movía → índice/headers colgados → ASE espurio en stores válidos; ahora **Old-pinned** como todo mirror) y `anewarray` nombraba `[LJ;`; en vez de `[J` (intérprete + verificador). Regresión → 64

Cumbre ✓ **StackOverflowError** (§6.3) — `MAX_FRAMES = 2000` en `push_frame_locked` (el embudo de los 5 invokes), chequeado **antes** de adquirir el monitor (no filtra locks de `synchronized`); `call_java` cubierto vía `pending_exception`. Test → 42

Cumbre ✓ **OutOfMemoryError** (§6.3) — agotamiento **recuperable para las alocaiones de bytecode** (`new / newarray / anewarray / multianewarray` → `try_malloc` → `throw`); las internas del VM conservan el `panic`, documentado. Test → 42 (recursión roteando 512 KiB/frame agota los 16 MiB reales)

Cumbre ✓ **Autoboxing / wrappers** (JLS §5.1.7) — los 8 wrappers reales: `Integer` / `Long` / `Short` / `Byte` con `valueOf` + **cache -128..127** (identidad `==` verificada: `100==100 true` / `200==200 false`), `Character` (0..127), `Boolean` (TRUE/FALSE canónicos), `Double` / `Float`. Test → 42. Colateral: destapó un **2º reproducir del heisenbug os-parallel** (single-thread ~50%, `Long.<clinit>` + Eden lleno → `ArithmeticException` espuria; `java/BxDbg{T,Y}.java`)

Cumbre ✓ **Object.clone()** + **Cloneable** (JLS §10.7) — copia shallow interceptada en `invokevirtual` e `invokespecial` (`super.clone()`), `Cloneable` → `CloneNotSupportedException`; **arrays incluidos** (`"I".clone()` pre-vtable). Clon alocado en **Old** (una alocación Eden podría mover el fuente a mitad de copia); refs vía write barrier. Test → 42

Cumbre ✓ **Class.getName()** / **getSimpleName()** — nativos sobre el mirror, formato JDK completo (clases con puntos; arrays `[Ljava.lang.String;` / `String[]` / `int[]`). Test → 42 (incl. `String.class.getName()` vía `ldc` de `Class`)

Base ✓ **Anotaciones en runtime** (JSR 175) — `Class.isAnnotationPresent(Class)` sobre el `RuntimeVisibleAnnotations` ya parseado. *Fuera de alcance*: `getAnnotation()` devolvería un **objeto** anotación → hacen falta clases proxy por `@interface` (dynamic proxies + modelo de `Method`)

Base ✓ **java.lang.ref: Soft + PhantomReference** — phantom con `get()` → null pero `referent` intacto (el GC encola igual; el override impide resucitar). La política soft vive en el **GC**: el referent se traza **fuerte** salvo en una colecta pedida por **presión** de memoria, donde muere como un weak. Verificado en ambas direcciones (42 normal / 36 bajo presión). *Fuera de alcance*: `Cleaner`; `finalize()` no se implementa por decisión

Base ✓ **Ciclo de vida: System.exit + Runtime** — interceptado en `invokestatic` (el bridge de nativos solo sabe *continuar*; terminar exige un `Step`): descarta los frames **sin** liberar monitores ni desenrollar —el apagado ordenado que `exit` no debe hacer— y propaga el status desde cualquier thread. Contrastado con el `java` real. *Fuera de alcance*: shutdown hooks

Base ✓ **Periféricos de Thread** — **UncaughtExceptionHandler** **invocado de verdad en Java**, y **antes** de popear el último frame: el lookup aloca y ese frame es la única raíz de GC que sostiene la excepción; un handler que lanza queda contenido por el `exception_floor`. Más `sleep(long,int)`, `onSpinWait()` y `ThreadGroup` (registro enteramente en Java)

Base ✓ **Regla fina de init de interfaces** (§5.5) — **era un bug**: el `<clinit>` de una interface con default methods nunca corría (ni directo ni indirecto). Ahora un BFS junta las superinterfaces que declaran un método con `Code`, y **devuelve vacío si la clase es una interface** — la regla "una interface no inicializa sus superinterfaces" queda en el tipo de retorno, no en un `if`

Base ✓ **Cubierto ya auditado**: class file 199/199, verificador, JMM, indy 5/6, j.u.c núcleo, records/patterns. **Fuera de alcance por diseño**: módulos (JSR 376) y NIO en la VM, `finalize()`, y lo que depende de reflexión completa (Fase C)

✓ **Éxito: auditoría CERRADA, 13/13** (2026-08-12) — el runtime contra la JVM Specification y los JSR estructurales (detalle: `holdon.md`). Los 8 hallazgos gruesos (default methods, uncaught exceptions, `ArrayStoreException`, `StackOverflowError`, `OutOfMemoryError`, autoboxing, `clone`, `Class.getName()`) y los 5 menores (anotaciones runtime, `Soft/Phantom refs`, `System.exit` / `Runtime`, periféricos de `Thread`, init de interfaces), **cada uno validado por el oráculo** (`green=os-gil=os` + `10x os-parallel`). Tres resultaron **bugs reales** que nadie había visto: los mirrors de clases array vivían en Eden, `anewarray` nombraba mal los arrays anidados, y el `<clinit>` de una interface con default methods no corría nunca. De paso apareció un 2º reproducir del heisenbug os-parallel. Lo único que queda es lo marcado *fuera de alcance* con su razón (`getAnnotation` sin dynamic proxies, `Cleaner`, shutdown hooks, `finalize()`).

Concurrencia — ruta al multithread profesional

La ejecución concurrente (Hito A6) merece su propio mapa, porque es donde el proyecto apunta a **multithread profesional**. Cada planta se apoya en la de abajo: no hay `ReentrantLock` sin CAS, ni CAS útil sin un modelo de memoria, ni modelo de memoria que *importe* sin hilos reales que lo hagan necesario. **El edificio está en pie hasta la planta 6**: el substrato pasó de green threads a hilos de SO con paralelismo real, y `java.util.concurrent` —la cumbre— es hoy el hito **H9** de KajiLibrary. Lo que queda son terminaciones, no estructura: AQS como base común, los *fences* explícitos, la preempción.

Y hay un giro que este mapa no anticipaba. La planta 6 se dio por hecha **sin** construir la planta 5 debajo suyo *en el sentido clásico*: en una VM cuyos hilos se intercalan **entre opcodes**, un contador atómico hecho con `synchronized` es *observacionalmente indistinguible* de uno lock-free con CAS. Los atómicos existen y funcionan, pero se apoyan en el monitor de la planta 2, no en un CAS de hardware. La torre sigue siendo la ruta correcta hacia el paralelismo real; lo que muestra el atajo es que **la semántica se puede tener antes que la implementación**.

RASCACIELOS DE LA CONCURRENCIA · construir hacia arriba ↑

[6] java.util.concurrent · núcleo + utils · falta vol. HECHO

[x] AQS (excl + shared) · ReentrantLock/RWLock · Condition

[x] Semaphore · CountDownLatch · CyclicBarrier · ArrayBlockingQueue

[x] ThreadPoolExecutor · ScheduledThreadPoolExecutor · Future

[x] ConcurrentHashMap · Linked/Priority/DelayQueue · Completabl

[] deques · resize/lecturas lock-free CHM · locks finos

[5] Atómicos / CAS · raíz lock-free HECHO

[x] compareAndSwap · AtomicInteger / AtomicLong / Reference

[4] Modelo de Memoria (JMM) · relajado (H4)

✓

●

[x] volatile=Acq/Rel · no-volatile Relaxed · no-tearing · Eden atómic

[3] API de Thread · control de hilos HECHO

[x] start · run · join · sleep · currentThread · yield

[x] getState (6 estados) · isAlive · name/id · start x2

[x] interrupt despierta sleep/join/wait · daemon/prio/TLocal

[2] Monitor intrínseco · reentrante PARCIAL

[x] monitorenter/exit · synchronized (bloques Y métodos)

[x] reentrancia · wait / notify / notifyAll · wait(timeout)

[] IllegalMonitorStateException · GC-safe · thin locks

[1] Substrato de ejecución · paralelismo real

✓

●

[x] green threads (cooperativo, default + visor)

[x] hilos de SO reales · park/unpark · os-gil (referencia)

[x] GIL removido: frame-local + lecturas + alloc lock-free

[x] W2 alloc: Eden en arena UnsafeCell (Miri-verificado)

[x] JMM (H4) relajado · [] locks por-estructura finos

▲ todo descansa en la planta 1 ▲

Estado:

✓

 hecho

●

 parcial

□

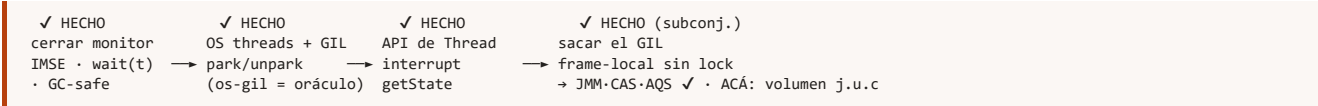
 falta

Planta	Estado	Qué falta para "profesional";
6 · java.util.concurrent	 núcleo + utils (H6)	Hecho: AQS (AbstractQueuedSynchronizer) en modo exclusivo y compartido —base de casi todo—, ReentrantLock + Condition, ReentrantReadWriteLock, Semaphore, CountdownLatch, CyclicBarrier, ArrayBlockingQueue / LinkedBlockingQueue / PriorityBlockingQueue / DelayQueue (una/dos locks / min-heap / delay), el framework Executor (ThreadPoolExecutor + submit / Future / FutureTask / Callable, y un ScheduledThreadPoolExecutor con min-heap por deadline), un ConcurrentHashMap (encadenamiento + lock striping) y un CompletableFuture (supplyAsync / thenApply / thenCompose / get, exceptionally y thenCombine), todo en Java sobre AQS (o el monitor) y validado por el oráculo (green==os-gil==os). Motivaron agregar Object.equals, java.util.function / Objects, Comparator / Comparable / Delayed y el intrínseco System.nanoTime. Falta (volumen): dequeues, resize/lecturas lock-free del ConcurrentHashMap.
5 · Atómicos / CAS	 hecho (H5)	Hecho: compareAndSwap (la instrucción atómica raíz: cas_u32 / cas_u64 sobre el AtomicRegion, AcqRel) y AtomicInteger / AtomicLong / AtomicReference —la CAS de referencia pasa por el write barrier—. Verificado: AtomicLong 64-bit + AtomicReference identity-CAS → 202.
4 · Modelo de Memoria (JMM)	 relajado	Hecho (H4): campos no-volatile lock-free (Relaxed), volatile = Acquire / Release, no-tearing de long / double (AtomicU64), Eden atómico Miri-verificado; el cache de condy es raíz de GC. happens-before vía los atómicos + el RwLock. Falta: semántica de final, VarHandle /CAS.
3 · API de Thread	 hecho (H1)	Hecho: start / run / join / sleep, currentThread, yield, nombre/id, isAlive, getState() (los seis estados; ThreadStatus pasó a cinco + NEW derivado, con sleep_until / joining_on plegados dentro), Thread(Runnable), start dos veces → IllegalThreadStateException, e interrupt / InterruptedException que despierta sleep / join / wait (re-adquiere el monitor antes de lanzar en el wait; la carrera notify/interrupt la resuelve el GIL). Periféricos hechos: ThreadLocal (aislamiento por-hilo verificado), prioridad (rango [1,10]), daemon (regla de ciclo de vida). Falta: UncaughtExceptionHandler.
2 · Monitor intrínseco	 casi	Hecho: monitorenter / exit, synchronized (bloques y métodos), reentrancia, wait / notify / notifyAll, IllegalMonitorStateException, wait(timeout) y monitor GC-safe (las claves se remapean por el forward del GC en minor/compact, así que se puede compactar con monitores tomados). Falta: interrupción del wait (llega con H1), biased/thin locks, detección de deadlock.
1 · Substrato de ejecución	 gran avance	Hecho (E1+E2): green threads (default y visor) y hilos de SO reales (std::thread por Thread.start()); park / unpark reales. Hecho (E3 — GIL removido): en JVM_THREADS=os los opcodes frame-local (aritmética int/stack/ramas) corren lock-free en paralelo sobre un RunningCtx por-hilo con code cache; solo SharedVm se lockea; GC con safepoint stop-the-world cooperativo. El Arc<Mutex<JVM>> de toda-la-VM desapareció; os-gil queda como oráculo. Ensanchado: frame-local completo (W1), lecturas concurrentes bajo RwLock (W3), y asignación lock-free con Eden en un arena UnsafeCell Miri-verificado (W2). JMM (H4) hecho: modelo relajado — campos lock-free Relaxed, volatile Acquire / Release, Eden atómico. Falta: locks por-estructura finos, preempción.

Los dos cimientos reales

De todo el edificio, **dos primitivas** sostienen el resto: **park / unpark** (planta 1) y **CAS** (planta 5). De esas dos se deduce *casi todo* lo de arriba — AQS, los locks, los atómicos, los pools. **Las dos están**, y el salto grande —cambiar el substrato de green threads a hilos de SO— ya se dio: hoy conviven los tres (verde cooperativo · SO con GIL · SO en paralelismo real). Lo que falta arriba es *reconstruir sobre ellas* lo que hoy descansa en el monitor.

Ruta crítica



El camino canónico — el mismo que recorrieron CPython y las JVM tempranas — es **GIL primero** (hilos de SO con un lock global que serializa el intérprete: simple y correcto, pero todavía sin paralelismo real) y después **sacar el GIL** (locks finos por estructura + TLABs + GC stop-the-world). Recién ahí **volatile** y los *fences* dejan de ser decorativos: con green threads no hay reordenamientos reales que tapar; con hilos de SO, omitirlos rompe el código de formas no-deterministas.

Para el LLM, el paralelismo pesado **no** vive en este rascacielos: vive en **kernels off-heap** (rayon/CUDA). Esta torre es para la **corrección** de la concurrencia Java, no para el cómputo numérico — que sale del heap por completo.

Fase B — El compilador (javac, escrito en Rust)

El front-end convierte texto en estructuras y la back-end las convierte en bytecode:

```
.java → lexer (tokens) → parser (AST) → análisis semántico → generación de bytecode → .class
```

B0 Lexer **logrado** Base

Base Scanner con **maximal munch**; comentarios; literales `int / long / float / double / char / String` (hex/bin/octal, **text blocks** `"""..."""` — el lexema crudo; el destripado §3.10.6 lo hace el parser)

Base `TokenKind` espejo **fiel** del enum de javac (115 constantes; las keywords contextuales `var / record / sealed...` van como identificador)

✓ **Éxito: logrado** — tokeniza el juego **completo** de Java (las 115 `TokenKind` de JDK 25).

B1 Parser **logrado (lenguaje completo, sin bordes)** Base

Base Gramática → AST. **Auditado contra el §14**: están **todas** las sentencias —incluida la **clase local** (§14.3)— bloque, declaración local (múltiple, `final`, y con declarador al estilo C `int y[]`), `;`, etiquetada, expresión, `if`, `assert`, `switch` (dos puntos y flecha, con patterns y guardas), `while`, `do`, `for` (básico/multi-init/infinito) y `for-each`, `break` / `continue` **con y sin etiqueta**, `return`, `throw`, `synchronized`, `try` / `catch` / `finally` con multi-catch y recursos, y `yield`. Más los **inicializadores** `static { }` y `{ }`, el **inicializador de array** `{1,2,3}` (§10.6) y el `instanceof` con **pattern**

Base *Precedence climbing* + *backtracking* (declaración local vs. expresión, `cast` vs. paréntesis, **cierre de genéricos** — parte `>>` / `>>>` con *undo log*)

Base **Genéricos**: argumentos de tipo (`List<String>`), *wildcards* (`? extends` / `? super`), parámetros con cotas (`<T extends A & B>`), el **diamante** `<>`

Base **Visualizador de AST** (árbol ASCII) + volcado de tokens en el binario `javac ; y javac-step`: visor **paso a paso de las pasadas** (tokens → AST → tabla → chequeo)

Base  **Formas de expresión modernas** — cerradas esta iteración, todas parseadas y guardadas fielmente en el AST (compilarlas es de B2/B3, y hasta entonces la barrera del emisor las corta): **lambdas** (§15.27, con el desambiguador (`-cast-vs-paréntesis-vs-parámetros` por *lookahead* de la flecha), **referencias a método** (§15.13, incl. `T[]::new` por los corchetes vacíos), **clases anónimas** (§15.9.5, cuerpo de clase con nombre vacío), **clases locales** (§14.3) y el **type witness** (`Collections.<String>emptyList()`) —que además **se aplica**: fija los parámetros de tipo del método en vez de inferirlos, así un override deliberado incompatible ahora falla—

Base  **Text blocks** (§3.10.6) — el lexer toma el lexema crudo `"""..."""` y el parser **decodifica** el valor con el algoritmo del JLS **en orden**: (1) normaliza los terminadores de línea a `\n` (CRLF/CR), (2) descarta la **línea de apertura** (tras `"""` solo puede ir *white space* + un terminador), (3) quita la **indentación incidental** —el mínimo común de sangría de las líneas **no-blancas** y de la del **delimitador de cierre**— y los **blancos finales** de cada línea, y (4) **recién entonces** procesa los escapes, con los dos propios del text block: `\s` (un espacio que **sobrevive** al destripado de finales, porque este corre antes) y la **continuación de línea** `\<LF>` (una barra al final de línea se **traga** el salto). El resultado es un `String` común que el resto del pipeline emite como cualquier literal (`ldc`), verificado de punta a punta

Base  **Anotaciones** (§9.7) — antes se **descartaban** (`skip_annotation`); ahora `modifiers()` las lee junto a los modificadores y las **cuelga** de la declaración (clase, método, campo, parámetro). Se parsean enteras: marcador, valor único, pares con nombre, arreglos y anotaciones anidadas. Metadata, no bytecode: emitir el atributo `RuntimeVisibleAnnotations` es de B3. **Lección de la auditoría**: los inicializadores `static { }` / `{ }` también se *parseaban* y se *descartaban* —el código desaparecía sin que nada fallara—; hoy se conservan. *Parsear y tirar* es peor que no parsear, porque no deja rastro

Avanzado  **Bordes cerrados** — el parser ya no tiene huecos: las anotaciones sobre un **parámetro de tipo** (`<@Foo T>`) y sobre una **constante de enum** ahora se **retienen** (`TypeParam.annotations` / `EnumConstant.annotations`), las **anotaciones de tipo/uso** (§9.7.4, `List<NonNull String>`, `@Foo int`) se **aceptan** —se descartan por ser metadata ajena a la *erasure*— y el **type witness de constructor** (`new <T>Foo()`) ya **no da error**. Y se cerró la última: la anotación de **nivel de array** (`String @A []`, una por dimensión en `int @A [] @B []`) se **acepta y descarta** —tras la base, un `@` suelto solo puede ser de nivel de array—. **El parser queda sin bordes**

✓ **Éxito: logrado** — AST por *recursive descent* de clases/miembros, sentencias y expresiones con toda la precedencia; **genéricos, lambdas, referencias a método, clases anónimas y locales, type witness y anotaciones** incluidos.

B2 **Análisis semántico** **pasadas 1 y 2 + chequeos** **Base**

Base **Pasada 1 — Enter/MemberEnter**: tabla de símbolos completa — paquetes, tipos **anidados**, `enum` / `record` / `@interface`, **genéricos con cotas**; **class finder real** (lee `.classpath` del classpath, incl. el atributo `Signature` → jerarquía genérica del JDK); resolución + validación con errores **posicionados**; síntesis implícita; y el **grafo tipado persistido** como salida

Base **Pasada 2 — Attribute**: resuelve nombres + tipa **los cuerpos**, **decorando el AST in situ** (tipo por nodo, *binding* local/campo/método, slot de cada local con categoría-2, **variables de patrón** de un `case T v` con su slot, y el **constructor resuelto** de cada `new` para que el codegen sepa qué descriptor emitir) — la salida que consume el codegen

Avanzado **Overload resolution en 3 fases** (JLS §15.12.2: estricta → *boxing* → *varargs*), con *most specific* y detección de ambigüedad

Avanzado **Genéricos** (etapas A-B-C): subtipado con *containment* de wildcards (invariancia), *erasure*, sustitución del receptor (`List<String>.get` ⇒ `String`), `lub` / `glb`, e **inferencia** de los argumentos de tipo de una llamada (Cap. 18: reducción → incorporación → resolución)

Avanzado **Errores a nivel de expresión** — cada error lleva la línea:col del **nodo**, no del método (`int b = a + noSuchVar`; apunta a `noSuchVar`, no a la sentencia); documentado en `docs/pasada2-attribute.md` y `docs/semantica-jdk25.md`. **Indulgencia con externos acotada** — antes, un miembro no hallado en un tipo del classpath se aceptaba en silencio. La raíz eran dos: no se cargaba la **cadena de supertipos** (un método heredado, `s.hashCode()` de `Object`, podía no aparecer), y los nombres del `.class` venían con puntos donde el *finder* esperaba barras (la superclase no cargaba). Cerrados los dos: se cargan los supertipos **transitivamente**, y la indulgencia quedó **acotada** — un **nombre inexistente** (`s.noExiste()`, `ArrayList.noExiste()`) ahora se reporta si la jerarquía está **completa**; sigue indulgente solo la **sobrecarga que no matchea** (nuestra resolución no cubre toda firma del JDK) y las jerarquías **incompletas** (classpath del JDK ausente → red de seguridad). Con esto **B2 no tiene deudas de rigor conocidas**

Avanzado **Chequeos semánticos** (esta iteración, en la pasada **Check** y en **Attribute**): **control de acceso** (§6.6 — `private` / `protected` / paquete sobre cada uso), **excepciones chequeadas** (§11.2 — toda `checked` que se lance tiene que estar capturada o en el `throws`; requirió **retener la cláusula** `throws`, que el parser descartaba, y guardarla en la tabla), **this / super en contexto estático** (§8.4.3.2), **reasignar un campo final** fuera de constructor/inicializador (§8.3.1.2) y el **acceso a tipo anidado cualificado** (`Outer.Inner.v()`), §6.5.5 — el único que *rechazaba código válido*). El chequeo de acceso y el de excepciones corren sobre el árbol **fuentes**, antes del desugar, para no tropezar con el código sintético (el constructor del `enum` llama a `Enum.<init>`, que es `protected`)

Éxito: logrado (núcleo) — acepta un programa bien tipado y rechaza uno con error de tipos/nombres, con **overload resolution** y **genéricos** reales; semántica **completa** de JDK 25 como norte, citada a la JLS 25.

B3 Generación de bytecode **✓ núcleo + StackMapTable + invokedynamic** **Base**

Base **✓ Pipeline completo en** `compile()` — las cinco pasadas en el orden de javac: `Enter` → `Attribute` → `Flow` → `Desugar` → `(re-Attribute)` → `Codegen`.

Flow va antes que Desugar (sus errores apuntan al código fuente, no al bajado) y se **re-atribuye después** (las reescrituras dejan nodos sin decorar y el emisor necesita tipo/binding/slot). Un programa con errores semánticos **no emite** `.class`

Base **✓ Constant pool** con deduplicación + escritor de `.class` (v69) propio:

`Utf8 / Class / NameAndType / Methodref / Fieldref / Integer / String / Long / Double / Float`, con el hueco que exige la JVMs tras cada `Long / Double` (ocupan dos entradas); sección de **campos** y atributos `Code / SourceFile`

Base **✓ Tipos anchos y String** — `lconst / dconst / fconst, ldc2_w` para constantes de categoría 2, `ldc` para `String`, y la **promoción numérica binaria** (§5.6.2) con los 12 opcodes `x2y` (`longA + 1` inserta el `i2l`), con los desplazamientos como excepción (§15.19)

Base **✓ Objetos** — `new + dup + invokespecial <init>`, `getfield / putfield`, `getstatic / putstatic`, `invokevirtual` con receptor, `checkcast` y los estrechamientos `i2b / i2c / i2s`; **asignación** a local y a campo; y el `super()` implícito de todo constructor (§8.8.7), sin el cual el `this` queda sin inicializar y el verificador rechaza el `putfield`

Base **✓ Flujo de control** — etiquetas con parcheo de saltos; `if / else, while, do-while, for, break, continue`; comparaciones por categoría (`if_icmp, if_acmp, lcmp / fcmp1 / dcmp1 + if*`) y `&& / ||` compilados como **saltos** (cortocircuito real, no un booleano calculado)

Base **✓ Excepciones** — `try / catch` con su tabla, `throw`, y el `finally` **duplicado** en la salida normal y en un handler `catch-all` que re-lanza (§14.20.2): la v69 ya no acepta `jsr / ret`, así que duplicar es la única vía. (Esto reubicó el *inlining* del `finally`, que teníamos mal anotado como tarea del **desugar**: es del emisor)

Avanzado **✓ StackMapTable** (§4.7.4) — el frame de cada destino de salto, como el **merge** de los estados que llegan ahí (un slot que no coincide en todos los caminos pasa a `Top`). Siempre `full_frame`: legal en cualquier posición y evita las formas comprimidas. Los *handlers* llevan `stack = [E]` y hay que **forzarles** el frame, porque no los alcanza ningún salto. La pila de operandos se lleva **tipada**, no como simple altura: es lo que el frame tiene que declarar, y sin eso un destino alcanzado con algo ya empujado (`f(a, b > 0 ? 1 : 2)`) declaraba la pila vacía. Eso incluye el tipo `uninitialized` (tag 8), que identifica un objeto recién creado por el **offset de su** `new`; corrido su `<init>`, todas sus apariciones —pila y locales— pasan al tipo definitivo (§4.10.2.4). Validada con el *cross-check* del verificador propio

Base **✓ Barrera de seguridad** — `generate()` devuelve `Result`: una construcción no soportada **falla con posición** en vez de emitir un `.class` a medias. Destapó dos agujeros graves que el propio desugar venía alimentando en silencio — `instanceof` (que produce todo *pattern switch*) y la **creación de arrays** (que produce todo *varargs*)—, que a partir de ahí se cerraron. Si un `for-each / assert / yield` llega al emisor, el mensaje dice que es un **bug del desugar**, no una limitación

Base **✓ Arrays e instanceof** — `newarray / anewarray`, `arraylength`, las familias `Xaload / Xastore` (donde `byte / char / short` llevan **su propio** opcode aunque compartan la categoría `int`), inicializadores (`new int[] {4,5,6}`) y escritura de elementos. Con esto **compilan y corren** las llamadas *varargs* y los *pattern switch*, que el desugar ya venía generando

Base **✓ Ternario, switch, saltos etiquetados y synchronized** — el `?:` con sus dos ramas **promovidas al tipo del todo** (§15.25); el `switch` como `tableswitch` o `lookupswitch` según la **densidad** de las etiquetas (la heurística de javac), con el relleno de alineación y los desplazamientos de 4 bytes, preservando el *fall-through* de la forma de dos puntos y cortándolo en la de flecha; `break / continue` **con y sin etiqueta** sobre una pila de sentencias «de las que se puede salir» —un `switch` interpuesto captura un `break` pero **no un continue** (§14.16)— más el **bloque etiquetado**; y `synchronized` con `monitorexit` en **las dos** salidas, la normal y un handler `catch-all` que re-lanza, con el monitor copiado a un local por el desugar para no reevaluarlo. De yapa, el `switch` sobre `String` pasó a compilar de punta a punta: el desugar ya lo bajaba a dos switches sobre `int`, pero ninguno de los dos se podía emitir

Base **✓ Emisión multi-clase** — `generate()` emite un `.class` **por tipo** (top-level y anidados, recursivo), cada uno con **su** nombre (`Multi`, `Multi$Nested`, `Otra`, el `E$1` del `switch -enum`). Era la última fuga silenciosa: antes se escribía **una sola clase** con el nombre del archivo, y un segundo tipo se perdía sin aviso. De paso, el `switch` sobre `enum` **ahora corre** de punta a punta (su `$SwitchMap` vive en `E$1`, que recién ahora llega al disco)

Avanzado **✓ invokedynamic** (lambdas, *method refs* y *record*) — un nodo **Indy genérico** (bootstrap + argumentos estáticos **tipados**: `MethodType / MethodHandle / Class / String`) que el desugar produce y el emisor serializa sin recomputar. Las **lambdas** se bajan con un `LambdaToMethod` propio: método sintético `lambda$...` con las capturas de cabecera (análisis *effectively-final*, reusando el de las lambdas), `this` capturado cuando el cuerpo lo usa (impl de **instanciación**, `REF_invokeSpecial`; si no, estática, `REF_invokeStatic`), y el `invokedynamic` con `LambdaMetafactory.metafactory` de *bootstrap* (`MethodType` borrado/instanciado + el `MethodHandle` de la impl como argumentos estáticos). Las **referencias a método** cubren las **cuatro formas** del §15.13.1 (estático, instancia **ligado/no-ligado**, constructor) apuntando al método/constructor **real**, sin sintetizar nada. El `equals / hashCode / toString` de un `record` (§8.10.2) va por `ObjectMethods.bootstrap`, con la `Class`, la cadena de nombres de componentes y un `MethodHandle` `REF_getField` por componente. El pool ganó `MethodHandle / MethodType / InvokeDynamic` + el atributo `BootstrapMethods`, y el **verificador propio** el opcode `0xba`. La **ejecución** del call site vive en la JVM (KajijDK); acá el oráculo es el verificador estricto

Avanzado **✓ Clases internas, locales y anónimas** (§8.1.3/§14.3/§15.9.5) — captura del entorno **completa**, las tres formas **corren y verifican**. Una **interna de instancia** captura la instancia envolvente en un campo `final Outer this$0` (parámetro de cabecera + asignación en el constructor, `new Inner() → new Inner(this)`, y el acceso sin cualificar a un miembro del enclosing reescrito a `this$0.f / this$0.m()`, con la resolución por la cadena de `owner` sumada a `Attribute`). Una **local** (registrada por la pasada mutable `register_local_classes` —un *body-walk* que la renombra a única y le da binary `Outer$1L` —, tipada *in situ* con los locales del método visibles) captura además los locales *effectively-final* en campos `val$x` —parámetros de constructor que `new L(caps)` pasa— reusando el análisis de las lambdas; y combina `this$0 + val$` en método de instancia. Una **anónima** (`new Base(){...}` o `new Runnable(){...}`) se baja a una **local sintética** `Outer$N` en un pase `hoist_anonymous` entre `Enter` y `Attribute`: recorre el mismo camino que la local (registro, tipado, levantado, captura), **extiende una clase o implementa una interfaz** (según el `ty` resuelto) con override y captura `val$ / this$0` — las dos corren y verifican. **✓ Argumentos de super** (`new ArrayList(10){...}`): la anónima sintetiza en `$N` un constructor que los **reenvía** a `super(...)`. **✓ Outer.this** (§15.8.4): baja a la cadena de `this$0 (this.this$0...)`, desambiguando un campo **sombreado** (`Outer.this.f` vs. `this.f` van a clases distintas) y resolviendo el **anidamiento multinivel** subiendo por la cadena de `owner`. **✓ outer.new Inner()** (§15.9.2): el **calificador** se empuja como `this$0` en vez de `this` —válido incluso en contexto **estático**, donde no hay `this`—. **✓ Colisión de nombres** (§14.3): dos locales homónimas (`L` en dos bloques del mismo enclosing) ya no colisionan —antes la tabla clavea el tipo por nombre y la 2ª se **descartaba en silencio**—. Una pasada mutable **renombra** cada local a un nombre único por unidad (`L → 1L, 2L ...`, binarios `Outer$1L / Outer$2L`) y **reescribe sus referencias con alcance léxico por bloque** (visible de la declaración al fin del bloque), dejando al resto del pipeline ver nombres únicos en un scope plano. Destapó de paso un bug del desugar —`has_this` no

se restauraba al procesar el cuerpo de una local, y una segunda local en un método **estático** creía capturar `this$0` —, ya cerrado.  **Local-en-local** (§14.3): una local declarada dentro del método de otra local se registra como **tipo anidado suyo** (`Outer$1L1$2L2`) —la pasada de registro recorre el cuerpo de una local tomándola a ella como nuevo enclosing— y el desugar la baja recursivamente.  **Multinivel implícito** (§8.1.3): el acceso **sin cualificar** a un miembro de un enclosing lejano (una interna dos o más niveles adentro que lee un campo/llama un método del abuelo) encadena tantos `this$0` como niveles (`this.this$0.this$0.f`), construyendo la cadena de enclosings por `owner`. Con esto las clases internas quedan **sin colas**

Avanzado  **Emisión de interfaces propias** — el `.class` de una `interface` sale con `ACC_INTERFACE`, la lista de super-interfaces y sus métodos **abstractos sin Code** (§4.6); el emisor ya no sintetiza constructor para ella. Se sumó `invokeinterface` (con `InterfaceMethodref`, tag 11, y su `count`) para las llamadas sobre un receptor de interfaz —antes se emitía `invokevirtual`, mal, así que también arregló las llamadas a `List / Comparator` —, y el *method ref* a método de interfaz (`REF_invokeInterface`) pasó a `InterfaceMethodref`. Con esto **las anónimas sobre una interfaz** (`new Runnable(){...}`, el caso más común) corren y verifican, y una **lambda contra una interfaz funcional propia** ya tipa y emite

Avanzado  **Sueltos de emisión cerrados** — `RuntimeVisibleAnnotations` (§4.7.16): se emite el atributo en clase/campo/método para las anotaciones **retenidas en runtime** (los `@interface` del fuente con `@Retention(RUNTIME)` + las conocidas del JDK; `@Override` / `@SuppressWarnings`, que son `SOURCE`, no se emiten), con los `element_value` taguados (String/primitivo/clase/enum/anidada/arreglo). **Plegado de constantes** (§15.28/§15.29): una `static final int` como etiqueta de `case` —o aritmética constante `case 1 + 4`, o una constante **cualificada** `case K.A` de otra clase que se **inlinea**— se pliega a un literal (mapa `campo → valor` a punto fijo, para constantes que refieren a otras). **Formas comprimidas del `StackMapTable`** (§4.7.4): cada frame se serializa en la forma más compacta según el anterior (`same / same_locals_1 / append / chop`, con `full_frame` de respaldo) — puro ahorro de bytes, validado por el verificador

 **Éxito: logrado** — el `javac` propio compila objetos, bucles y excepciones, y el `.class` **corre en tu JVM** (`fib(10)=55`, `fact(5)=120`, `new M(10); m.add(5); m.get() → 15`) **y pasa el verificador JVMs-estricto**, que hace *cross-check* de nuestra `StackMapTable` contra su propia inferencia.

B4 **Compilador robusto** **núcleo cerrado** **Avanzado**

Avanzado *  **Overload resolution** (**hecho en B2**) y chequeo de override — **pasada** *Check propia* (el `Check.java` de javac: bien-formación de las declaraciones, no de los usos). Cubre las cuatro reglas de §8.4.8 sobre firmas override-equivalentes* (mismo nombre y mismos parámetros **borrados**, §8.4.2): no sobrescribir un `final`, no cruzar la frontera `static` / instancia, no **reducir** la visibilidad, y **retorno covariante** — idéntico para primitivos, subtipo para referencias—. Más §8.1.1.1: una clase concreta tiene que implementar todo lo `abstract` que hereda, con las jerarquías que tocan un tipo **externo exentas** (de esos solo conocemos los miembros que aparecieron en alguna firma, así que una ausencia no prueba nada).  **Y ahora** `@Override` sin override (§9.6.4.4) —se reporta un `@Override` que no sobrescribe ni implementa nada; el chequeo es **sound**: solo dispara cuando el único ancestro externo es `Object` (cuyo set de métodos conocemos) y el método no es uno de los suyos (`toString` / `equals` / ...), y con un supertipo del JDK se calla— y el **throws más ancho** (§8.4.8.3) —una excepción **chequeada** que lanza un override tiene que ser subtipo de alguna del método sobrescrito—

Avanzado  **Inferencia** desde los argumentos *y desde el target type* — la atribución pasó a tener modo checking (`attrib_expr_to`), no solo síntesis: el tipo que el contexto espera baja a la expresión y es lo único que puede instanciar una variable que **no aparece** en los argumentos (`Caja<String> c = fabricar();`). Vale también para el **diamante** (§15.9.3), que antes pasaba por indulgente — `new Caja<>()` daba un parametrizado con cero argumentos, que se compara con cualquier cosa sin comparar nada— y ahora infiere de verdad, del target y del constructor. La **incorporación** (§18.3) arbitra entre las dos fuentes: un target que contradice a los argumentos se descarta, para que el error lo reporte el chequeo de asignación y no una inferencia inventada. Contextos cubiertos: los tres de asignación (§5.2 — inicializador, `return`, `=`) y ahora el de **argumento de llamada** por el algoritmo de **dos fases** (§15.12.2.6): la fase 1 resuelve la sobrecarga con los tipos provisionales/lenientes de los argumentos, y la fase 2 —resuelta ya— re-atribuye cada argumento **poly** (lambda, method ref, diamante) con el tipo de su parámetro como target. Es lo único que tipa el cuerpo de una **lambda-argumento** (`call(x -> x + 1)`) —sin él quedaba **Unresolved** y el emisor no podía bajarla— e instancia un **diamante-argumento** (`take(new Box<>())` infiere `String`); el diamante sin target se deja **crudo** para que la aplicabilidad lo acepte unchecked.  **Y la desambiguación de sobrecargas por el argumento poly** (§15.12.2.5): un arg lambda se tipa **Unresolved** (indulgente), así que sin esto sería aplicable a **toda** sobrecarga con un parámetro funcional; se descartan las incompatibles con la **forma** de la lambda —aridad del SAM y valor-vs-void—, así `f(x -> x + 1)` elige `Function` sobre `Consumer` (antes daba ambiguo). Cola menor: la misma disambiguación para method refs* (por aridad)

Avanzado  **Análisis de flujo (Flow)** — **asignación definitiva** de locales (Cap. 16, con los flujos *when-true/when-false* de `&&` / `||` / `!`), las reglas de variables `final` (conjunto *definitely unassigned*: no reasignar, asignación única) y **alcanzabilidad** (§14.21: sentencias inalcanzables), con `break` / `continue` **etiquetados** (§14.15/§14.16: la pila de contextos lleva la etiqueta a la que salta cada `break`)

Avanzado  **Desugar** — sentencias (`for-each` → `for` / `while` + `Iterator`, `try`-recursos → `try/finally` + `close()` con la **excepción del `close()`**) **suprimida** vía `addSuppressed` (§14.20.3.1), `assert` → `if(!$assertionsDisabled && !cond) throw` con su campo sintético y el `<clinit>` que lo calcula de `C.class.desiredAssertionStatus()` (§14.10), `++` / `--` **en descarte** → `x += 1` (§15.14.2), **switch-expresión** en cola → switch-sentencia que asigna (§15.28, con `yield` desde bloques/bucles vía `break` **etiquetado** sobre el switch generado), **switch sobre `String`** → dos switches sobre `int` con `hashCode` + `equals` (§14.11), **switch sobre `enum`** → switch sobre un `$SwitchMap` sintético (`int[]` indexado por `ordinal()`), alojado en una clase anidada `C$1` y poblado en un `<clinit>` con `try/catch NoSuchElementException`, **fiel a javac** §14.11), * **switch** con patterns (*incl. deconstrucción de records, que extrae cada componente por su accessor y es recursiva*) → cadena de `instanceof` en un bloque **etiquetado** (§14.11.1: cada brazo es un **if suelto** —no `else if`— para que una **guarda que falla** caiga al **case** siguiente; el binding se materializa con un cast, y un selector `null` sin `case null` lanza **NPE**) y expresiones (concat de `String` → `StringBuilder`, `x op= y` → `x = (T)(x op y)`), **varargs** → `array` en el call site). Habilitado por una **síntesis a nivel clase nueva**: `Member::StaticInit` (los bloques `static { }` ya no se descartan), tabla de símbolos **mutable** en el desugar, el `enum extends Enum` implícito y el **binding de variables de patrón** en `Attribute/Flow`. Además **materializa los miembros implícitos de un `record`** (§8.10.3: campos, constructor canónico y accessors, respetando lo declarado a mano) —sus símbolos ya venían de `enter`, pero sin cuerpo en el AST el `.class` los referenciaba **sin declararlos**, y la deconstrucción llamaba a un método inexistente. La síntesis de estos miembros (enum/record/assert) reencuentra el símbolo del tipo por su **FQN con paquete** (§7.1) —el desugar lo armaba sin el paquete, así que un `enum` / `record` en un **paquete nombrado** perdía **toda** su maquinaria (sin constantes/ `values()` / `valueOf()` / `$VALUES`): bug #21, ya cerrado con test—. El inlining del `finally` resultó ser del **codegen**, no de acá. Queda como **asimetría** que varias de estas reescrituras producen construcciones que el emisor **todavía no soporta** (`instanceof`, `new int[]`): desde que hay barrera, eso **falla** en vez de salir roto. Aparte (no son desugar, van en su propia pasada): el **boxing** es `TransTypes*` ( ítem aparte) y las lambdas/ `invokedynamic` son `LambdaToMethod`

Avanzado  **TransTypes** — **boxing/unboxing dirigido por tipo** (§5.1.7/§5.1.8) — el `TransTypes` de javac, acotado al **boxing** (la *erasure* de genéricos ya la resuelve el sistema de tipos por *erased ids*). Dos capas: (1) **Attribute lo acepta** — el contexto de asignación (§5.2) permite `int` → `Integer` / `Number` / `Object` (box + *widening* de referencia) y `Integer` → `int` / `long` / `double` (unbox + *widening* primitivo), sumado a la fase de boxing del *overload*; (2) una **pasada propia** (tras Flow, antes del desugar) que recorre el árbol tipado e **inserta la conversión** en cada sitio: primitivo donde se espera referencia → `Integer.valueOf(v)`, *wrapper* donde se espera primitivo → `v.intValue()`. Contextos: inicializador de *local* / campo, `return`, `=`, **argumentos** (con el tipo de parámetro resuelto — un genérico borrado *boxea*), **operandos** de aritmética/relacionales/lógicos (`i + 1` → `i.intValue() + 1`), índice de array, ramas del ternario, y el **cuerpo de una lambda** contra el retorno de su SAM. Las inserciones quedan sin decorar y las **materializa la re-atribución**; el emisor saca de ahí `invokestatic valueOf` / `invokevirtual xxxValue`. Cierra el caso insignia de las *poly expressions*: `Function<Integer,Integer> f = x -> x + 1` de punta a punta (el `int` del cuerpo se *boxea* al `Integer` del SAM), validado por el **verificador estricto** — ejecutar `valueOf` / `intValue` es de la JVM (KajijDK), track aparte—. *  **Y el wrapper→primitivo más ancho** (`Long l = anInteger`): tras `intValue()` (`int`) se inserta un **Cast** al target, que el emisor baja a `i2L` / `i2d`. Cola menor: el `==` / `!=` con unboxing dirigido (se dejó por la identidad de referencia entre dos wrappers*)

Avanzado  **Miembros implícitos del `enum`** (§8.9.3) — constantes como campos, `$VALUES`, el constructor privado (`(String, int)` y `values()` / `valueOf()`). Destruirlo pidió cerrar antes la **invocación explícita de constructor** (`super(...)` / `this(...)`, §8.8.7.1), que **no existía**: el parser la producía, la atribución la rechazaba, y sin ella no se puede heredar de ninguna clase con constructor parametrizado. Dos desvíos deliberados de javac, ninguno semántico: `values()` copia con un **bucle** en vez de `$VALUES.clone()` (lo que importa de `clone()` es devolver un array fresco), y `valueOf` compara contra los nombres literales en vez de delegar en `Enum.valueOf(Class, String)`, que va por reflexión. Se sumó `java.lang.Enum` a `bootstrap/` + `boot/`: sin ella el `.class` emitido no se podía ni verificar ni correr.  **Y ahora el `enum` con constantes parametrizadas** (`ROJO("rojo")`, §8.9.2): a cada **constructor propio** se le antepone (`String $name, int $ordinal`) tanto en el AST como en el **Resolved del símbolo** (la re-atribución no re-resuelve firmas, y sin eso `new E("A", 0, ...)` no encontraría el ctor), y se le inyecta `super($name, $ordinal)` de cabecera —o se le **propaga** a un `this(...)` delegado—; cada constante se construye con sus argumentos (`new Color("ROJO", 0, "rojo")`), y el ctor (`(String, int)`) por defecto solo se sintetiza si el usuario no declaró uno. Corre de punta a punta (un `enum Size { SMALL(1), LARGE(3); ... }` da 4) y verifica

Avanzado  **Inicializadores de campo** (§8.6/§12.4.2) — `int v = 7;` se vuelve una sentencia, unificada en **orden de fuente** con los bloques `{ }/static { }:` las de instancia al frente de cada constructor, las estáticas al `<clinit>`. Hasta ahora **no llegaban al** `.class`: un campo con inicializador compilaba a un campo en cero, y el emisor ni siquiera generaba `<clinit>` —así que el `$VALUES` de un `enum`, el `$SwitchMap` de su `switch` y el guard del `assert` quedaban declarados y vacíos—

Avanzado  **Generación de `StackMapTable`** — hecha en B3 (ver ahí): la pila de operandos se lleva **tipada**, no como altura, y el tipo `uninitialized` (tag 8) identifica cada objeto por el offset de su `new`

✓ **Éxito: logrado** — compila programas con sobrecarga, herencia y flujo no trivial. Los siete frentes están cubiertos; quedan **colas menores** documentadas en cada ítem (`@Override` sin override y `throws` en el chequeo, el `target type` en posición de argumento, el `enum` con constantes parametrizadas) — ninguna bloquea el criterio.

B5 Cumbre del compilador núcleo cerrado **Cumbre**

Cumbre  **Genéricos** — `type erasure`, `wildcards` (containment), subtipado e inferencia por argumentos ya funcionan

Cumbre *  **Inferencia por target type*** — hecha en B4 (modo checking en la atribución; el diamante incluido). Las **poly expressions** ya se tipan y emiten (lambdas y method refs contra su interfaz funcional, con `invokedynamic`). El **boxing dirigido por tipo** (TransTypes), el `target type` en posición de **argumento de llamada** (dos fases §15.12.2.6) y la **capture conversion*** (§5.1.10) ya están **cerrados** (ítems aparte)

Cumbre  **sealed / non-sealed + exhaustividad de `switch`** (§8.1.1.2/§9.1.1.4/§8.1.6/§14.11.1.1) — el parser reconoce las keywords **restringidas** `sealed / non-sealed` (con `lookahead` que las distingue de un tipo/campo llamado `sealed`) y la cláusula `permits`; el grafo de símbolos gana la lista de **subtipos autorizados** (`permitted`), con el `permits` **implícito** (§8.1.6) rellenado de los subtipos directos de la unidad, así el resto del pipeline ve el conjunto completo. **Check** valida las reglas de buena formación: un subtipo directo de un `sealed` debe ser `final / sealed / non-sealed` (o implícitamente `final`: `enum / record`), un `permits` explícito solo puede listar subtipos **directos** y todo subtipo debe estar autorizado, y `non-sealed` exige un super `sealed`. Y la **exhaustividad**: una `switch`-**expresión** (siempre) y una `switch`-***sentencia con patterns** `case null sin default` **deben cubrir todos los valores — la jerarquía `sealed` recursiva hasta las hojas, todas las constantes de un `enum`, y `true / false` de un `boolean` (JEP 455) —, y un patrón con guarda (`when`) no cubre. El `.class` emitido lleva el atributo `PermittedSubclasses` (§4.7.31), re-parseado por la JVM propia: el tipo queda realmente sellado. **Destapó de paso un bug del parser** — `case RED -> 1` se leía como la lambda `RED -> 1`; una etiqueta constante es una conditional-expression, no una assignment-expression*, ya cerrado**

Cumbre *  **Capture conversion*** (§5.1.10) — la wildcard capture con **variables de captura frescas**, no una aproximación a cotas. Un `RType::Capture{id, upper, lower}` lleva sus cotas **inline** (sin tocar la tabla, que `Attribute` tiene inmutable) y un `id` fresco (contador con mutabilidad interior) que la hace **distinta** de otra captura igual —la freshness del §5.1.10—. La conversión (`capture`) transforma `G<?...>` en `G<...CAP_i...>`: `? / ? super B` → cota superior la del parámetro (`Object`); `? extends B` → `glb(B, U_i)`; y `? super B` guarda además la cota **inferior** `B`. Se aplica en **acceso a miembros** (el receptor se captura **una vez**, así todas las sustituciones de la llamada comparten las mismas variables) y en **argumentos de invocación**. El subtipado ganó las dos reglas: `CAP <: t` por su cota **superior**, y `s <: CAP` solo por la **inferior** (`? super`). Efectos observables: `List<?>.get()` → `Object`, `List<? extends Number>.get()` → `Number`, el **lado escritura** de `? super Integer` acepta un `Integer` pero **no** un `Object` (por campo, donde el chequeo es estricto), dos capturas del mismo `?` **no unifican** (`a.val = b.val` se rechaza), y el caso insignia `swap(List<?>) → swapHelper(List<T>)` ahora **infiere** `T` = la captura (para lo que se hizo aplicable un `<T> m(Box<T>)` a un argumento cuyo tipo lo fija la inferencia, no la invariancia). La erasure de una captura es la de su cota superior, así que el emisor y el verificador la ven como su upper bound*.  **Y el lado escritura por llamada** (cerrado en B6): la aplicabilidad **sustituye el receptor** en los params, así `list.add(x)` sobre `List<? super X>` también se chequea estricto —antes solo por campo—

Cumbre *  **Bridge methods (§8.4.8.3 / §15.12.4.5)** — los **métodos puente sintéticos que hacen que la sobrescritura funcione a nivel de bytecode** cuando la erasure difiere. El emisor, tras las clases reales, detecta por cada método propio concreto de instancia los métodos de un supertipo que **sobrescribe** —los params de aquél **sustituídos** (`T := Integer`) y borrados coinciden con los suyos (§8.4.2)— cuya erasure* **sin substituir** difiere, y sintetiza uno con el **descriptor borrado del supertipo** (`ACC_BRIDGE + ACC_SYNTHETIC`) que reenvía al real: `this` + los argumentos con un `checkcast` al tipo angosto + `invokevirtual` al método real + el `return` (el subtipo del retorno es assignable). Cubre las dos causas: el **parámetro genérico borrado** (`class INode extends Node<Integer>` con `set(Integer) → puente set(Object)`, y el clásico `Foo implements Comparable<Foo> → compareTo(Object)`) y el **retorno covariante** (`A.get():Object / B.get():String → puente Object get()`). Se dedup por descriptor y no pisa un método real ya presente. Verificado de punta a punta: el `.class` lleva el puente con sus flags, **verifica** (`checkcast + invokevirtual`), y el **despacho** funciona —`a.pick()` visto como `A.pick():Object` sobre un `B` corre el puente que reenvía a `B.pick():String`—. Acotado a **clases** (en una interfaz el puente sería un `default`, aparte)

Cumbre  **Módulos** (§7.7 / §4.7.25) — el sistema de módulos (JPMS) del lado del **compilador**: parseo de un `module-info.java` y emisión de su `module-info.class`. El parser reconoce las keywords **restringidas** (`module`, `open`, `requires`, `exports`, `opens`, `uses`, `provides`, `to`, `with`, `transitive` — todas identificadores salvo `static`) y arma la `[open] module nombre { ... }` con las **cinco directivas**: `requires [transitive] [static]`, `exports [to]`, `opens [to]`, `uses, provides ... with` (con la sutileza de `requires transitive`; donde `transitive` seguido de `;` es el **nombre** del módulo, no el modificador). El **emisor** produce un `module-info.class` fiel: `ACC_MODULE`, `this class = module-info`, sin super/campos/métodos, y el atributo `Module` (§4.7.25) con los flags (`ACC_OPEN / ACC_TRANSITIVE / ACC_STATIC_PHASE`) y el `requires java.base` **mandated** implícito; el pool ganó `CONSTANT_Module / CONSTANT_Package` (nombres de módulo con **puntos**, de paquete/servicio en forma **interna** con `/`). **Check** valida la buena formación (§7.7): directivas **duplicadas** (mismo módulo/paquete/servicio), implementación repetida en un `provides`, y auto-`requires`. Verificado re-parseando el `module-info.class` con la JVM propia (que ya leía los tags 19/20). La **resolución del grafo** de módulos —legibilidad, accesibilidad entre módulos— es de tiempo de ejecución (JPMS), track de la JVM aparte

✓ **Éxito: logrado** — los grandes frentes de **genéricos** y de **Java SE 25** están cerrados: erasure/wildcards/inferencia, captura, sealed+exhaustividad, bridge methods y módulos. La **equivalencia plena** con `javac` (emitir `Signature`, inferencia completa del Cap. 18) es cuestión de **fidelidad**, y se trackea en B6.

B6 Fidelidad de `javac` **esquinas cerradas + tooling + diferencial de emisión** **Cumbre****Cumbre**  **Atributos de class-file** — se emiten

Code / SourceFile / StackMapTable / BootstrapMethods / RuntimeVisibleAnnotations / RuntimeVisibleParameterAnnotations / AnnotationDefault / PermittedSubclass y  **Signature**.  **Signature** (§4.7.9) —el de mayor valor, ya **cerrado**: la firma **genérica** que la erasure borra del descriptor, emitida en **clase** (`<T:Ljava/lang/Object;>Ljava/lang/Object;`), **método** (`<T:Ljava/lang/Object;>()Ljava/lang/Object;`) y **campo** (`Ljava/util/List<Ljava/lang/String;>;`, `TT;`), con las cotas (interfaz tras un `:` extra), los wildcards (`+/-/`) y solo cuando el elemento **usa genéricos**; sin él una clase/método/campo genérico perdía sus genéricos en el `.class` y la reflexión + la compilación separada los veían borrados.  **InnerClasses** (§4.7.6) —también **cerrado**: cada `.class` lista su **cadena de enclosing** (él mismo si es anidado + sus ancestros) y las clases que **contiene**, clasificando **miembro** (con dueño y nombre) / **local** (`Outer$1L`: sin dueño, con nombre) / **anónima** (`Outer$1`: sin dueño ni nombre) por el sufijo del binary, con los flags de la declaración; es lo que la reflexión necesita para `getEnclosingClass` / `getDeclaringClass` / `isAnonymousClass`.  **EnclosingMethod** (§4.7.7) —el **complemento** de InnerClasses para local/anónimas, también **cerrado**: el desugar registra, al **levantar** la clase, el (nombre, descriptor) del método/constructor envolvente (o `None` en un inicializador → `method_index` 0), y el emisor pone `class_index` = la clase envolvente y `method_index` = ese `NameAndType`; con esto `getEnclosingMethod` / `getEnclosingConstructor` andan—.  **Record** (§4.7.30) —también **cerrado**, fiel: un **record** ahora extiende `java.lang.Record` (agregada a `boot/`, extraída del JDK), lleva `ACC_FINAL` y el atributo `Record` con sus componentes (nombre + descriptor, + `Signature` por componente genérico), lo que la reflexión exige para `isRecord` / `getRecordComponents`; de paso destapó y cerró dos bugs latentes —el parser no tomaba un **record genérico** (`record Box<T>(...)`) y el descriptor de una **variable de tipo** salía `LT;` en vez de la erasure `Ljava/lang/Object;` (no verificaba)—.  **NestHost** / **NestMembers** (§4.7.28/§4.7.29) —también **cerrado**: el nest (Java 11) que da acceso privado entre anidadas sin puentes sintéticos; cada clase **anidada** lleva `NestHost` a su **top-level**, y la top-level un `NestMembers` con **todas** sus anidadas (miembro/local/anónima, transitivo) —un nest **plano** por árbol de anidamiento—.  **MethodParameters** (§4.7.24) —también **cerrado**: los **nombr**es de los parámetros formales (+ `ACC_FINAL` si se declaró `final`, y `ACC_SYNTHETIC` para las capturas `this$0` / `val$x`), lo que hace que la reflexión (`Method.getParameters`) devuelva los nombres reales en vez de `arg0` / `arg1`; incluso un **record** conserva los nombres de sus componentes en el ctor canónico—.  **Exceptions** (§4.7.5) —también **cerrado**: las clases de la cláusula `throws` de cada método (los índices `Class` de las excepciones **chequeadas** que declara lanzar), lo que hace que `Method.getExceptionTypes` funcione y que la compilación separada vea las **checked exceptions**; va tanto en un método concreto (junto al `Code`) como en uno **abstracto/nativo** (sin `Code`); verificado diferencialmente contra el `javap` de Temurin (lee el atributo con las clases correctas y en orden).  **LineNumberTable** (§4.7.12) —también **cerrado**: anidado en el `Code`, mapea offset de bytecode → línea del fuente (el emisor marca la línea al entrar a cada sentencia, deduplicando por pc y por línea; el `super()` implícito de un ctor se mapea a la línea de su declaración), lo que da los **números de línea en los stack traces** y habilita breakpoints por línea en un debugger; verificado contra `javap -L` de Temurin.  **LocalVariableTable** (§4.7.13) —también **cerrado**: anidado en el `Code`, da los **nombr**es (+ descriptor + slot + rango de vida) de `this`, los parámetros y los locales del cuerpo, lo que hace que un debugger muestre los nombres reales. El emisor mantiene una **pila de scopes** que espeja la de la pasada 2 (que reusa slots al salir de un bloque): así dos locales que **comparten slot** en bloques disjuntos dan rangos **no solapados** (verificado contra `javap -L`, con `a/b` en el mismo slot y rangos disjuntos, igual que `javac`); los locales **muer**tos (rango de largo 0) se descartan, también como `javac`. Cubre método/`Block` / `for` / `switch` / `synchronized` / `try-catch-finally`; quedan afuera los pattern bindings (instanceof/ `switch`) —incompletos pero **nunca solapados**—.  **ConstantValue** (§4.7.2) —también **cerrado**: un **static final** con inicializador de **expresión constante** (§15.29) lleva el atributo (`Integer` / `Long` / `Float` / `Double` / `String`, coercionado al tipo del campo), que la JVM asigna al **preparar** la clase; el desugar lo deja **sin bajar** al `<<clinit>>` (no se inicializa dos veces, igual que `javac`) y así las constantes se pueden inlinear en compilación separada (§13.4.9); un evaluador acotado pliega literales con `+/ -` unario (las expresiones aritméticas complejas caen al `<<clinit>>`, correcto pero sin inlinear); verificado contra `javap` de Temurin.  **RuntimeVisibleTypeAnnotations** (§4.7.20, JSR 308) —el último atributo, **cerrado**:  **Fase 1** las anotaciones sobre **parámetros de tipo** (`class C<@Foo T>` target `0x00`, `<@Foo T> void m()` `0x01`);  **Fase 2** las anotaciones **líder** de declaración ruteadas por su `@Target` (`@NonNull String` sobre **campo** `0x13`, **retorno** `0x14`, **parámetro** `0x16`, con exclusión del `RuntimeVisibleAnnotations` para las `TYPE_USE`-only);  **Fase 3** los usos **anidados** con su `type_path` reconstruido por el parser —argumentos de tipo (`List<@A String>` → `[TYPE_ARGUMENT(0)]`), wildcards (`? extends @A T` → `[..., WILDCARD]`) y arrays con la **dirección** correcta del path (`int @A []` → path vacío en la dimensión externa, `List<@A X>[]` → `[ARRAY, TYPE_ARGUMENT(0)]`), `extends` / `implements` (`@x10`, `supertype_index` `0xFFFF` / índice de interfaz), **cotas** de parámetro de tipo (`@x11` clase / `@x12` método, con el `bound_index` que arranca en 1 cuando la primera cota es una interfaz —el `Object` implícito ocupa el 0—, como `javac`), `throws` (`@x17`) y los **targets de Code** —cast (`@x47`, offset del `checkcast` + `type_argument_index`), `instanceof` (`@x43`), `new` (`@x44`, objeto y array, con el offset del opcode / del inicio de la creación) y **variable local** (`@x40`, con la **tabla de rangos de vida** {start_pc, length, index}, el mismo rango que el `LocalVariableTable`, y filtradas por `@Target(TYPE_USE)`), estos cuatro en un `RuntimeVisibleTypeAnnotations` **anidado en el Code**. Para retener la anotación por posición de tipo el parser acumula un buffer `pending_type_annos` (el `Type` del AST no tiene slot) que las declaraciones/expresiones drenan con su `type_path`, y el emisor la rutea al target + offset correcto. **Todo verificado byte-fiel** contra el `javap` de Temurin (parámetros de tipo, cotas, extends/implements, throws, field/return/param, y los cuatro targets de `Code`). Con esto **RVTa queda cerrado** y no quedan atributos de fidelidad **reflexiva** pendientes —solo variantes menores de bajo valor (`RuntimeInvisible{,Type,Parameter}Annotations`, `LocalVariableTable`)—

Cumbre  **Inferencia completa (Cap. 18)** — cerrada, con **barrido diferencial** contra `javac` **real** como evidencia.  **Lado escritura de la captura por llamada** — **cerrado**: la aplicabilidad ahora **sustituye el receptor** en los parámetros (§15.12.2.2), así una variable de tipo de **clase** pasa a su argumento (incluida la variable de **captura** de un `? super B`), y `list.add(x)` sobre `List<? super X>` se chequea **estricto** (`add(X)` vale, `add(Object)` no); las variables de tipo de **método** las deja la sustitución como están, para que la inferencia las resuelva.  **lub con intersección** (§4.10.4) — **cerrado**: un `RType::Intersection(A & B)` cableado por el sistema (subtipado `A&B <: t` sii algún miembro; `s <: A&B` sii todos; `erasure` = primer miembro, la **clase**), y el `lub` se computa por **supertipos borrados comunes mínimos** (así `Comparable` no se pierde por invariancia) recuperando la forma parametrizada cuando hay uno solo; ahora `lub(Integer, Long) = Number & Comparable`, y el ternario/inferencia se puede usar como cualquiera de los dos. *  **Y el solver transitivo (§18.3/§18.4)** — cerrado: **se siembran las cotas declaradas de cada variable** (`<C extends List<T>> => C <: List<T>`), la incorporación **cierra los bounds a punto fijo cruzando** `S <: a` con `a <: U` (y reduciendo lo derivado, que puede fijar otra variable), y la resolución **instancia en orden de dependencia sustituyendo lo ya resuelto (un ciclo se fuerza junto)**. Así una variable sin cota directa se deduce **fluyendo por la de otra**: `<T, C extends List<T>> T first(C c)` sobre un `List<String>` infiere `T = String` (antes caía a `Object`). De paso destapó un bug de `enter`: las cotas de un **parámetro de tipo de método se resolvían en global** en vez del scope propio del método, así que un `<C extends List<T>>` no veía ni al hermano `T` ni a las clases del paquete.  **Y el containment de `? super X` (§18.2.3)** — el caso `Super` de `reduce_arg`, que se caía: `List<? super X>` como **parámetro** acota `X` por arriba (`sup(aListOfAnimal)` con `<T> T sup(List<? super T>)` infiere `T = Animal`).  **Y la detección de `false` (§18.5.1) en su caso claro** — dos igualdades incompatibles sobre una misma variable (`m(List<Foo>, List<Bar>)` sobre `<T> void m(List<T>, List<T>)`) hacen fallar la inferencia y se reporta el error (en métodos propios; sobre un externo se mantiene la indulgencia, que no modela toda firma del JDK).  **Y el argumento fuera de la cota declarada (§4.5.1/§18.3)**: `S <: a` con `S` fuera de la cota de `<a extends B>` (`m("x")` sobre `<T extends Number>`) ahora hace

fallar la inferencia —la aplicabilidad, indulgente con la variable de método, lo dejaba pasar—. Con esto el `false` de §18 queda cubierto en lo que importa. Las contradicciones cota-inferior/superior derivadas del target (`Integer n = id("hola")`), donde el contexto impone `T <: Integer` y el argumento `String <: T` se dejan a propósito al chequeo de asignación, que da mejor mensaje; reportarlas también en la inferencia las duplicaría.  Y las variables de inferencia frescas (§18.1) con solving combinado de anidadas (§18.5.2.1): cada invocación genérica introduce variables frescas (`RType::InferVar`, no el propio parámetro de tipo), y una llamada genérica anidada como argumento (`pick(empty(), strs())`) ya no se resuelve aislada —lo que daba `empty() → Box<Object>` y rechazaba la llamada—: sus variables entran en el mismo conjunto de constraints que las de la externa, así `empty()` toma su `E` del contexto (`E = String`) y el nodo se re-tipa.  Y la aplicabilidad por inferencia (§18.5.1) con desempate de sobrecargas (§18.5.4): un overload genérico cuya inferencia de argumentos es insatisfiable (`m(T, List<T>)` sobre `(List<String>, List<String>)`): el 2º arg fija `T = String`, el 1º pide `List<String> <: T` ya no es aplicable —lo detecta un `hard_false` sobre los bounds sin poder, gateado solo para desambiguar entre varios candidatos—, así que el desempate elige el mismo método que javac. Con esto, y completando la tabla de containment del §18.2.3, un barrido de 13 casos borde (desempate, capturas, wildcards, functional-interfaces, lub/glb) matchea a javac uno a uno.  Y las tres esquinas que faltaban —que se creían no-op por capturar en el sitio de uso, y resultaron ser tres divergencias observables reales, las tres de indulgencia (aceptábamos programas que javac rechaza, o rechazábamos uno que acepta)—, cerradas y verificadas contra Temurin 21: (a) §18.3 capture-in-bounds —un método que devuelve `G<? extends T>` toma su `T` del target (`Box<? extends Number> b = make() ⇒ T = Number`); antes no se miraba dentro del wildcard del retorno, `T` caía a `Object` y se rechazaba una asignación válida (nuevo `reduce_target_arg: ? extends inner` contra `? extends X ⇒ inner <: X, y ? super` análogo). (b) §18.4 —la variable de una llamada anidada que vive solo bajo un wildcard del retorno (`take(make())`) se resuelve por sus cotas (`⇒ su declarada Object`), no heredando el target externo; la clausura transitiva ya no le pasa cota superior a una variable de inferencia pelada, así `take(make())` con target `Number` falla como javac (antes lo aceptábamos). (c) §5.1.10 —pasar un argumento `G<? extends X>` a un parámetro invariante `G<α>` captura (`α = CAP` fresca); si el target además fija `α = T` concreto son igualdades incompatibles y la llamada se rechaza (`reduce_arg` gana las arms de captura contra param invariante, con cota concreta para que el conflicto sea proper). Un barrido diferencial de 9 casos anidados (capture/wildcard/nido) contra Temurin 21 matchea uno a uno. El único resto del formalismo —el second attempt literal del §18.4, la variable de tipo `Y` de rescate— es un no-op observable verificado: la capture conversion* (§5.1.10) en el sitio de uso ya produce la `Y` equivalente, así que no hay ningún programa que rescate (implementarlo pediría mantener bounds `G<α> = capture` en el conjunto, reescritura arquitectónica para cero cambio observable). La inferencia del Cap. 18 queda cerrada, con barrido diferencial contra javac real como evidencia —ya no por argumento de equivalencia—

  Diagnósticos y recuperación de errores (cerrado) —  recuperación del parser: en vez de abortar en el primer error de sintaxis, el parser entra en **panic-mode** y resincroniza —a nivel **unidad** (salta al comienzo de la próxima declaración de tipo, rastreando llaves), **miembro** y **sentencia** (salta al próximo `;/` del cuerpo)—, acumula **todos** los errores en un `Vec<Error>` y sigue construyendo el AST con las partes buenas; así una unidad con tres declaraciones rotas reporta los **tres** errores en una corrida y aún parsea los miembros/tipos válidos de alrededor. La **semántica** ya reportaba varios (Enter/Attribute/Check/Flow acumulan), y `--check` ahora lista los sintácticos primero y luego los semánticos.  **Frame de render tipo javac (cerrado)**: el CLI ya no imprime un escudo `línea:col: error: msg` sino el marco de javac — `archivo:línea: error: mensaje` (la columna se transmite con el **caret**, no en el texto), la **línea del fuente**, una línea con el `^` bajo la columna (preservando los tabs del prefijo para alinear), las **notas** indentadas, y un resumen `N error[s]` al final—; el tipo `Error` ganó un campo `notes: Vec<String>` que el render imprime.  **Mensajes ricos (cerrados los tres más comunes)**: el **cannot find symbol** lleva las sub-líneas `símbolo: / ubicación: (método fro(int) / clase String`, con las etiquetas alineadas en los cuatro sitios —método, campo, nombre pelado, *method ref*—; los **candidatos de sobrecarga descartados** (§15.12.2) se listan cuando el nombre existe pero ninguna sobrecarga aplica, cada uno con su firma (receptor sustituido) y el **motivo** (aridad distinta, o el primer argumento que no convierte, espejando `applicable`); y el **did-you-mean** sugiere el nombre más cercano en scope por **distancia de edición** (Levenshtein, umbral `long/3`) —método/campo del receptor, o locales + campos de la envolvente para un nombre pelado—.  **Nodos de error en el AST (cerrado)**: la recuperación a nivel **expresión**, no solo estructural. Una expresión que no parsea se vuelve un nodo `ExprKind::Error` y la **estructura de alrededor sobrevive** —cableado en el inicializador de un `local` (la variable **igual queda declarada**) y en los **argumentos** de una llamada (`f(bad, ok)` no pierde `ok`)—; el parser registra el error, sincroniza a un límite de expresión (`(, / ; / ) / ] / }` de profundidad 0) y sigue, y la atribución tipa el nodo `Unresolved` sin un error nuevo. Así `int x = @@; int z = x + 1;` reporta **solo** el error de sintaxis: `x` queda declarado y `x + 1` **no cascadea** «no se encuentra `x`». Con esto el ítem queda **cerrado**; el único resto es puramente cosmético (los **mensajes ricos** de javac que faltan —candidatos de constructor, notas de `unchecked`—, de bajo valor)

  **Esquinas de lenguaje (A/B/C/D)** — cerradas las cuatro, verificadas contra javac real: (A) evaluación **completa** de expresiones constantes (§15.28/§15.29) a `ConstantValue` —referencias entre `static final`, concatenación de `String` (§15.18.1) y condiciones booleanas de `dead-code` (§14.21); (B) **bridges** en **interfaz** (métodos `default` puente §9.4.1.3 — `invokeInterface` + `InterfaceMethodref`, con la vtable plegando los `default` de superinterfaces §5.4.3.3); (C) exhaustividad con **record patterns anidados** (JEP 440/441, reducción columna a columna); (D) asignación definitiva de un `switch` **exhaustivo sin default** (§16.2.9, el `default` implícito), ya integrada con Flow

  **Nivel herramienta** (el javac como herramienta, fuera del núcleo del lenguaje) —  `-Xlint` (§9.6.4.5): un pase de lint con **nueve** categorías (`empty`, `cast`, `fallthrough`, `deprecation`, `rawtypes`, `unchecked`, `finally`, `serial`, `this-escape`), con **severidad** en el diagnóstico, `@SuppressWarnings` (§9.6.4.6) y render estilo javac.  **APT — fase 1**: la API de `javax.annotation.processing` / `javax.lang.model`, y la resolución de **tipos anidados de clases externas** —leyendo el atributo `InnerClasses` — como `Diagnostic.Kind`. ☐ **Falta**: el `round loop` del procesamiento y `javadoc`

  **Diferencial de emisión** — un **arnés** (`tools/emitdiff`) que vuelve **medible** la fidelidad: compila cada `.java` con **nuestro** javac y con el **real**, desensambla ambos con el mismo `javap -p -c` y compara el resultado **normalizado** —sin los índices del constant pool (difieren por el orden de emisión), con las líneas **ordenadas** para que el orden de los miembros no genere ruido—. En la primera pasada destapó y **cerró** un lote de divergencias: el `default` de interfaz salía **sin** `ACC_PUBLIC` (§9.4); `x++ / x += c` no usaban `iinc`; las comparaciones contra 0 iban por `iconst_0`; `if_icmp...` en vez de `ifgt / ifle / ...`, y un `-literal` por `push`; `neg` en vez de `iconst_m1`; una clase **anidada** emitía `ACC_STATIC` en los `access_flags` de **clase** (bug de correctitud: la JVM rechaza ese `.class`, §4.1) y su ctor por defecto salía siempre `public` en vez de heredar el acceso (§8.8.9); un `enum` no emitía el `Signature` de su supertipo `Enum<E>` (§8.9) y su `values()` copiaba con un **bucle** en vez de `(E[]) $VALUES.clone()` —lo que exigió soportar `array.clone()` de punta a punta (§10.7)—; y una **switch-expresión** sobre `String` en cola no compilaba. Lo que queda son **desvíos deliberados** semánticamente equivalentes (`valueOf()` / `$values()`), el `finally` duplicado, la `switch-expresión` materializada en un temporal)

 **Éxito**: el `.class` emitido y los diagnósticos son **indistinguibles** de los de javac para el núcleo del lenguaje: se emiten **todos** los atributos, la inferencia cubre el Cap. 18, y los errores se **recuperan** en vez de cortar en el primero. Los cinco frentes grandes de Java SE 25 ya están (B5); esto cierra la brecha de **fidelidad** que queda.

Compilador — pasadas y estructuras internas

El compilador es **multi-pasada** sobre un **único AST**: se construye una vez (parser) y todas las pasadas lo recorren — la 1 lo lee para armar la tabla de símbolos, la 2 lo relee y lo **decora** con tipos y bindings. El objetivo es un compilador **completo** de Java SE 25; la semántica de cada regla está documentada y citada a la JLS 25 en [docs/semantica-jdk25.md](#).

```
.java → [lexer] → tokens → [parser] → AST
                                     |
                                     | (un solo árbol, reusado por las 5 pasadas)
                                     |
                                     | 1 Enter      → tabla de símbolos
                                     | 2 Attribute   → decora el AST (tipos + bindings)
                                     | 3 Flow        → asignación definitiva + alcanzabilidad
                                     | 4 Desugar     → baja la azúcar sintáctica
                                     | 5 Codegen     → bytecode → [write] → .class
```

Estado: ☒ hecho ☐ parcial ☐ falta

Pasada	Qué hace	Hito	Estado
Lexer	texto <code>.java</code> → tokens	B0	☒
Parser	tokens → AST (cada nodo con su posición de fuente)	B1	☒
1 · Enter	declaraciones → tabla + grafo tipado (class finder real, resolución)	B2	☒
2 · Attribute	resuelve nombres + tipa + decora el AST (overload, genéricos, inferencia)	B2	☒
Check	bien-formación de declaraciones (override §8.4.8, abstractos §8.1.1.1) + usos (acceso §6.6, excepciones chequeadas §11.2)	B4	☒
3 · Flow	asignación definitiva (+ <code>final</code>) + alcanzabilidad	B4	☒
4 · Desugar	baja azúcar (sentencias, concat, compuesta, varargs, <code>++ / --</code> , <code>switch-expr</code> → sentencia, <code>switch</code> sobre <code>String / enum / patterns</code> , guard del <code>assert</code> , records, miembros del <code>enum</code> , inicializadores de campo), LambdaToMethod (lambdas/method refs → <code>invokedynamic</code>), <code>record</code> por <code>ObjectMethods</code> , y la captura del entorno de las clases internas/locales/anónimas (<code>this\$0 / val\$</code> , con <code>hoist_anonymous</code>); el boxing va en <i>TransTypes</i> , su pasada aparte	B4	☒
5 · Codegen	AST decorado → bytecode → <code>.class</code> (v69): objetos, tipos anchos, flujo de control completo (<code>switch</code> , ternario, saltos etiquetados), excepciones, <code>synchronized</code> , <code>StackMapTable</code> e <code>invokedynamic</code> (lambdas, <i>method refs</i> , <code>record</code>)	B3	☒

Las pasadas no inventan código: **ordenan y anotan**. El orden lo imponen las dependencias — **Enter** antes que **Attribute** (por las referencias hacia adelante), Attribute antes que Flow y Desugar (necesitan tipos), Desugar antes que Codegen. La 1 lee "a lo ancho" (declaraciones), la 2 "a lo hondo" (cuerpos).

Estructuras intermedias

Además del AST, las pasadas comparten un **bolso de servicios globales** (la *Session*) y usan estado **transitorio** por recorrido. Dos que se olvidan siempre y duelen después: el **log de errores** (acumular, no frenar en el primero) y el **interner de nombres**.

Estructura	Rol	Vida
Tabla de símbolos	catálogo de todo lo declarado, con sus tipos	persistente
Arena de tipos (<i>interned</i>)	tipos resueltos, comparables por <code>TypeId</code> en O(1)	persistente
Interner de nombres	identificadores → <code>NameId</code> (comparación e identidad baratas)	persistente
Log de errores	acumula errores/warnings — no aborta al primero	persistente
Decoración del AST	cada nodo lleva su <code>tipo</code> y su <code>binding</code> in situ (salida de Attribute, estilo <code>JCTree.type / sym</code>)	persistente
Álgebra de tipos (<code>types</code>)	subtipado, erasure, sustitución, <code>lub / glb</code> — el <code>Types</code> de javac	persistente
Motor de inferencia (<code>infer</code>)	variables de inferencia + <i>bounds</i> + resolución (Cap. 18)	transitorio
Tabla de constantes	valores de expresiones constantes (JLS §15.29)	persistente
<i>Class finder</i>	carga los tipos externos (<code>.class</code> del classpath)	persistente
Env (contexto)	<i>scope</i> / clase / método / tipo esperado actuales	transitorio
Estado de <i>Flow</i>	<i>bitsets</i> de asignación definitiva y alcanzabilidad	transitorio
Estado de inferencia	variables de inferencia + restricciones (genéricos/lambdas)	transitorio

Representación: **arenas + IDs** (símbolos por `SymbolId`, tipos por `TypeId`, nodos por `NodeId`) en vez de punteros — la forma idiomática de Rust para grafos con ciclos, y coherente con el *metaspace* de la JVM, que ya referencia por `MethodId` / `Class ID`. La **tabla de símbolos** es un **árbol de scopes enlazados** que persiste (no una pila que se destruye al salir); el **Env** lleva solo el *scope* actual y sube/baja por esa cadena.

Fase C — Las bibliotecas (en Java, compiladas por tu javac)

C0 Núcleo de java.lang Base

Base `Object`, `String`, `System`, `wrappers`, `Math`, `StringBuilder`

✓ **Éxito:** un programa que usa `String` y `System.out` corre en tu JVM.

C1 Excepciones Base

Base Jerarquía `Throwable` / `Exception` / `RuntimeException`

✓ **Éxito:** lanzar y atrapar excepciones de la biblioteca propia.

C2 Colecciones e IO Avanzado

Avanzado `java.util`: `List`, `ArrayList`, `Map`, `HashMap`

Avanzado `java.io`: `InputStream` / `OutputStream` / `PrintStream`

✓ **Éxito:** un programa que usa `ArrayList` / `HashMap` corre.

C3 Cumbre de la biblioteca Cumbre

Cumbre Resto de `java.base` (`net`, `nio`, `time`, reflexión completa...)

✓ **Éxito:** cubrir lo necesario de `java.base` para programas reales.

Fase D — Herramientas (la “DK” = Development Kit)

No se construyen en bloque, sino que se van completando como subproducto de las otras fases.

Herramienta	Cuándo se logra	Horizonte
<code>javap</code> (desensamblador)	Se logra con el Hito A0	Base
<code>java</code> (lanzador de la JVM)	Se logra durante la Fase A	Base
<code>javac</code> (compilador)	Es toda la Fase B	Base
<code>jar</code> (empaquetador de <code>.class</code>)	Tras la Fase B	Avanzado
<code>javadoc</code> , <code>keytool</code>	Largo plazo	Cumbre

Fase E — Cerrar el círculo

El momento épico: las tres piezas funcionando juntas.

- **Cumbre** Compilar las bibliotecas (Fase C) con tu propio `javac` (Fase B)
- **Cumbre** Ejecutar un programa real que use esas bibliotecas en tu propia JVM (Fase A)
- **Cumbre** Conformance: comportamiento fiel a la especificación

✓ **Éxito:** un `.java` que escribís → lo compila tu `javac` → usa tus bibliotecas → corre en tu JVM, sin tocar nada del JDK de Temurin.

Más allá del JDK — la JVM como plataforma

La JVM no es un fin en sí mismo: es la **carrocería** de un proyecto más grande. Como construimos VM + compilador propios, controlamos el stack entero y podemos **inventar** más allá de lo que `javac`/HotSpot permiten. Estos dos tracks son extensiones aspiracionales apoyadas en las fases A–E.

Fase F — `burst`: el optimizador (rendimiento)

Módulo de optimización **opcional** atornillado sobre el intérprete ingenuo, que actúa de **chasis y oráculo de corrección**. `burst` debe dar resultados **idénticos** y se valida con **differential testing** (mismo programa por los dos caminos → `assert` de igualdad). Importante: un optimizador **no necesita JIT** — el intérprete tiene su propia caja de herramientas (típico 2–10× sin compilar a nativo).

Estado (2026-08-15): la caja de herramientas del intérprete se abrió primero y rindió lo prometido — **campos –86%, virtuales –45%, llamadas –40%** sin compilar nada, solo dejando de recomputar lo mismo—, y después **el JIT arrancó de verdad**: emisor x86-64 propio (sin dependencias), memoria W^X, OSR, deopt con reconstrucción de estado, alocaión en Eden y llamadas por inlining con *virtual frames*. Hoy compila **444 de 785 métodos del corpus (~57%)**, corre bucles **~120× más rápido** y acelera **55×** el patrón `new X(...)` en bucle con campos (17,3 millones de opcodes interpretados → 620). El *differential testing* que este módulo exige no hubo que construirlo: el oráculo `green=green-gil=green` del proyecto **ya lo es**, con el JIT activo en green y apagado en os-parallel — y la suite entera pasa idéntica con `JVM_JIT=0`. Detalle por hito abajo; la disciplina de medición que lo sostiene está en `F-bench`.

F-bench **Infraestructura de medición** **hecha** Avanzado

Avanzado **Workloads** (`java/Bm*.java`): `BmLoop` (aritmética frame-local = el piso), `BmArray`, `BmInvoke` (`invokestatic`), `BmVirtual` (`invokevirtual` polimórfico), `BmField` (`new` + `get/putfield`). Valores esperados contrastados contra el `java` real del JDK 25, y un test **no-ignorado** los verifica para que los benchmarks no se pudran en silencio

Avanzado **Protocolo, aprendizaje a los golpes**: el ruido de *code layout* de esta máquina es **±3-12%, mayor que el efecto de casi cualquier cambio**. Demostrado con un experimento nulo: agregar un campo `HashMap` **sin usar** movió un control +3.4%. Por eso: workloads de **control de efecto-cero** (verificados *contando* la operación, no razonando), **cuadrado latino** con los binarios pre-compilados, medianas y mínimos, y la regla "si la dispersión intra-rama es mayor que el efecto que declarás, todavía no lo mediste"

Avanzado **Medir antes de optimizar** — dos cuellos "obvios" resultaron falsos: el GC parecía el culpable de que `BmField` fuera 8.6× el piso (era **≤5%**), y el scan lineal `O(#clases)` de `class_name_at_mirror` parecía la gran palanca de `BmVirtual` (valía ~3%: hay **5 mirrors**, no cientos)

✓ **Éxito: hecho** — sin números honestos antes/después, todo el track es fe. `bench_baseline` (`#[ignore]`, `cargo test --release --lib bench_baseline -- --ignored --nocapture`) mide 5 workloads que aíslan una dimensión cada uno; los conteos de opcodes son exactos y reproducibles, así que **ns/opcode** compara entre workloads aunque los tiempos absolutos tengan ruido.

F0 **Quickening** **hecho** Avanzado

Avanzado **El diagnóstico**: las tres optimizaciones tenían la **misma forma** — un *cache keyeado* por `&str` alcanzado *alocando* un `String`, *recomputado* por *ejecución en vez de por call site*. Y esas alocaiones no eran necesidad semántica sino **impuesto del borrow checker** (`class_of()` / `name()` ya devuelven `&str`; se clonaban solo para poder llamar métodos `&mut self` después). Por eso se pudo sacar el 86% sin tocar semántica: no se hizo el trabajo más rápido, **se dejó de hacerlo dos veces**

Avanzado **Campos** (`field_sites`): `field_offset` **reconstruía el layout entero de la clase** — un `Vec<(String,String)>` fresco, recorriendo la cadena de superclases — **en cada acceso**. Cache por (`MethodId`, `pc`): una celda `AtomicU64` por byte de código, sin hash ni strings. De paso cerró un peligro real: `layout_fields_ref` truncaba en silencio ante una superclase no cargada, devolviendo offsets **demasiado chicos**; sin cache eso solo causaba una escalación, **con** cache habría inmortalizado un offset incorrecto — ahora solo se cachea si la cadena se recorrió entera

Avanzado **Llamadas** (`call_sites`): callee resuelto, `vtable_slot` del tipo estático, anchos de slots (antes un `Vec` fresco por llamada), y la cadena de **~7 comparaciones de string** contra nombres de intrínsecos reemplazada por un enum `Intrinsic` en el `MethodBody` del callee — no en el sitio — porque por `vtable` el sitio no puede saber la respuesta: depende del receptor. El bit de "ya inicializada" solo se pone con la clase en `Done`, así que `Erroneous` sigue tirando `NoClassDefFoundError` en cada uso

Avanzado **Pool de frames**: reusar `Frame`s (y la capacidad de sus `Vec`) en vez de dropearlos → **~11/12% más en llamadas**, ~72 ns/llamada recuperados. El riesgo era de GC — un frame retirado retiene referencias stale — y se blindó con **una sola puerta de entrada** (`recycle`, que hace `scrub` incondicional), `debug_assert!(is_scrubbed())` en cada reuso **con la suite entera corrida en modo debug**, y la verificación de que el pool no está en ningún camino de raíces del colector

✓ **Éxito: logrado, y con creces**: un método que invoca en bucle ya no re-resuelve nada — pero la medición mostró que el mismo patrón estaba en **tres** opcodes, no solo en los invokes. Resultado: `getfield` / `putfield` **–86%**, `invokevirtual` **–45%**, `invokestatic` **–40%** (acumulando el pool de frames), sin cambiar una sola regla de la spec.

F1 **Superinstrucciones / fusión** **salteado a propósito** Avanzado

Avanzado Queda disponible si en algún momento el piso de despacho vuelve a ser el cuello y hay instrumentación capaz de resolverlo

✓ **Éxito: decisión, no omisión**: fusionar `iload,iload,iadd` atacaría el piso de despacho (~28-30 ns/opcode) con una mejora estimada de 10-20% — **por debajo del ruido de layout de esta máquina (±3-12%)**. O sea: no podríamos *demostrar* que funciona. El esfuerzo se redirigió a F3, donde el efecto (2 órdenes de magnitud) es medible muy por encima del ruido.

F2 **Inline caching + stack caching** ☐ **pendiente** **Cumbre**

Cumbre **Inline cache monomórfico** (receptor → método). F0 ya dejó el lugar preparado: cachea el `vtable_slot` del tipo estático, y lo único que queda por llamada es `vtable_method(runtime_class, slot)` — que depende del receptor. `SiteKind::Vtable` está formado para alojarlo

Cumbre **Stack caching**: tope de la pila de operandos en registro. En el JIT es lo mismo que asignar registros a la pila de operandos (hoy cada operación Java es load/op/store contra L1)

✓ **Éxito**: dispatch dinámico acelerado en llamadas repetidas.

F3 JIT (bytecode → nativo) primer tier corriendo Cumbre

Cumbre **Emisor x86-64 propio** (`src/burst/x64.rs`), sin dependencias — las funciones de Windows se declaran a mano con `extern "system"`, igual que en su día se escribió el cursor de bytes en vez de usar `nom`. `ALU`, `idiv`, `jcc` con labels y parcheo de desplazamientos, ABI **Microsoft x64** (args en RCX/RDX/R8/R9, 32 bytes de shadow space, RSP alineado a 16). Verificado **ejecutando**: bucles, factorial, y un test de código generado **llamando a código generado**, que cubre de una sola vez las tres reglas del ABI que solo muerden en un `call`

Cumbre **Memoria W^X como garantía del tipo** (`exec_mem.rs`): `CodeBuf` (escribible) → `.make_executable()` `consume self` → `ExecMem` (ejecutable). Después de la transición no queda ningún handle escribible — no es que "recordamos no escribir", es que el tipo lo hace imposible

Cumbre **Compilador** (`compile.rs`) del subconjunto **frame-local de enteros** — el mismo que ya corría lock-free (W1), o sea sin heap, sin llamadas, sin referencias: un método así es una **función pura de sus locales**. Cualquier otro opcode ⇒ no se compila. La profundidad de la pila se recalcula **del CFG**, sin confiar en el `StackMapTable`: un desacuerdo entre ramas se rechaza en vez de generar código sobre una suposición

Cumbre **Las tres trampas de semántica**, con prueba: (1) *normalización* — todo `int` en registro es la sign-extensión de sus 32 bits; se emite `movsxd` tras la aritmética pero **no** tras `iand/ior/ixor`, con la demostración de por qué (los 33 bits altos de un sign-extendido son copias del bit 31); (2) *shifts* — Java enmascara a 5 bits, x86 a 6 en ops de 64, y `iushr` es lógico sobre 32 (`-1>>>1` debe dar `MAX_VALUE`); (3) *división* — `INT_MIN/-1` trunca como manda el JLS §15.17.2 en vez de fallar con `#DE`. Los 11 operadores binarios contrastados contra un modelo en Rust sobre **361 pares de valores de borde**

Cumbre **Deopt por pureza**: como el subconjunto no tiene efectos secundarios, ante cualquier situación que el nativo no pueda manejar (divisor cero) el código **abandona y el intérprete ejecuta el método desde cero** — re-ejecutar es indistinguible. Eso evita emitir manejo de excepciones en nativo, que es donde estos proyectos se vuelven inmanejables

Cumbre **OSR + poll de safepoint**, ambos restringidos a back-edges con **pila de operandos vacía** (un `goto` de retorno de `while` / `for` la tiene vacía → cubre los bucles reales), con lo que el estado a transferir se reduce a *locales + pc* en las dos direcciones. Sin OSR, un método con un bucle largo entrado **una sola vez** nunca calentaba: es exactamente el caso de `BmLoop`. El poll **sale hacia el intérprete** en vez de manejar el GC desde el nativo

Cumbre **El differential testing sale gratis**: con el JIT activo en `green/os-gil` y apagado en `os-parallel`, el oráculo `green==os-gil==os` que el proyecto usa hace meses **se convierte en la comparación JIT-vs-intérprete** sobre el corpus entero. La suite pasa igual con `JVM_JIT=0` y sin él. Además, un `static` estable para la palabra de poll: el `gc_pending` del driver es un `Arc` construido **fresco en cada corrida** (e inexistente en green), así que hornear su dirección habría sido corrupción silenciosa

Cumbre  **Subconjunto ensanchado** — prefijo `wide` (hoy un `+= 256` ya no descalifica un método), los siete opcodes de pila que faltaban (`dup_x1` ... `dup2_x2`, `pop2`, `nop` — que en su mayoría **no emiten código**: la pila se simula en compilación y son permutaciones de slots), `tableswitch` / `lookupswitch` (cadena de comparaciones; con el padding de alineación a 4 bytes, el `default` como target real, y los arms que saltan hacia atrás registrados como cabeceras de bucle para OSR), y `getstatic de int`. Guarda linda: `wide iinc` mide **6 bytes**, y un decodificador que lo tratara como 4 se re-sincronizaría sobre el byte bajo del delta — que en `0x0102` es `0x02`, un `iconst_m1` perfectamente decodificable, o sea corrupción que sigue "ejecutando" — así que hay un test clavado para eso

Cumbre **Dos rechazos razonados, que valen tanto como lo agregado**. `putstatic`: la regla "compilar solo si el método no tiene sitios de deopt-por-reinicio" era sólida, pero aceptarla convertiría el invariante "*el código compilado no escribe nada observable*" en "*...salvo cuando no puede deoptar*", y **cada guarda futura** tendría que re-derivar su interacción con esa excepción; el censo dice que bloquea 6 de 682 métodos, así que se paga poco por mantenerlo limpio. `long`: **no es esfuerzo, es un límite estructural** — `lreturn` no entra en el protocolo de retorno (`RAX = (status<<32) | value` deja 32 bits, suficientes para un `int`, imposibles para un `long`), así que exige mover la frontera misma y con ella el marshalling, el write-back de OSR y cada `unsafe` que la cruza

Cumbre **El techo, medido** (censo `subset_census` sobre el corpus): **77 de 710 métodos** compilan — 31% del techo de 246, ya que `ireturn` es la única salida posible. Los bloqueantes, en orden: `aload_0` **354** (todo método de instancia y constructor: `this` es una referencia), `new` 96, `invokestatic` 41, `statics no-int` 26, `ldc2_w` 18. Nota: los 18 métodos `void` que aparecen bloqueados **no valen nada** — un método puro y frame-local que devuelve `void` no puede tener ningún efecto observable, es literalmente un no-op. O sea que ensanchar más el subconjunto de *opcodes* ya tiene rendimiento decreciente: **lo que sigue son referencias y llamadas**

Cumbre  **Referencias** — el premio: `aload_0` **desapareció** de la lista de bloqueantes. Entraron `aconst_null`, `aload / astore (+ wide)`, `ifnull / ifnonnull`, `if_acmp`, `getfield de int`, `arrayLength`, `iaload` y `areturn` (que sube el techo del censo, porque `ireturn` dejó de ser la única salida). Y **no hicieron falta stack maps para el colector**: como el nativo no aloca, corre solo en `green/os-gil` y en los polls sale*, **ningún GC puede observar un frame compilado** — el objeto graph está congelado mientras hay nativo en la pila. Lo único necesario es un **mapa de tipos** (int/ref por local y por posición de pila, por interpretación abstracta sobre el mismo punto fijo que ya calculaba la profundidad) en la frontera de marshalling

Cumbre **Dos hallazgos que el diseño no anticipaba**. (1) **El heap son DOS buffers**: Eden vive en un `AtomicRegion` aparte y survivors/Old en `memory`, así que `base + offset` es una función de **dos ramas** — asumir un solo buffer habría hecho que todo acceso a un objeto en Eden leyera memoria ajena. (2) **Los exception handlers son aristas que el recorrido hacia adelante no ve**, así que el intérprete puede llegar a un loop header habiendo ejecutado código que el mapa nunca analizó (alcanzable de verdad: un deopt lo manda a re-ejecutar, y esa pasada puede lanzar y ser atrapada) — por eso un método con tabla de excepciones compila pero **no recibe entradas por OSR ni sitios de poll**

Cumbre **El test del peor modo de falla, verificado como detector**. Devolver una referencia como `Value::Int` en el write-back haría que el GC tratara un entero como puntero (o al revés) y no fallaría donde está el bug. No alcanzó con testearlo: se **rompió el código a propósito** para confirmar que el test efectivamente falla, y recién después se restauró. Un test que nunca viste fallar no es un test. En la misma línea, los tests machine-level usan un `FakeHeap` con **dos buffers**, porque uno solo dejaría pasar a un compilador que ignorara la partición de Eden

Cumbre **Censo: 77 → 96 métodos** (32% de un techo que subió de 246 a **299** gracias a `areturn`). `BmVirtual` dejó de ser un control —sus overrides son `aload_0`; `getfield`; `ireturn` — aunque gana solo 1.15×: los cuerpos son diminutos y domina el cruce de frontera. Rechazado a propósito: los campos `volatile`, aunque una lectura volátil es un `mov` común en x86-64 — apoyarse en el modelo de memoria de una arquitectura dentro de código generado merece su propio paso

Cumbre  **Deopt de verdad** — el deopt dejó de ser *por reinicio* (abandonar y re-ejecutar el método, válido solo mientras todo fuera puro) y pasa a **reconstruir el estado del intérprete en un pc**: locales, **pila de operandos** y posición. La pieza que faltaba ya existía: el mapa de tipos del paso anterior sabe, en cada pc, cuántos operandos hay y de qué tipo. Con eso entraron las **escrituras** (`putfield`, `iastore`, `putstatic de int`). Efecto lateral elegante: polls de safepoint y deopts **se unificaron** en una sola tabla de `resume_sites` — un header de bucle resultó ser un sitio de resume con pila vacía

Cumbre **La regla que hace correctas las escrituras:** *toda guarda de deopt se emite **antes** del primer efecto observable de su instrucción, y nada después del efecto puede deoptar.* O sea: cada instrucción tiene una fase de guardas (sin efectos) y una de efectos (sin deopts). De ahí sale que el pc devuelto siempre nombra una instrucción cuyo efecto **no** se aplicó, y el par «intento nativo + resume interpretado» escribe **exactamente una vez**. Validado rompiéndolo: reponer el viejo reinicio da 832497 en vez de 832289 — 208 de diferencia, que son las escrituras duplicadas

Cumbre **✓ Alocación (new) sin romper el invariante** — alocar puede disparar un GC, y todo el diseño depende de que eso no pase con nativo en la pila. Solución de los JIT reales: emitir **solo el camino rápido** (bump de Eden con `lock xadd` sobre *la misma palabra* que usa el intérprete, así una reserva nativa nunca se solapa con una suya) y **deoptar** cuando no entra — si el bump entró, no hubo colección. El bookkeeping del GC (el índice de objetos jóvenes, que es un `Mutex`) resultó **diferible**: se drena en un único lugar de toda la VM, la entrada al GC, así que la ventana entre el bump y el registro no contiene ninguna colección *por construcción*; el código nativo anota `(offset, size)` en un array plano y el trampolín lo replica al volver

Cumbre **✓ Llamadas por inlining** — el último bloqueante: `invokespecial` refusaba 379 métodos. Se eligió inlining sobre llamada nativa (sin ABI, sin unwind info); el costo es el deopt, resuelto con **virtual frames**. La clave fue que **el intérprete ya tenía la convención necesaria**: en un invoke el pc del caller queda *apuntando al invoke* y es el `return` del callee el que lo avanza — así que un deopt desde código inlineado se reconstruye como «caller en el pc del invoke con los argumentos ya popeados + callee en el pc del deopt», y lo termina la maquinaria existente **sin ningún caso especial**. La recursión se corta por **identidad de método**, no por profundidad (atrapa `f→g→f`, que ningún límite de profundidad reconocería), y un callee con rama hacia atrás se rechaza de plano: solo los headers de la raíz tienen poll, así que un bucle inlineado sería código del que no se puede salir

Cumbre **✓ Merge del mapa de tipos** — un slot que llegaba `Int` por una arista y `Reference` por otra (la forma corriente de javac cuando el slot está *muerto* ahí) descalificaba el método entero. Ahora el retículo tiene un elemento de **conflicto**: no descalifica, y lo que prueba que el slot está muerto **no es una suposición sobre javac sino el propio código** — toda lectura pide `Int` o `Reference`, así que un slot en conflicto **falla toda lectura**; el rechazo es la prueba. En un resume ese slot simplemente **no se escribe de vuelta**: el frame conserva su `Value` anterior, que es seguro para el GC porque no puede producir ninguna de las dos corrupciones (un offset marcado `Int`, un int marcado `Reference`). Costo documentado: una referencia vieja puede quedar viva de más — una fuga menor, nunca corrupción. Y donde no podía *probar* la deadness declinó asumirla: un cuerpo con exception handlers no recibe conflictos, porque las aristas hacia un handler son justamente las que el recorrido no sigue

Cumbre **Con eso `BmField` compila: 98.7 ms → 1.8 ms = 55×** — el patrón `new X(...)` en bucle con campos, o sea código Java completamente corriente, y el último de los cinco workloads que el JIT nunca había podido tocar. Se promovió `BmArray` a control pinneado para que el harness conserve dos brazos de efecto-cero contados. En el mismo paso, el riesgo que el anterior había dejado anotado —un `resume_state` que devolviera `None` tras haber corrido código nativo **reiniciaría el método en silencio**, re-aplicando escrituras— pasó de *mitigado con un `debug_assert`* a **estructuralmente imposible**: una compilación cuyos sitios de resume no sean reconstruibles **no se instala**

Cumbre **Censo: 21% → ~57% (152 → 444 de 785 métodos)**, con los rechazos por `invokespecial` cayendo de **379 a 3**. Rendimiento sobre bucles: **~120×** (`BmLoop` 637 ms → 5.3 ms), y ~98× / ~70× en workloads de array y de campos. La disciplina de *romper el código a propósito para verificar que el test detecta* se volvió rutina: en el paso de inlining se sabotearon cinco cosas distintas y **el primer juego de tests no atrapaba ninguna** — dos tests nuevos y uno reformado existen exactamente por eso

Cumbre **□ Pendiente:** el techo ya no son las llamadas sino el **merge del mapa de tipos** — un slot que llega `int` por una arista y referencia por otra (la forma habitual de javac cuando un slot está muerto) descalifica el método; es lo que todavía deja afuera a `BmField`. Después: **registros** para la pila de operandos; y habilitar el JIT en `os-parallel`, deliberadamente apagado porque es el único modo con el heisenbug abierto y no se le agrega una variable a un problema con ~17 sesiones de acotamiento

✓ Éxito: la cumbre, alcanzada en su primer tier: hay un compilador propio que emite x86-64 y lo ejecuta. `BmLoop` pasó de **475 ms a 4.1 ms — 116×** (medido de nuevo tras ensanchar el subconjunto: **125×**) — y el número que lo explica no es el tiempo sino el conteo de opcodes: **17.325.011 → 620**. El bucle dejó de existir para el intérprete. Los cuatro workloads de control quedaron planos.

Fase G — `plain_data` / value types (modelo de datos)

Los “**huérfanos de Object**”: tipos planos sin header, sin identidad, sin monitor, flatteables. Viable porque tenemos compilador propio (Fase B) que puede emitir el constructo y una VM que lo trata especial. Es, en chiquito, el principio de representación de un **tensor** — puente directo a sistemas de ML. Inspiración: value classes de Valhalla, `struct` de .NET.

G0 `plain_data` plano en el heap Cumbre

Cumbre Tipo sin header: `[field0 | field1]` vs objeto normal `[class_ptr | fields]`

Cumbre El compilador (Fase B) lo marca; la VM lo guarda inline

✓ **Éxito:** un value type sin header conviviendo con los objetos normales.

G1 Arrays flatteados + semántica de valor Cumbre

Cumbre `plain_data[]` contiguo: sin los N punteros, ni los N headers, ni N alocações

Cumbre Igualdad por valor (comparar bytes), sin `null`, sin GC para ellos

✓ **Éxito:** comparar (medible) memoria/layout de `plain_data[]` vs un array de objetos.

G2 Escape analysis lite Cumbre

Cumbre Versión declarativa/estática (el automático completo es del JIT, Hito F3)

✓ **Éxito:** aplanar en *load time* cuando se prueba que un objeto no escapa.

Fase H — KajiLibrary (la biblioteca estándar, en Java, dogfooded)

La versión **concreta y accionable** de la Fase C: la biblioteca escrita a mano en `KajiLibrary/` (nuevo directorio en la raíz), **compilada por el `javac` propio** (dogfooding — cada clase que no compile es un ítem concreto de B6) y **ejecutada en KajiJDK**. Clasificada por *tiers* según su dependencia: ● Java puro (autocontenido, se compila y corre sin depender del runtime) · 🐞 necesita **intrínsecos del VM** (avanza con el track KajiJDK). El objetivo es la **Fase E** (cerrar el círculo): tu `.java` → tu `javac` → tus bibliotecas → tu JVM.

Escala hoy: 768 clases públicas (476 de `java.*` + 292 de `jakarta.*`), todas pasando el gate contra el JDK 25 y los jars de referencia — y ese «25» dejó de ser una suposición: hasta hace poco `api_shape.py` invocaba `javap` a secas y tomaba el del `PATH`, un JDK 21. Lo destapó `Inflater.close()`, que figuraba como divergencia nuestra siendo que el método existe: lo que faltaba era `implements AutoCloseable`, que entró en Java 22. Hoy el gate resuelve el oráculo por orden y **anuncia cuál usó y en qué release**, porque un PASS sin saber contra qué no dice nada. Cambiarlo de 21 a 25 no produjo **una sola divergencia nueva** en 746 clases. En los paquetes que abrimos, `java.*` va **476 de 646** (74%), y **trece paquetes están cerrados al 100%**:

`java.util.function`, `java.util.zip`, `java.lang.constant`, `java.lang.invoke`, `java.math`, `java.util.random`, todo `java.time` —incluidos `chrono`, `temporal`, `format` y `zone`—, más `jakarta.validation` y `jakarta.persistence`, la primera expansión externa en cerrar entera. **Pero los trece no valen lo mismo**, y el documento lo dice caso por caso: `java.util.zip` cierra y se validó ejecutando; `java.lang.invoke` cierra con la mitad que importa declarada y lanzando.

Procedencia, dicho de frente. Casi todo esto está escrito de cero: las **512 clases de `java.*`** y las **89 de `jakarta.validation`** no tienen una línea copiada. La excepción es `jakarta.persistence`, cuyas anotaciones se compilaban primero desde las *fuentes* de la especificación para obtener sus `default` exactos; de esos 165 archivos ya se **regeneraron 126 desde el binario y quedan 39**, que conservan su cabecera `EPL-2.0 OR BSD-3-Clause` y su atribución a Oracle/Eclipse. El detalle completo —qué se regeneró, qué no y por qué— vive en `THIRD-PARTY.md`, en la raíz del repositorio.

El gate compara **firmas**: nuestra superficie pública tiene que ser un **subconjunto** de la del oráculo. Que exija subconjunto y no igualdad es lo que permite **omitir un miembro** que el compilador todavía no puede emitir y quedarse con la clase — la técnica que cerró las últimas 7 de JPA. Pero es una vara dura y a la vez insuficiente: **pasar el gate no dice que la clase funcione**. Los peores bugs de este track fueron invisibles para él, y eso dejó de ser una intuición — está **medido**: escaneando el bytecode ya emitido, **35 clases gatean limpio y no pueden ejecutar** (26 por el #110, 14 por el #112, 1 por el #119, con 6 en común).

H1 KajiLibrary — la biblioteca estándar propia ✅ tiers 0–5 cerrados Cumbre

Base ✅ **Tier 0 — Núcleo / raíces** (mayormente 🐞 nativo) — `Object`, `Class`, `String`, `System`, la jerarquía `Throwable` → `Exception` / `RuntimeException` / `Error` + subclases, `Enum`, `Integer`, `Math`, `Thread`, `java.lang.ref/*`, `java.io.PrintStream`. Escrito **de cero** (el `bootstrap/` es solo **referencia**). **Hecho**: compila y pasa el gate; `String` implementa `Comparable` + `CharSequence`; #6 (`Object` `super_class`) arreglado → `#0`. Correr pende del runtime

Base ✅ **Tier 1 — Interfaces fundamentales** (● Java puro) — `Comparable`, `Comparator`, `CharSequence`, `Iterable`, `Iterator`, `Runnable`, `AutoCloseable`, `Cloneable`, `Appendable`. Triviales y **base de casi todo lo demás. Cerrado**

Base ✅ **Tier 2 — Wrappers + texto** (●) — `StringBuilder` (implementa `CharSequence`); `Number` + los 8 wrappers (`Integer` / `Long` / `Short` / `Byte` / `Character` / `Boolean` / `Float` / `Double`, todos `Comparable` con **bridge** `compareTo(Object)`). **Hecho** (`Void` incluido; correr el boxing/unboxing pende del runtime)

Avanzado ✅ **Tier 3 — Interfaces funcionales** `java.util.function` (●) — `Function` / `Consumer` / `Supplier` / `Predicate` con sus default de composición (`andThen` / `compose` / `identity`, `and` / `or` / `negate`). **Confirmado que el `javac` propio emite lambdas** (`invokedynamic` + `LambdaMetafactory`). `java.util.function` **completo**: + `BiFunction` / `BiConsumer` / `UnaryOperator` (extends `Function`) / `BinaryOperator` (extends `BiFunction`)

Avanzado ✅ **Tier 4 — Colecciones** `java.util` (●, gran superficie) — interfaces `Collection` / `List` / `Set` / `Queue` / `Map` (+ `Map.Entry`) e implementaciones `ArrayList` implements `List` / `HashMap` implements `Map` / `HashSet` implements `Set` con `iterator()` real (retrofit hecho: #9/#10 arreglados; el iterador va en clase same-file por #13). Las utilidades `Objects` / `Optional` / `Arrays` / `Collections` pasaron a **H2**; los contenedores que faltaban (`Deque` / `LinkedList` / `ArrayDeque` / `TreeMap` / `TreeSet` / `Hashtable` / `LinkedHashMap` / ...) se cerraron en **H11**

Cumbre ✅ **núcleo Tier 5 — I/O básica** (🐞 en los bordes) — framework `InputStream` / `OutputStream` / `Reader` / `Writer` + concretas Java-puro `ByteArray` / `String` / `BufferedReader` (dogfoodean `StringBuilder`). `PrintStream` real y `FileInputStream` / `OutputStream` arrastran intrínsecos del VM → track **KajiJDK** (runtime). La **capa de decoradores completa** (14 clases) llegó en **H11**

✓ **Éxito**: un programa real —con `StringBuilder`, colecciones y lambdas sobre sus interfaces funcionales— se **compila con el `javac` propio y corre en KajiJDK**, sin tocar el JDK de Temurin. La biblioteca se escribe a mano en `KajiLibrary/` (nuevo directorio en la raíz), en *tiers* por dependencia: ● Java puro (autocontenido, se dogfoodea entero) · 🐞 necesita **intrínsecos del VM** (avanza con el track KajiJDK/runtime).

H2 Utilidades y conveniencia (`java.util`) 🚧 en curso Cumbre

Avanzado ✅ `java.util.Objects` — utilidad estática (`requireNonNull`, `equals`, `hashCode`, `isNull` / `nonNull`, `toString`). Hecho (8 miembros). Pero cablear su `requireNonNull` en los `default` de Tier 3 **espera el fix de #11**: el `javac` no resuelve llamadas estáticas a `java.util` (ni `Objects` / `Arrays` / `Collections`)

Avanzado ✅ `java.util.Optional<T>` — contenedor sin-`null` (`of` / `empty` / `ofNullable`, `get` / `isPresent` / `isEmpty` / `orElse` / `orElseGet`, `map` / `filter` / `ifPresent`). Dogfoodeó genéricos + inferencia + lambdas sobre las interfaces funcionales. Sumó `NoSuchElementException`

Avanzado ✅ `java.util.Arrays` — 20 miembros: `toString` / `equals` / `fill` / `copyOf` / `hashCode` sobre `int` / `char` / `boolean` / `Object[]` + `sort` (insertion). Escrita; **consumirla** desde otras clases espera #11

Cumbre ✅ `java.util.Collections` — `swap` / `reverse` / `fill` / `sort` ×2 (natural con type param acotado + por `Comparator`), sobre acceso indexado de `List`. Escrita; **consumo** espera #11 y que los concretos sean `List` (#9). Faltan `emptyList` / `unmodifiable*` / `binarySearch` (necesitan backing concreto o iteración)

Avanzado ✅ **Stragglers de tiers previos** — `Void` (T2), `Appendable` (T1) y `BiFunction` / `BiConsumer` / `UnaryOperator` / `BinaryOperator` (T3). `java.util.function` quedó **43/43** en H11

✓ **Éxito**: el esqueleto de tiers 0–5 se vuelve una biblioteca **usable**: las clases-pegamento que aparecen en casi todo el código real (utilidades estáticas y contenedores), todas ● **Java puro** y **desbloqueadas ya** (no dependen de los gaps del compilador #4–#10).

H3 java.util.stream — pipelines funcionales **núcleo (T1–T5)** **Cumbre****Avanzado** **T1 — Substrato funcional** — las 6 especializaciones primitivas de `java.util.function``(IntFunction / ToIntFunction / IntPredicate / IntConsumer / IntUnaryOperator / IntBinaryOperator)` + la interfaz `Collector<T,A,R>`**Avanzado** **T2 — Stream<T> core (eager)** — intermedias `filter / map / flatMap / distinct / sorted / limit / skip / peek + mapToInt / mapToLong / mapToDouble`; terminales `forEach / reduce / count / toList / toArray / anyMatch / allMatch / noneMatch / findFirst / min / max`; factories `of / empty . toList()` devuelve un `ArrayList` real. `collect()` + `Collection.stream()` esperan **#17****Avanzado** **núcleo T3 — Collectors** — `toList / toSet / joining / counting` (+ el `Collector` concreto `CollectorImpl`), `gate-clean`. El `collect()` que los consume espera **#17** (override de variable de tipo a nivel método). Faltan `groupingBy / mapping`**Avanzado** **T4 — Streams primitivos** — `IntStream / LongStream / DoubleStream` eager`(range / of / sum / average / map / filter / reduce / sorted / limit / skip / ... + boxed())` con `OptionalInt / OptionalLong / OptionalDouble . mapToObj` diferido (#7 / array genérico)**Cumbre** **núcleo T5 — Avanzado** — `flatMap` (eager) + bridges primitivo→objeto (`Stream.mapToInt / mapToLong / mapToDouble , *Stream.boxed()`). Fuera de alcance del modelo eager (piden rediseño lazy): `Splititerator / lazy` real, `iterate / generate` infinitos, paralelismo verdadero

✓ **Éxito:** el pago de Tier 3 (interfaces funcionales) + Tier 4 (colecciones): `coleccion.stream().filter(...).map(...).collect(...)`. Dogfoodea el compilador como nada — lambdas encadenadas, genéricos, method refs. **Eager primero** (cada op intermedia materializa en un backing interno; la terminal consume): correcto para streams finitos y mucho más simple que el lazy; el lazy real (`Splititerator` + pipeline diferido) es un tier avanzado. Se apoyó en los **retrofits de #9** (ya hechos: `ArrayList / HashMap / HashSet` son `List / Map / Set` reales con `iterator()`), así que `toList()` devuelve un `ArrayList` de verdad. Núcleo T1–T5 hecho; solo `collect()` + `Collection.stream()` esperan **#17** (override de variable de tipo a nivel método).

H4 java.time — fecha y hora como value types **COMPLETO (value types + offsets + formato + chrono + 2 calendarios)** **Cumbre****Cumbre** **T1 — Abstracción** `java.time.temporal` — 7 interfaces`(TemporalAccessor / Temporal / TemporalField / TemporalUnit / TemporalAmount / TemporalQuery / TemporalAdjuster)` + enums `ChronoField / ChronoUnit`. El substrato sobre el que se montan los value types**Cumbre** **T2 — Duración e instante** — `Duration` (seg+nanos, `TemporalAmount`) e `Instant` (epoch seg+nano, `Temporal`, `now()` 🐛 = `System.currentTimeMillis` native). Normalización de nanos con floor manual**Cumbre** **T3 — Fecha local** — `LocalDate`: la conversión **epoch-day** ↔ (**año,mes,día**) del JDK (bisiestos, largos de mes) compilada y andando; `plusDays / plusMonths / getDayOfWeek . +` + enums `Month / DayOfWeek`**Cumbre** **T4 — Hora y fecha-hora** — `LocalTime` (h/m/s/nano, wraparound de día) y `LocalDateTime` (compone T3+T4, la aritmética de horas carga días en la fecha)**Cumbre** **núcleo T5 — Períodos y parciales** — `Period` (`TemporalAmount`), `Year / YearMonth` (`Temporal`), `MonthDay` (`TemporalAccessor`)**Cumbre** **JT1b — Conveniencias + parse** — `get / plus / minus / with` genéricos y `parse(CharSequence)` en los **9 value types** (round-trip ISO con `toString`; `Instant.parse` reusa `LocalDate / LocalTime`, `Duration.parse` con signo/fracción). `InstantSource` (Java 17) + `Clock implements InstantSource` → paquete principal 19/19**Cumbre** **JT2 — Offsets** — `ZoneOffset`, `OffsetTime`, `OffsetDateTime`, `Clock`; validado `toEpochSecond / toInstant` vs JDK**Cumbre** **JT3 — DateTimeFormatter** — `ofPattern + format(TemporalAccessor)`, motor de patrones (`u / y · M [num/nombre] · d · H · h · m · s · S · a · E`, literales/comillas) leyendo campos por `getLong(ChronoField)`; **48/48** vs JDK. + `LocalDate/Time/DateTime.format(fmt)`**Cumbre** **JT4 — Zonas** — `ZoneId` (abstracto; `of()` de formas offset), `ZoneOffset extends ZoneId`, `ZonedDateTime`; **238/0** vs JDK. **Y el muro de tzdb cayó:** `java.time.zone` cierra **5/5** (`ZoneRules`, `ZoneOffsetTransition`, `ZoneOffsetTransitionRule`, `ZoneRulesProvider`, `ZoneRulesException`) apoyado en una tabla de datos propia (`TzData`) en vez de los ~600 KB de la IANA. Gate PASS; **lo que cubre esa tabla frente a la tzdb completa es lo que queda por medir****Cumbre** **JT5 — Chrono + calendarios** — `Chronology / AbstractChronology / IsoChronology / Era / IsoEra / ChronoLocalDate`; `ThaiBuddhistDate` (+543, **25504/0**) y `MinguoDate` (–1911, **21004/0**) vía refactor de `ChronoLocalDate` a `default methods`. **Backlog cerrado:** las interfaces `Chrono{LocalDateTime,ZonedDateTime,Period}` y los calendarios **Japanese** (con sus eras Meiji→Reiwa) e **Hijrah** (tabla lunar) — `chrono` queda **21/21**

✓ **Éxito:** el dogfood de las **clases inmutables**: cada tipo es `final` con campos `final`, y `plus / minus / with` devuelven instancias nuevas. Casi todo es aritmética pura — el único seam 🐛 es el reloj (`now()` → un `System.currentTimeMillis()` native). **Ahora COMPLETO más allá del núcleo ISO:** value types + **offsets** (`ZoneOffset / OffsetDateTime / OffsetTime / ZonedDateTime` con offset fijo, `ZoneId` abstracto) + **formato por patrón** (`DateTimeFormatter.ofPattern + format`) + **chrono** (`IsoChronology` + los calendarios **Minguo** y **ThaiBuddhist** —vía refactor de `ChronoLocalDate` a `default methods`, para que los `Date` tiereden en vez de declarar—) + `InstantSource`. **Paquete principal java.time 19/19; gate 721 match, 0 mismatch, 0 extra, PASS. Y ahora los subpaquetes también cierran:** **chrono 21/21, temporal 16/16, format 8/8** —las tres interfaces `Chrono{LocalDateTime,ZonedDateTime,Period}`, los calendarios **Japanese** e **Hijrah**, el `DateTimeFormatterBuilder`, `IsoFields` y `WeekFields` —, y **zone 5/5**, con lo que **H4 queda cerrado entero**. El **round-trip ISO** cierra (`parse` en los 9 value types), y todo lo demás está validado con **decenas de miles de checks** contra el JDK como oráculo. Findings del compilador destapados y resueltos: **#22** (resolución de métodos heredados —super/interfaz—, que habilitó el refactor chrono) y bridges covariantes verificados.

H5 java.util.regex — motor de expresiones regulares **T1–T4 + T5 parcial** CumbreCumbre **T1 — API pública** — `Pattern (compile / matcher / pattern / quote / matches) + Matcher``(matches / find / lookingAt / group / start / end / groupCount / reset) + PatternSyntaxException`. La superficie sobre la que se apoya todoCumbre **T2 — Parser** — regex → árbol de nodos: literales, `.`, clases `[...]` (rangos, negación), cuantificadores greedy `*` / `+` / `?` / `{n,m}`, grupos `(...)`, alternación `|`, anclas `^` / `$`, escapes `\d` / `\w` / `\s` y sus negacionesCumbre **T3 — Motor de matching** — backtracking recursivo continuation-passing; `matches()` / `find()` / `lookingAt()`; `Quantifier` greedy con loop-back + `Prolog` (reset de contador para cuantificadores anidados) y guard de zero-widthCumbre **T4 — Grupos y reemplazo** — grupos capturadores → `group(n)` / `start(n)` / `end(n)` (con guardar/restaurar en backtrack);`replaceAll / replaceFirst / quoteReplacement` con referencias `$n`Cumbre **T5 (parcial) — Avanzado** — HECHO y validado: reluctant `?` / `+` / `{}`?, grupos non-capturing `(?:...)`, lookahead `(?=...)` / `(?!...)`, backreferences `\1` – `\9`. **Backlog:** possessive `+`, lookbehind `(?<=...)`, grupos con nombre `(?<n>...)`, clases Unicode/POSIX `\p{...}`, flags DOTALL/MULTILINE

✓ **Éxito:** un motor de regex de verdad, 100% Java puro — el dogfood más algorítmico de la fase: un **parser** de la sintaxis regex + una **máquina de matching por backtracking recursivo** sobre una **jerarquía de nodos con match polimórfico** (cada nodo hace `match(m, pos, seq)` y, al tener éxito, llama al nodo siguiente — *continuation-passing*, lo que habilita backtrackear en los cuantificadores). NFA con backtracking (no DFA), char-based. **Núcleo cerrado y VALIDADO:** como el track de biblioteca no ejecuta, se portó el mismo algoritmo a Java puro y se corrió con el JDK contra `java.util.regex` como oráculo → **138/138 checks** (matches + find-all con spans por grupo + reemplazos). Cero findings de correctitud del compilador. 18 clases (Pattern/Matcher/PatternSyntaxException + jerarquía de nodos + parser).

H6 java.util.Formatter / String.format — formato printf **COMPLETO (T1–T5)** CumbreCumbre **T1 — API + parser + conversiones base** — `java.util.Formatter (format / toString / out / flush / close, Closeable / Flushable) + String.format`; parser del format string → chunks literales + specifiers; conversiones `%s` / `%S`, `%%`, `%n`, `%b`, `%c`Cumbre **T2 — Enteros** — `%d` / `%x` / `%X` / `%o` (radix vía `Integer / Long`), con flags `-` (left-justify) / `0` (zero-pad) / `+` / espacio / `(` (negativos) / `,` (grouping) y widthCumbre **T3 — Punto flotante** — `%f` / `%e` / `%E` / `%g` / `%G` con precisión (default 6), redondeo, width y flags. La parte algorítmica dura: `double` → string decimalCumbre **T4 — Flags/width/precision + índice de argumento** — pulido transversal: `#` (alt-form), precisión en `%s`, índice `%n$` y relativo `%<`, edge cases de width/precision por conversiónCumbre **T5 — Avanzado (hecho)** — `%a` (hex-float) + `Formattable, Double / Float.toString` (shortest-decimal), fecha/hora `%t` / `%T`, `Locale` (separadores), e integración con `java.text (DecimalFormat / NumberFormat / MessageFormat)`

✓ **Éxito:** el formato de texto estilo printf, 100% Java puro — mismo espíritu que regex: un **parser** de un mini-lenguaje (el format string `%[arg$][flags][width][.prec]conv`) + un **motor de conversión** que formatea cada argumento según su especificador. `Formatter` envuelve un `Appendable` y `String.format` es el atajo. **Cerrado T1–T5 (incl. el avanzado): 322 casos validados** contra `java.util.Formatter` del JDK. T5 completo: `%a` / hex-float + `Formattable, Double / Float.toString`, fecha/hora `%t`, `Locale`, e integración con `java.text (DecimalFormat / NumberFormat / MessageFormat)`.

H7 Fuera de alcance por ahora **backlog** CumbreCumbre `java.nio` — I/O de canales y buffers (👉 descriptors de archivo → runtime)Cumbre `java.util.concurrent` — salió del backlog y es hoy el hito **H9:** se pudo hacer **en Java puro** sobre los monitores que la VM ya tenía, con sólo dos natives nuevosCumbre **reflexión** — la metadata la tiene el runtime (metaspace: field/method tables + anotaciones ya parseadas); faltaban los **natives que la puenteen a Java**. **Hecho (natives + tests):** `Class.getName / getModifiers / getSuperclass / isInterface / isAssignableFrom` (leen el metaspace) + `ldc de class literals (X.class → mirror)` en el intérprete; de paso se destapó y arregló un bug latente de `load_class` (dedupeaba por *id* en vez de por *mirror*). `Class.getDeclaredFields()` — **aloca un `java.lang.reflect.Field` por campo** desde el metaspace (clazz/name/type/modifiers/slot) y arma el `Field[]` (GC-safe: `malloc` no dispara GC), `Field.get` de campos de referencia. API `java.lang.reflect` completa a nivel de clase:

`Modifier / Member / AccessibleObject / Field / Method / Constructor` (todas gate ok; `Method.invoke / Constructor.newInstance` + `getDeclaredMethods / Constructors` = natives pendientes). Tests: `class_reflection...` = 11022, `field_reflection...` = 206. **Lectura de anotaciones en runtime** — el bloqueante del motor de validation, resuelto sin proxies dinámicos: natives `Reflect.hasConstraint/constraintLong/constraintString(Field, tipo, elemento)` que leen los `RuntimeVisibleAnnotations` **ya parseados** por el metaspace (accessors nuevos en `annotations.rs`). Test `reads_constraint_annotations_off_fields` = 102101 (`@NotNull` + `@Size(min=2,max=10)`). **Falta (👉 runtime):** `Field.set`, boxing de primitivos en `Field.get`, `Method.invoke / Constructor.newInstance`. El motor de Bean Validation (H8) ya no espera al runtime: lo bloquea el **#102** del compilador

✓ **Éxito:** lo que queda para **acotar** el scope, todo **acoplado al runtime** (espera el track KajiJDK): I/O de bajo nivel, concurrencia y reflexión. `java.text` (formato por patrón) es autocontenido y vive como el T5 diferido de H6.

**H8 jakarta.validation (JSR 303/380 — Bean Validation) superficie COMPLETA (88/88) **

  `java.lang.annotation.` — `Annotation` / `ElementType` / `RetentionPolicy` / `Retention` / `Target` / `Documented` / `Inherited` / `Repeatable`; gate 26 match vs JDK**. Llena el hueco JLS §4

  **Base** — `Payload`, `Constraint`, `ConstraintValidator<A,T>`, `ConstraintValidatorContext`; anotaciones **runtime reales** (`RuntimeVisibleAnnotations` + `AnnotationDefault` emitidos)

  **23 constraints** — `@NotNull` / `@Size` / `@Min` / `@Max` / `@Pattern` / `@Email` / `@NotBlank` / `@Digits` / `@DecimalMin` / ... cada una con `message` / `groups` / `payload` + `params` + `@interface List` (repeatable). `Pattern` con enum anidado `Flag`. **Gate 111 match, 0 mismatch vs jar**

  **Core** — `Validator` / `ValidatorFactory` / `ValidatorContext` / `Validation` / `Configuration` / `ConstraintViolation` / `Path` (+ `Node` +8 subnodos) / `MessageInterpolator` / `ClockProvider` / `TraversableResolver` + la jerarquía de excepciones completa. Truco clave: los genéricos **auto-referenciales** (`Configuration<T extends Configuration<T>>`, `ProviderSpecificBootstrap`, `spi.ValidationProvider`) se modelan **raw** con los métodos devolviendo el bound — así el descriptor calza con la erasure del JDK **sin allowlist**, esquivando el #100

  **Metadata (17/17)** + `spi/bootstrap/executable/valueextraction/groups/constraintvalidation`. **Superficie total: 88/88 clases, gate 357 match / 0 mismatch / 0 extra**, 6 divergencias allowlisteadas ($4 \times \#100 + 2 \times \#101$). ☐ **El motor** (un `Validator` que reflexione sobre el bean y produzca `ConstraintViolation`) queda pendiente: la lectura de anotaciones en runtime **ya está** (natives), pero lo bloquea el #102 API

✓ **Éxito:** una expansión **más allá de java.base**: la spec de Bean Validation (`jakarta.validation`, sucesor moderno de JSR 303/380). Como no está en el JDK, el **oráculo del gate es el jar** `jakarta.validation-api-3.1.1` (javadoc por classpath). Es todo **anotaciones + interfaces + reflexión** (98 clases). Prerequisito resuelto: `* java.lang.annotation.` propio — el **javac propio compila el bootstrap circular de meta-anotaciones auto-referenciales** (`Retention` anotado con `@Retention`) **sin cambios**. **Findings del compilador destapados:** #100 (variable de tipo acotada **erasa a Object en vez de su bound, JLS §4.6**) y #101** (referencia calificada a tipo anidado `Outer.Nested` no resuelve; con `import cross-file` emite `Object[]` — codegen silencioso incorrecto).

**H9 java.util.concurrent (JSR 166) COMPLETO (C1–C5) y validado ejecutando **

  **C1 — atomic (13) + TimeUnit** — `AtomicInteger` / `Long` / `Boolean` / `Reference`, los arrays, `Markable` / `Stamped` y los adders/accumulators. Lectura y escritura simples son *un* opcode (atómicas de por sí); sólo los read-modify-write toman el monitor. Los adders extienden `Number` directo (el JDK pasa por `Striped64`, que sin contención no aporta). `TimeUnit` usa el `cvt` con saturación del JDK. **La lección más cara del hito vive acá:** ese `cvt` estaba **validado 112/112 contra el JDK** y sin embargo **toda** conversión grueso→fino trapeaba en nuestra VM —leía `Long.MAX_VALUE` con `getField` (#110); sólo `src == dst` esquivaba el bug, que es justo la forma que cubrió la validación. **Validar en Java real no prueba nada sobre nuestra VM**

  **C2 — locks** — `Lock` / `Condition` / `ReadWriteLock` + `ReentrantLock` y `ReentrantReadWriteLock` (con las clases anidadas `ReadLock` / `WriteLock` reales). Identidad de dueño por `Thread.currentThread()`; el `await` de las condiciones es **lost-wakeup-free** (entra al monitor de la condición *antes* de soltar el lock). El RW lock soporta reentrancia y el **downgrade** write→read. **Dos natives agregados al runtime:** `Thread.currentThread()` y `Object.wait(long)`

  **C3 — sincronizadores** — `CountDownLatch`, `Semaphore`, `CyclicBarrier`, `Exchanger`. La barrera modela las generaciones como un **contador**, con `brokenGen` marcando la que rompió un `reset`: así ven la excepción exactamente las partes que estaban esperando, y la barrera queda reusable

  **C4 — executors** — `Executor` / `Callable` / `Future` / `ExecutorService`, `FutureTask`, `ThreadPoolExecutor` (workers lazy hasta el core size; el último en salir marca *terminated*) y las factories de `Executors`. Es la pieza más didáctica: los hilos son caros, así que se crean una vez y se **reusan**, y la cola absorbe las ráfagas

  **C5 — colecciones concurrentes** — `ConcurrentHashMap`, `CopyOnWriteArrayList` (el array se reemplaza en cada escritura; los lectores no toman lock y el iterador ve una *snapshot*), `BlockingQueue` + `ArrayBlockingQueue` / `LinkedBlockingQueue`. El productor/consumidor sobre una cola **acotada** —donde ambos lados bloquean de verdad— es el test insignia

  **C6 — el resto de la cola** — `Phaser` (partes **dinámicas**: se registra una tercera *entre fases*, que es lo que un `CyclicBarrier` no puede; la terminación va en el bit de signo), `SynchronousQueue` (capacidad **cero** — no es un contenedor sino un punto de encuentro: `size()` siempre 0), `PriorityBlockingQueue` / `DelayQueue` (delegan el heap a nuestro `PriorityQueue` y sólo agregan monitor y espera), `ConcurrentLinkedQueue` / `Deque`, `CopyOnWriteArraySet`, `ThreadLocalRandom` y `ExecutorCompletionService` (entrega por orden de **terminación**, no de envío)

 ☐ **Cola** — los `FieldUpdater` (piden `Field.getInt/setInt` como native) y `LockSupport` / AQS (piden `park` / `unpark`: hay que coordinar con el track del runtime).  **Bug del runtime, latente en código ya entregado:** un `wait(ms)` que **expira** queda en el wait-set* del monitor — `expire_timed_block` sólo lo saca si el estado es `Waiting`, y un wait temporizado está en `TimedWaiting` —; después un `notify` dreña esa entrada rancia y **bloquea al hilo que corre**. Afecta a `ArrayBlockingQueue.poll(timeout)`, `LinkedBlockingQueue` y `SynchronousQueue.poll(timeout)`. Fix de **una línea**, del track runtime; hay repro y un test `#[ignore]` que pasa a verde cuando se arregle

✓ **Éxito:** el paquete de concurrencia **entero en Java puro**, sin tocar el runtime salvo dos natives mínimos. La clave conceptual: en una VM cuyos hilos se intercalan **entre opcodes**, un atómico hecho con `synchronized` es *indistinguible* de uno lock-free con CAS — así que todo se construye sobre los monitores que la VM ya tiene (`synchronized` / `wait` / `notify`). Gate limpio y —lo que de verdad importa— **24 tests de comportamiento corriendo la biblioteca sobre el intérprete real** (`tests/concurrency_locks.rs`), no sólo forma de API. **Es el único paquete validado end-to-end ejecutando**, y por eso es el que destapa los bugs que el gate no ve: de los 62 tipos públicos del JDK están los que no piden `Unsafe`.

H10 jakarta.persistence (JSR 338 — JPA 3.2) ✔ COMPLETA — 203/203 clases (P1–P6) Cumbre

Cumbre ✔ **P1 — enums, excepciones y markers** — 20 enums (`AccessType` / `CascadeType` / `FetchType` / `LockModeType` ...), la jerarquía de `PersistenceException` (11) y las interfaces de opción nuevas de 3.2 (`FindOption` / `LockOption` / `RefreshOption`) + `Timeout`

Cumbre ✔ **P2 — las 93 anotaciones de mapeo** (`@Entity` / `@Table` / `@Column` / `@Id` / `@OneToMany` / ...) **con sus defaults reales**. Primero se compilaron las fuentes del `-sources.jar`, que era la única forma de tener los `default` exactos — `javap` a secas no los muestra, y una anotación cuyos defaults difieren no es la misma anotación—. Eso dejaba **165 archivos de Eclipse/Oracle** dentro del repositorio, con su cabecera `EPL-2.0 OR BSD-3-Clause`. **Después se regeneraron 126 desde el BINARIO**: los miembros salen de los descriptores y los `default` del atributo `AnnotationDefault`, que los tiene exactos — la misma relación que este proyecto ya tiene con el JDK en todos lados, leer la forma pública de un `.class` y reimplementarla. **Quedan 39 copiados**, con su licencia y atribución intactas: nueve son interfaces con métodos `default` y seis son clases con cuerpo, y **un cuerpo es bytecode, no fuente** — reconstruirlo pediría decompilar, y una decompilación aproximada reemplazaría comportamiento por una suposición. Los 24 restantes esperan fixes del compilador. **Todo declarado en** `THIRD-PARTY.md`, en la raíz del repositorio

Cumbre **Lo que el gate no podía ver, y por eso se midió aparte**. Un `default` **no está en el descriptor**, así que la comparación de firmas da verde aunque los valores desaparezcan — y en la primera pasada de la regeneración **se perdieron 22 en silencio**. `tools/check_jpa_defaults.py` los compara uno por uno contra el jar: **197 idénticos, 0 distintos**. De paso destapó el finding **#127**: un `default` **negativo** o con **valor de anotación** no se emite, así que `int secondPrecision() default -1;` compila sin una queja y el `.class` sale sin el atributo

Cumbre ✔ **P3/P4 — core y metamodel** — `EntityManager` completo (64 métodos), `EntityManagerFactory` (21), `Query` / `TypedQuery`, `Persistence`, `Cache`, `EntityGraph`, `Tuple` ... y el paquete `metamodel` (17). Los mismatches iniciales resultaron **artefactos del orden de compilación**, no bugs: recompilando con todas las dependencias presentes dan PASS limpio

Cumbre ✔ **P5/P6 — criterios y spi** — destrabados por la técnica `cppjar-minus-self`: extraer el jar a un **directorio** (el finder lee dirs, no jars) y compilar cada fuente con ese classpath **habiendo borrado su propio** `.class` —lo segundo evita la trampa de perf del #19—. Así los ciclos `metamodel` ↔ `criteria` ↔ `EntityManager` resuelven contra las clases reales **sin ordenar nada**. La lección general: un mismatch `→Object` suele ser un **artefacto de orden**, no un bug; siempre hacer un pase final con todas las deps antes de juzgar

Cumbre ✔ **Las 7 últimas — cerradas sin tocar el compilador**. Lo que las destrabó fue una observación sobre el gate, no un fix: **exige subconjunto, no igualdad**, así que omitiendo *el miembro* que no compila entra *la clase* entera, y el hueco queda **declarado** en vez de escondido.

`SetJoin` / `CollectionJoin` / `ListJoin` / `MapJoin` omiten su `getModel()` covariante (#123) —que igual **heredan** de `PluralJoin`: la única pérdida real es el retorno covariante, que obliga a castear— y `MapJoin` también `entry()` (#101). `CriteriaBuilder` omite 5 métodos por dependencias de **otros módulos** (`java.sql`, `java.math`). `PersistenceProviderResolverHolder` es la única con **cuerpo propio**: la de referencia usa `ServiceLoader` + `WeakReference` por class loader + `java.util.logging`, nada de lo cual existe; como tampoco hay proveedor que descubrir, el resolver por defecto devuelve **lista vacía** —honesto en vez de simulado— y `setPersistenceProviderResolver` sigue funcionando, que es el punto de la indirección

Cumbre ⚠ **La deuda que el verde tapa** — `CriteriaBuilder` gatea con **14 entradas de allowlist, todas del #100**: una variable de tipo **acotada** erasa a `Object` en vez de a su cota (`N extends Number` → `Number`, `Y extends Comparable` → `Comparable`, `C extends Collection`, `M extends Map`). La fuente declara las cotas bien; lo que sale mal es el **descriptor emitido**, así que son 14 métodos que **no linkearían** contra un llamador real. Hoy no molesta —JPA es *gate-only*, nunca ejecuta— y por eso mismo conviene tenerlo escrito: es exactamente el tipo de deuda que un gate en verde disimula

✔ **Éxito**: la segunda expansión fuera de `java.base`, con el mismo molde que H8: **API pura, sin motor** (un proveedor JPA real es ORM + JDBC + una base de datos, inequívocamente fuera de alcance). Oráculo del gate: el jar `jakarta.persistence-api 3.2.0`. **Cerrada: 203/203 clases** (root 152 · `criteria` 26 · `metamodel` 17 · `spi` 8), gate **PASS**. El grueso dio **990 match / 0 mismatch / 0 extra** con 5 allowed; las 7 últimas entraron después, con un costo que vale la pena mirar de frente (abajo).

H11 Cierre de superficie de `java.base` — ~200 clases en paralelo **lote grande**

  **java.util (78/110)** — los **6 esqueletos abstractos** (`AbstractCollection` / `List` / `Set` / `Queue` / `SequentialList` / `Map`, que son justo los que generaban nuestras entradas de allowlist: al extenderlos, los métodos pasan a **heredarse** en vez de declararse, como en el JDK), las 10 interfaces del orden (`Sorted` / `Navigable` / `Sequenced`), y los contenedores que faltaban: `TreeMap` / `TreeSet` (**árbol rojo-negro de verdad**, con un `checkInvariants()` que devuelve la altura negra o `-1`), `LinkedList` / `ArrayDeque`, `PriorityQueue` (heap binario), `Hashtable` (**encadenamiento separado**, a propósito como contraste contra el **direccionamiento abierto** de nuestro `HashMap` — dos respuestas a la misma pregunta), `LinkedHashMap` (la tabla **más** una lista doblemente enlazada que pasa por los **mismos** objetos entry, con `accessOrder` + `removeEldestEntry`: el par que lo vuelve una caché LRU), `IdentityHashMap`, `WeakHashMap`, `EnumMap` / `EnumSet`, `BitSet`, y los codecs `Base64` / `HexFormat` / `UUID`

  **java.io (51/81)** — la jerarquía de excepciones, `DataInput` / `DataOutput` y los **14 decoradores**, que es donde vive el algoritmo: el `fill()` real de `BufferedInputStream`, `PushbackReader`, `LineNumberReader`, `StreamTokenizer` ... El patrón decorador es el contenido del paquete; lo que falta pide **descriptores de archivo** (🐭 runtime)

  **java.util.function 43/43 — COMPLETO**, y **java.lang (82/107)** con `StringBuffer`, `ThreadLocal`, un `StrictMath` acotado y la jerarquía entera de `Error` / `Exception`

 **Lo que hizo funcionar la flota**, que es el aprendizaje transferible: (1) pasarle a cada agente el **manual de minas completo** —los workarounds numerados de los findings abiertos—, porque sin eso pisan todas; (2) **exigir validación de comportamiento**, no sólo gate: el agente de codecs corrió **86 773 comparaciones** contra el JDK y encontró un bug propio, el de `java.io` corrió 105/105 y encontró otro; (3) prohibirles tocar `src/`, commitear y editar el archivo de findings —que reporten y consolide uno solo—; (4) exigir que el algoritmo se porte **mecánicamente** al JDK para validar, así se prueba el código que se entrega y no una reescritura

  **java.lang.constant 11/11 — el paquete de los descriptores nominales.** `ClassDesc`, `MethodTypeDesc`, la familia `MethodHandleDesc` / `DirectMethodHandleDesc` (con su enum `Kind`), `DynamicConstantDesc`, `DynamicCallSiteDesc` y los ~50 `CD_` de `ConstantDescs`. *Lo que define al paquete es lo que se le quitó: cada tipo declara un `resolveConstantDesc(MethodHandles.Lookup)` —el método que convierte la descripción en la cosa viva— y **todos están omitidos**, porque resolver es `java.Lang.invoke`, o sea maquinaria de la VM y no biblioteca (`ConstantDesc` queda como interfaz **vacía**). Sobrevive la mitad útil: construir, comparar e imprimir un descriptor **sin resolver nada**, que es exactamente lo que lo hace usable en tiempo de compilación. Contenido algorítmico real: el parser de descriptores —que son **auto-delimitados**, y por eso una lista de parámetros no lleva separadores— y `invocationType()`, donde el descriptor de lookup y el tipo de invocación* se separan: un método de instancia recibe el receptor como primer argumento, un constructor **devuelve** la clase que construye, y un accessor de campo no tiene descriptor propio sino uno sintetizado. **Cuidado que casi se pierde**: el campo `name` de `ConstantClassDesc` **lo lee el intérprete por nombre** para matchear labels de `switch` sobre enum, así que `descriptorString()` computa el descriptor **encima** del nombre binario en vez de reemplazarlo — el contrato quedó escrito en el archivo*

 **El patrón que destapó el lote, ya innegable.** Seis findings (**#108, #111, #114, #118, #119, #120**) comparten una sola raíz: **cuando la resolución falla, el compilador emite silencio en vez de un error** — la llamada desaparece y deja operandos colgados o la pila vacía. Y cuatro son puntos ciegos del **lector de classpath** (#104 `Exceptions`, #110 `ACC_STATIC`, #118 `ACC_VARARGS`, #120 la tabla de métodos del supertipo): lo que *escribimos* está bien, lo que *leemos* se interpreta mal. **Si el track del compilador arregla una sola cosa, que sea la guarda** — una llamada que no resuelve tiene que ser error de compilación: eso convierte seis bugs mudos en seis mensajes. Caso testigo: `WeakHashMap` **gatea limpio y no puede correr** (#119), sin workaround posible desde el fuente

✓ **Éxito:** el hito **menos conceptual y más de kilometraje:** llenar los huecos que quedaban en los paquetes ya abiertos, hechos por una **flota de agentes en paralelo** (6 + 4 relanzados), cada uno dueño de un paquete. **La biblioteca entera pasa el gate: 3074 match / 0 mismatch / 0 extra, 47 allowed, PASS** sobre **681 clases públicas** (396 de `java.` + 285 de `jakarta.`) — la cobertura de los paquetes que tocamos subió de ~38% a ~68%. Lo interesante no fue el volumen sino **qué hizo falta para que el paralelismo rindiera**.

**H12 java.util.zip — compresión de cero COMPLETO (21/21) y validado ejecutando **

  **Checksums** — `Adler32` y `CRC32 / CRC32C` son **dos respuestas distintas a la misma pregunta**, como `Hashtable` frente a nuestro `HashMap`. Adler son dos sumas corridas donde la segunda acumula la primera, y ese detalle es todo: una suma simple de bytes no ve un reordenamiento, pero sumar el total corrido hace que cada byte dependa de su **posición**. CRC es división polinómica, con una tabla que precomputa ocho pasos de bit por vez. Adler es mucho más barato y bastante más débil; zlib eligió ese trade a propósito

  **Inflater** — **DEFLATE (RFC 1951) en Java puro**. Tres cosas que cuesta acertar y fallan calladas: el stream se lee **LSB-first** pero un código Huffman se empaqueta **MSB-first**, y los dos órdenes conviven en el mismo stream; una *back-reference* puede **solapar** los bytes que todavía está produciendo, así que hay que copiar byte a byte —y eso es lo que hace que mil bytes iguales cuesten tres—; y como el input llega en pedazos, el decodificador es **resumible por checkpoint**: el cursor se guarda tras cada símbolo completo y en *underflow* rebobina ahí

  **Deflater** — **lo que NO hace, escrito arriba del archivo**. Emite sólo bloques **STORED**: es DEFLATE válido y cualquier inflater del mundo lo lee, pero **no comprime**. Las dos mitades no son el mismo problema — decodificar es mecánico, el formato te dice qué hacer; **codificar es donde vive el algoritmo** (el buscador de coincidencias sobre 32 KB, elegir entre literal y back-reference, armar tablas de Huffman que le ganen a las fijas lo suficiente como para pagar transmitirlos). Con el encoder de bloques stored andando, los streams y el escritor de zip funcionan de punta a punta, y el *match finder* entra después detrás de la misma superficie

  **El contenedor ZIP, que no es DEFLATE**. Se conflacionan porque una extensión cubre las dos, pero DEFLATE es un stream de bytes sin noción de archivos y ZIP es el formato que carga muchos streams más un índice. Cada entrada lleva un **header local** delante y el archivo termina con un **directorio central** que repite toda la metadata: esa duplicación es el diseño —permite listar sin leer, y permite escribir sin poder hacer *seek*—, y es también por qué un zip truncado suele ser parcialmente recuperable. Las tres capas del sobre (DEFLATE · zlib · gzip) quedan separadas: `GZIPOutputStream` no comprime *más* que un `DeflaterOutputStream`, son los mismos bytes en otro envoltorio

  **Los dos bugs que encontré la ejecución**. (1) `needsInput()` **mentía**: con bytes que no alcanzan para el símbolo siguiente devolvía `false`, el consumidor no alimentaba más y `inflate` devolvía 0 para siempre — el self-test se colgó. «Se quedó sin bits a mitad de símbolo» es un estado **distinto** de «no queda input». (2) El tamaño comprimido del descriptor salía corto **por 5 bytes exactos**, el bloque final de `def.finish()`, porque `deflate()` escribe directo a `out` sin pasar por el contador de posición. El lector del JDK lo cazó al instante («*invalid entry compressed size, expected 15 but got 20*»); **el gate, nunca**

  **ZipFile es un caso especial, y está escrito así**. Su razón de existir es el **acceso aleatorio** —saltar al final, leer el directorio, abrir sólo la entrada que se quiere— y no hay `java.io.File` ni descriptors de archivo. El constructor **falla a propósito** en vez de fingir que sostiene un archivo, mismo criterio que `Validation.buildDefaultValidatorFactory`. El parseo que necesitaría no falta: vive en `ZipInputStream`, donde sí puede correr

✓ **Éxito**: el paquete que mejor separa **lo que el gate puede decir de lo que no**. Cierra 21/21 con **207 match / 0 mismatch / 0 extra y CERO entradas de allowlist** —raro para un paquete de este tamaño—, pero lo que de verdad lo sostiene es que se validó **ejecutando contra el JDK: 747 checks** del inflater, **30** del deflater y **7** del contenedor. Esa validación encontró **dos bugs propios que el gate jamás habría visto**, que es exactamente el argumento de esta fase.

**H13 java.lang.invoke — donde la biblioteca se topa con la VM 19/19 (con la mitad declarada) **

  **MethodType (33 miembros, cero divergencias) — el trabajo de verdad**. El tipo de un handle con todas sus derivaciones (`insert / drop / change`, `erase`, `generic`, `wrap / unwrap`), y su `describeConstable()`, que lo convierte en el `MethodTypeDesc` de **H11**: el mismo dato, una vez con clases cargadas y otra con nombres. Salió gratis un truco: el descriptor de una `Class` se deriva de `getName()` **sola** —el JDK ya devuelve `int` para un primitivo y `[Ljava.lang.String;` para un array, así que los tres casos caen del primer carácter—, lo que evitó necesitar `isPrimitive / isArray`, que nuestra `Class` no tiene

  **Por qué invoke NO puede escribirse en Java**. Es **signature-polymorphic** (JLS §15.12.3): se declara una vez como `(Object...)Object`, pero el *call site* **no adapta sus argumentos** — el compilador emite un `invokevirtual` con el descriptor **del call site** y la VM lo linkea contra esa única declaración. Una llamada como `mh.invokeExact(1, "x")` compila a un método que no existe en ningún lado. La regla de resolución vive en la VM; por eso van `native`, igual que en el JDK

  **Lo que sí quedó completo** — `SerializedLambda` (la *receta* de un lambda: como su clase se genera en runtime no puede serializarse como sí mismo, así que viaja la receta), `MethodHandleInfo` (el gemelo cargado del enum `Kind` de H11), `SwitchPoint` y la familia `CallSite`, que son las tres respuestas a «¿puede cambiar el destino, y quién tiene que verlo?»: nunca, sí sin promesa de orden, o sí visible ya para todos

  **El caso feliz: TypeDescriptor sin allowlist**. Sus dos interfaces anidadas son *self-bounded* genéricas y daban **9 mismatches** por el #100 (variable acotada erasa a `Object`). Modelándolas **raw con los métodos devolviendo la cota** —el mismo truco que usó `jakarta.validation` con `Configuration<T> extends Configuration<T>`— el descriptor emitido queda **idéntico** al del JDK: nueve divergencias eliminadas, cero entradas de allowlist

 **Verificado antes de escribir: publicar estos nombres no rompe los lambdas**. `LambdaMetafactory`, `StringConcatFactory` y `ConstantBootstraps` son declaraciones que la VM **nunca carga** — nuestro `invokedynamic` los reconoce **por nombre** y sintetiza el comportamiento en Rust (el de lambdas incluso *spinea* la clase implementadora). Se comprobó en el intérprete antes de crear los archivos

✓ **Éxito: el hito cuyo número más engaña, y conviene decirlo de frente**. Cierra 19/19 con 172 match / 0 mismatch / 0 extra, pero **la mitad que importa no puede ser Java**: `MethodHandle.invoke` es *signature-polymorphic* y `Lookup` captura a su llamador caminando la pila. Lo que sí quedó real es el **tipo** de un handle y toda la familia de descriptors; el resto son declaraciones honestas que lanzan.

H14 Barrida de superficie contra OpenJDK 25 — seis paquetes y el gate de API **hecha**

Avanzado  **El gate (`tools/apidiff/`).** `recompile.py` rehace la biblioteca con el `javac` congelado en un árbol aparte; `apidiff.py` compara la superficie declarada contra el `javap` del JDK 25. Dos decisiones cambian el número: se mide sobre **bytecode recién emitido**, porque los `.class` versionados son artefactos de compiladores viejos y comparar contra ellos mezcla *qué dice la fuente* con *qué `javac` la compiló*; y el denominador es el **contrato** (`public` + `protected`), no todos los miembros — los privados de HotSpot no son API y por regla no se implementan, así que contarlos daba 38.7% donde el número honesto es 56%.

Avanzado  **La regla que separa lo obligatorio de lo libre.** Hay que respetar lo que el usuario puede observar: miembros `public` / `protected`, nombres y jerarquía de las clases públicas, y sus modificadores semánticos. Es **libre** todo lo inalcanzable: clases `package-private`, internos, nombres de helpers y algoritmos — nuestra implementación no tiene por qué parecerse a la de HotSpot, y de hecho no puede, porque no hay `Unsafe` ni `MemorySegment`. Aplicarla retiró dos findings propios que medían internos como si fueran API.

Base  **Seis paquetes ampliados.** `java.nio` miembros 34% a 82% y `ByteBuffer` **vuelve a ser abstracta** (18 métodos que estaban concretos, sin inventar ninguna clase); `java.lang.invoke` 58% a 97.6%; `java.util.stream` contrato 51% a 81% con `Collectors` de 4 a **37 factories**; `java.util.regex` **paridad exacta de API pública**; `java.text` 14 a 21 clases; `java.lang.reflect` 16 a 29. Y el modelo de APT completo: `javax.lang.model` 3/3, `javax.lang.model.type` **0/18 a 18/18** (el paquete no existía), `javax.lang.model.element` 4/21 a 21/21, `javax.tools` 2 a 16.

Base  **Cinco bugs reales que la superficie ocultaba.** El conteo de clases no los veía: `flatMap` **no funcionaba** (encadenar sobre el resultado de una llamada al classpath emitía bytecode roto), `Pattern` **guardaba los flags y los ignoraba** por completo, `Pattern.quote()` estaba roto, `MethodType.wrap()` era la identidad en silencio, y `Float / DoubleBuffer.compareTo` ordenaba `NaN` antes que todo.

Cumbre  **50 findings nuevos (#200 a #249), y los peores son silenciosos.** Del compilador: descarta **llamadas y concatenaciones enteras** sin diagnosticar (#114/#247/#248), pierde `ACC_STATIC` del classpath, `ACC_VOLATILE` y `ACC_VARARGS` en la emisión, y **degrada un tipo calificado de `java.lang` a `Object`** sin avisar (#210), con lo que un override deja de sobrescribir. De la VM: no aplica `ConstantValue`, así que **todo `static final` primitivo lee 0** (#216), y no despacha `invokevirtual` a un método resuelto `abstract` (#230).

Cumbre  **El hallazgo estructural (#246): hay dos bibliotecas estándar divergentes y la que se desarrolla no es la que corre.** La VM bootea contra `boot/`; `KajiLibrary/` se compila, se mide y se audita, pero **nunca se ejecuta**. Ya hay tres divergencias medidas: `Throwable` declara `backtrace` en `boot/` y no en `KajiLibrary` (toda excepción no capturada panica), e `Integer` tiene `equals / hashCode` en `boot/` y **cero** en `KajiLibrary` (un `HashMap` con claves `Integer` nunca encuentra nada). Mientras siga así, *pasa el gate y funciona* hablan de dos artefactos distintos — es la Fase E, cerrar el círculo.

 **Éxito:** la biblioteca deja de medirse por impresión y pasa a medirse contra el JDK real: **592 de 732 clases públicas (81%)** en los paquetes empezados, con **17 de 29 completos**, y un gate reproducible que cualquiera puede correr.

Fase I — Depurador (JVMTI / jdb) — el detalle de la Fase D

El **detalle accionable** del depurador de la **Fase D — Herramientas** (que da el overview del toolchain). Replicar el **toolchain** más allá de `javac / java`, empezando por el depurador. La espina es el **depurador**, y su arquitectura es el stack **JPDA**: **JVMTI** (los ganchos, en la VM) ← **JDWP** (protocolo de cable) → **JDI** (API cliente) ← **jdb**. Se construye sobre lo que ya existe — `jvm-step` ya para en cada opcode y muestra pila + locales, así que la introspección está casi hecha. **Dos caminos sobre el mismo JVMTI**: un **jdb pragmático** in-process (I1) y uno **fiel** por protocolo (I3–I4), con la **fidelidad** (I5) como cumbre — que un IDE real *attachee* sin notar que la VM es propia. `javap` ya está tildado (`javap -c Lon`); el resto del toolchain (`jar / javadoc / jshell`) queda como ampliación del hito.

I0 JVMTI mínimo — los ganchos en la VM **logrado** Avanzado

Avanzado  `trait JvmtiAgent` (callbacks `push: breakpoint / single_step / ...`) + `JvmtiEnv<'a>` — el handle **acotado** que el callback usa para volver a entrar a la VM (introspección + control). La lectura casi ya está: `frames() / pc() / locals() / thread_views()` del intérprete mapean 1:1 a las funciones de introspección de JVMTI

Avanzado  **Ganarle al borrow checker sin `unsafe`**: la VM **posee** al agente, pero al disparar un evento **destructura `self` en campos disjuntos** (`agent` ↓ los campos de `env`). Acotar el `env` es lo que lo hace posible — si fuera `&mut JVM` entero no compilaría

Avanzado  Hooks de `breakpoint + single_step` en `step()`, una `BreakpointTable`, y `Capabilities` con **fast-path** (`if !caps.any()`) al frente → cero overhead: «pagás por lo que activás», el modelo de JVMTI

✓ **Éxito: logrado** — un agente pide un breakpoint y recibe el evento en medio de la ejecución, sin que la VM pague nada cuando no hay agente atacheado. Módulo `jvmti.rs` (`JvmtiAgent / JvmtiEnv / BreakpointTable / Capabilities`) + `attach_agent / fire_debug_events` en la `JVM`, con el gancho al tope de `step()`; 3 tests (incl. uno de integración: un agente pone un breakpoint, corre `add(3,4)` en la JVM propia, recibe el evento y lee el local 0).

I1 `jdb` propio — front-end interactivo **logrado** Avanzado

Avanzado  Depurador CLI **in-process** (el prompt corre *dentro* del callback del agente, inspeccionando/controlando por el `JvmtiEnv` y devolviendo el control con `step / cont`). `command()` puro/testeable. El `JvmtiEnv` ganó lectura del **Method Area** (`method_name / code`) para mostrar nombres. Cola: `stop at C.m` por nombre, `list / disassembly`, `print` por nombre de variable (con la `LocalVariableTable`)

✓ **Éxito: logrado** — un binario `jdb` (in-process, sobre JVMTI de I0) frena la ejecución y abre un prompt: `step / cont, break / clear <pc>, where, locals, print <slot>, help, quit`. Verificado de punta a punta: `jdb Add.class add 3 4` frena en `add@0`, `where` muestra el nombre del método, `locals` da los args, `step` avanza, `cont` completa.

I2 Más eventos JVMTI **logrado (incl. field watchpoints)** Avanzado

Avanzado  `method_entry / method_exit / exception` — enganchados en los *chokepoints* centrales (`push_frame_locked / pop_frame / unwind_with`), con el mismo destructure disjunto y su *capability* que gatea el overhead; entry no dispara para el método de entrada ni `<clinit>`, exit salta sintéticos, exception cubre el `throw` explícito y las implícitas del VM. 2 tests de integración (trace `outer→inner`; `ArithmeticException` atrapada)

Avanzado  **Field watchpoints** (`field_access / field_modification`) — el gancho `fire_field_watchpoint` al tope de `step()` decodifica el opcode de campo (`getfield / putfield / getstatic / putstatic`) y resuelve el *fieldref read-only* (`cf.fieldref_target`), leyendo el objeto/valor de la pila —**sin tocar los handlers de opcode**, que otra sesión edita—. `FieldWatchTable` en el `env`. 2 tests de integración (un agente vigila `Watched.value`, corre `run()` y recibe la modificación con el nuevo valor y el acceso; sin watch, cero eventos). Cableado por JDWP en I5 (validado con `jdb watch`)

✓ **Éxito:** trazar entradas/salidas de método, atrapar excepciones y **vigilar campos**.

I3 JDWP — el protocolo de cable **logrado** Cumbre

Cumbre  **Codec** (`jdwp.rs`, puro/sin I/O): handshake de 14 bytes, framing de paquetes (header de 11), y una serialización tipada (`Writer / Reader`: byte/int/long/id de 8 bytes/string big-endian) — se testea de punta a punta sin socket

Cumbre  **Command handlers** (`JdwpSession`): `VirtualMachine.Version / IDSizes / Suspend / Resume` y `EventRequest.Set / Clear` (parsea `eventKind / suspendPolicy / modificadores`; del `LocationOnly` saca la posición y aloca el `requestID`). Puro: registra la intención, sin tocar la VM

Cumbre  **Transporte:** `handshake / serve / listen` genéricos sobre `Read + Write` — la VM como servidor `dt_socket` (server=y). El core no sabe de sockets → se testea con un stream en memoria

Cumbre  **Bridge JVMTI↔JDWP** (`bridge.rs`): un `JvmtiAgent` que **empuja** los eventos de la VM como *Composite packets* (1) y **aplica** los `EventRequest` del cliente a la VM real por el `JvmtiEnv` (!: un breakpoint pedido por cable **frena de verdad**). El mismo `serve_until_resume` corre en `vm_init` y tras cada evento; el truco del borrow otra vez (`session / stream` campos, `env` parámetro → disjuntos). Genérico sobre el transporte → 4 tests con un `MockStream` dúplex

Cumbre  `jvm-jdwp`: el binario servidor — `bind → accept → handshake → attach_agent(JdwpBridge)`. Equivale a `agentlib:jdwp=transport=dt_socket,server=y,suspend=y,address=<port>`. La VM arranca suspendida hasta el primer `Resume`. ☐ **Cola** (rumbo a I5): breakpoints por *ubicación* necesitan que el cliente resuelva el `methodID` real vía `ReferenceType.Methods / ClassesBySignature`, todavía no implementados — por eso el smoke test usó single-step

✓ **Éxito: logrado** — un cliente JDWP **externo** (otro proceso) habla con nuestra VM por el protocolo estándar sobre un socket TCP. Verificado end-to-end: un cliente en Python *attachea* a `jvm-jdwp java/Add.class add 3 4`, hace el handshake, pide un `EventRequest` de single-step, y recibe los **4 Composite events** (uno por opcode de `add`) frenando y reanudando por el cable, hasta `Some(Int(7))`. 12 tests del módulo `jdwp` + 4 del `bridge` + 3 del binario.

I4 JDI — la API cliente **logrado**

  **Módulo** `jdi.rs` — la API cliente de alto nivel, contraparte del *bridge* del lado del debugger. Depende **solo** del codec `jdwp` (ni VM ni JVMTI), así que un cliente y el `JdwpSession` servidor se prueban de punta a punta **en memoria** (un `Loopback` que es un servidor síncrono real). *Mirrors*: `Vm` (la conexión), `Location`, `EventRequest`, `Event` (ya decodificado). API: `attach` (handshake cliente), `version`, `resume / suspend`, `set_breakpoint / set_step_request / clear`, `next_event` (drena una cola: los eventos que llegan mientras se espera una respuesta **no se pierden**)

  Queda explícito que el **jdb pragmático** (I1, in-process) y el **jdb fiel** (I3+I4, por protocolo) son **dos front-ends sobre el mismo JVMTI**. 
Cola: empaquetar un `jdb` interactivo completo sobre la JDI (hoy `jdi-attach` es el front-end de demostración)

✓ **Éxito: logrado** — el front-end habla con la VM por *mirrors* tipados (`Vm / Location / Event`) en vez de bytes de protocolo, y un cliente propio (`jdi-attach`) maneja la ejecución por socket. Verificado **dos procesos por el cable, todo nuestro** (reemplaza al cliente de Python): `jdi-attach ↔ jvm-jdwp` sobre `Add.add(3,4) → Version, requestID, los 4 eventos single-step y Some(Int(7))`. 4 tests del módulo `jdi` + 3 del binario.

I5 Fidelidad del debugger **logrado (validado vs jdb real)**

  **I5a — inspección de pila y valores**: `VirtualMachine.AllThreads`, `ThreadReference.Name/Frames/FrameCount`, `StackFrame.GetValues`. Un cliente parado ve la **call stack** (frames como `frameID = profundidad + location`) y las **variables** (locales tageados; `float / double` por sus bits). Los sirve el *bridge* leyendo el `JvmtiEnv` que ya existía —sin tocar la inmutabilidad del metaspace—; un dispatcher `reply_for` los intercepta y delega el resto a la sesión pura

  **I5b — breakpoints por línea**: `AllClasses / ClassesBySignature`, `ReferenceType.Signature/Methods/SourceFile`, `Method.LineTable`. El cliente traduce `Add.java:3 a (methodID, índice)`. **El muro y su salida**: listar métodos con su `MethodID` real exige resolverlos (`&mut metaspace`), pero en un callback el metaspace es inmutable → un **snapshot** capturado al attachear; como `resolve_method` **cachea**, el `methodID` del snapshot es *el mismo* que la VM usa al correr, por eso el breakpoint **frena de verdad**

  **Fidelidad para un JDI real** (lo que destrabó el attach de `jdb`, hallado iterando contra él): las variantes `WithGeneric` que emite el JDI de Java 5+ (`AllClassesWithGeneric / MethodsWithGeneric / SignatureWithGeneric`); la **negociación** (`Capabilities / CapabilitiesNew`, `ClassPaths`, thread groups, `Dispose`); el evento automático `VMStart` (para que fije su «hilo actual»); y los **ids no-cero** —JDWP reserva el `0` para *null*—: `classID`, `threadID` y `methodID` = `MethodID + 1` (sin esto, `<obsolete> / line=-1 / Invalid frame location`)

  **locals** (`Method.VariableTable`) — con la VM parada, `jdb locals` muestra los argumentos y las variables **por nombre y valor** (`a = 3, sum = 7`). Lee la `LocalVariableTable` del `.class` (solo con `javac -g`) capturada en el snapshot; de paso destapó un bug real: `StackFrame.GetValues` lleva `threadID` y `frameID`, y solo leíamos el frame

  **Field watchpoints + inspección de objetos** — `jdb watch Watched.value` pone el watch por protocolo (`ReferenceType.Fields`, modificador `FieldOnly`, caps `canWatchField`) y muestra el evento **completo**: `Field (Watched.value) is 0, will be 7`. El «is 0»* (valor **actual**) sale de leer el objeto del heap: el `JvmtiEnv` ganó `heap read-only`, con `ObjectReference.ReferenceType/GetValues` (clase real del objeto por el mirror del header; valor del campo por su offset precomputado). Cada gap se halló iterando contra `jdb` (Superclass, Interfaces, las caps, el `threadID` de `GetValues`)

  **Sesión de jdb 100% limpia**: `threads` (`Group main: 1 main running`), `classes`, `stop at / cont`, `where`, `locals`, `watch` — todos sin errores. Lo único fuera de alcance es correr el IntelliJ **gráfico** (no headless acá); `jdb` es el mismo `com.sun.jdi` que hay debajo. **El hito I queda completo. 838 tests** en verde

✓ **Éxito: logrado** — el **jdb de referencia de Temurín** (el mismo `com.sun.jdi` que usa IntelliJ Platform) attachear a nuestra VM por `dt_socket` **sin saber que es propia**, pone un breakpoint **por línea de fuente** y lo **frena**. Salida real de esa sesión: `VM Started: Add.add(), line=3 bci=0 · Breakpoint hit: Add.add(), line=3 bci=0 · where → [1] Add.add (Add.java:3)`. Es *differential testing* contra el debugger real como oráculo, el espejo de la «Fidelidad de `javac`» (B6) para el depurador.

Fase J — `jlink`: la imagen de runtime — el detalle de la Fase D

El **detalle accionable** de `jlink`, que la **Fase D — Herramientas** listaba como «largo plazo». `jlink` no necesita un `java.base` completo, necesita uno **bien definido**: resuelve un grafo de módulos y empaqueta lo alcanzable, así que con la biblioteca que tengamos sale una imagen más chica y nada más. Cada hito se verifica **diferencialmente** contra la herramienta real (`java --describe-module, jlink, jimage`), igual que el parser contra `javap`.

J0 `ModuleDescriptor` — leer el módulo ✓ logrado Base

Base `ModuleDescriptor` en `parser/attributes/module.rs`: `Requires{transitive, static_phase, mandated}`, `Exports{package, to}`, `Provides{service, with}`, más `ModulePackages / ModuleMainClass`. Mismo corte que en `annotations.rs`: los `print` siguen sirviendo a `javap`, y al lado viven los **accessors** que las herramientas usan para *preguntarle* cosas al descriptor en vez de imprimirlo.

Base El círculo propio cierra: `module-info.java` de nuestro `java.base` → compilado por `bin/javac-frozen.exe` → descrito por nuestro `jlink`.

Base Fixture `java/kaji.sample/` — un módulo *exploded* de verdad (el **nombre del directorio es el nombre del módulo**) que ejercita **todas** las directivas: `open module`, `requires transitive / static`, `exports / opens` cualificados y no, `uses`, `provides ... with` múltiple. Confirmado bit a bit contra `javap -v (ACC_OPEN, ACC_TRANSITIVE, ACC_STATIC_PHASE)`.

✓ **Éxito: logrado** — `jlink --describe-module` sobre el `module-info.class` real de `java.base` da **233 líneas idénticas** a `java --describe-module java.base` (el descriptor más complejo que existe: 100+ `exports`, cualificados, `uses`, `provides`). Normalizando sólo CRLF y el orden de las listas `to`, que el tool real emite en orden de iteración de un `Set` — no es orden especificado.

J1 Resolución del grafo (JPMS) ✓ logrado Avanzado

Avanzado **Alcanzabilidad**: cierre transitivo sobre `requires` desde las raíces, con el módulo faltante nombrando **quién lo requería**. `requires static` **no se resuelve** — es una dependencia de compilación, y que falte en `link/run` es legal.

Avanzado **Legibilidad implícita**, que es lo jugoso: la relación `reads` es **más grande** que las aristas `requires` de las que sale. `requires transitive N` propaga — quien me lee, lee `N`. Verificado en el JDK real: `java.sql.rowset` **no requiere** `java.xml` ni `java.transaction.xa` pero **los lee**, porque `java.sql` los requiere `transitive`.

Avanzado **Split packages** como error de link: dos módulos resueltos con el mismo paquete harían ambigua la carga. Se chequea sobre el conjunto **resuelto** — dos módulos pueden estar bien por separado y no juntos en una imagen.

Avanzado `Configuration::reads(from,to)` y `exports_to(to,pkg,from)` — la base de J5: **legibilidad no es acceso**, el dueño además tiene que haber exportado el paquete.

✓ **Éxito: logrado** — el conjunto resuelto coincide con el del `jlink` **real en 68 de 68 raíces comparables** (las 2 restantes, `jdk.jlink / jdk.jpackage`, el tool de referencia las rechaza por venir sin `jmods`, no por diferencia de resolución) más 3 casos multi-raíz. Módulo `jvm/modules.rs`.

J2 Imagen *exploded* ✓ logrado Avanzado

Avanzado Layout `<out>/modules/<módulo>/...` — la forma *exploded* del propio JDK — más el `release` en la raíz. El tool real mete el mismo contenido en el contenedor `lib/modules`: **mismo contenido, distinto contenedor**, que es exactamente la costura donde entran J3 y J4.

Avanzado `release` **verificado contra el real**: linkeando las mismas raíces del JDK, nuestra línea `MODULES="java.base java.logging"` sale **idéntica** a la del `jlink` de Temurín. Decisión propia: **se omite** `JAVA_VERSION` cuando los módulos no traen versión, porque un `JAVA_VERSION=""` *afirma* una versión vacía en vez de admitir que no hay.

Avanzado El descriptor de nuestro `java.base` vive ahora en `Kajilibrary/module-info.java`, junto a la biblioteca que describe; `run-headless --boot <dir>` es lo que permite comprobar que la imagen sirve de runtime.

Avanzado *Gotcha del camino*: un module path copiado a medias (236 de 427 clases) se manifestó como `invokeinterface: could not resolve the receiver's class` — parecía un bug de la VM y era `ArrayList.class` que sencillamente no estaba en la imagen. Un recordatorio de que en una imagen **lo que falta no da «no encontrado», da errores raros río abajo**.

✓ **Éxito: logrado** — `jlink --module-path ... --add-modules java.base --output img/` produce una imagen de **427 clases** y un **programa corre desde ella**: `ImgSmoke` (que usa `ArrayList`, `StringBuilder` y `String`) devuelve 49, y `Add.add(3,4)` devuelve 7, con el bootclasspath apuntando a `img/modules/java.base`. Ese `java.base` es **el nuestro**: una imagen de runtime producida por nuestro toolchain, con nuestra biblioteca, ejecutando nuestro bytecode.

J3 `jimage` — el lector **logrado**

Cumbre **Dos endianness en un mismo archivo:** la cabecera son 7 `u32` **little-endian** (a diferencia del `.class`, que es big-endian), pero los valores de atributo dentro de `locations` son **big-endian**. Leer una mitad con el criterio de la otra produce basura que *parece* plausible — el tipo de error que no se cae, se desvía.

Cumbre `locations`: cada atributo es un tag `(kind << 3) | (len-1)` seguido de `len` bytes. `strings`: cadenas NUL-terminadas, y los nombres se guardan **partidos en cuatro** (módulo · parent · base · extensión). Compartir esos pedazos es lo que hace que 30473 nombres entren en 680 KB.

Cumbre **El orden de los módulos no es por nombre sino por directorio.** La referencia pone `java.management.rmi` antes que `java.management`; son seis pares y todos del mismo tipo. La clave es `nombre/`: comparando `java.management/` con `java.management.rmi/`, en esa posición hay `/` (47) contra `.` (46), y `.` es menor — así que el módulo más largo va primero. Ordenar por el nombre pelado da exactamente el orden inverso en esos seis.

Cumbre **Entradas meta**: la imagen guarda 2358 entradas (70 módulos + ~2288 paquetes) que se describen a sí misma y que `list` no muestra. Por eso `resource_count` (30473) no es la cantidad de recursos (28115).

Cumbre **Lo que NO está:** la compresión (todo recurso de un JDK de fábrica trae `compressed == 0`; una imagen con `jlink --compress` necesitaría el descompresor de J6 — el código *reporta* la entrada comprimida en vez de escribirla mal), y el **hash perfecto**, que está parseado pero todavía sin usar: `list/extract` recorren la tabla entera. La búsqueda O(1) por nombre es lo que J4 tiene que **construir**, y por eso el lector va primero.

✓ **Éxito: logrado** — los tres comandos verificados contra el `jimage` real sobre la imagen de 145 MB: `info` idéntico (10 campos), `list` idéntico (28256 líneas · 70 módulos · 28115 recursos), `list --verbose` idéntico (offset/size/compressed de las 28115), y `extract` con **55 archivos byte a byte idénticos** en un módulo completo más **28115 extraídos con 0 faltantes y 0 de tamaño distinto**. Módulo `jvm/jimage.rs`, binario `bin/jimage.rs`.

J4 `jimage` — el escritor **logrado**

Cumbre **Primero descubrir la función de hash, no asumirla:** FNV-1a sobre los bytes del nombre con el bit de signo limpiado, y `0x01000193` de multiplicador y de semilla inicial. La validación fue implementar la búsqueda y correrla contra la tabla ya construida de la imagen real: **30473 de 30473** entradas se encuentran a sí mismas, y un nombre inventado da `None` — confirmar el hit es lo que evita devolver el recurso de otro.

Cumbre Las dos primeras fallas fueron las reveladoras: `/modules` y `/packages`, las entradas **raíz**, no tienen módulo y guardan el nombre completo en `base`, así que componérselo duplicaba los separadores.

Cumbre **Construirlo es el problema inverso:** los buckets de un solo nombre se resuelven directo (índice negado) y los que colisionan necesitan una **semilla** que los desparrame a slots libres distintos; se resuelven los grandes primero, mientras queda espacio.

Cumbre **El bug de verdad del hito: la búsqueda de semilla no terminaba.** Con factor de carga 1.0 los últimos buckets casi no tienen slots libres, y para algunos conjuntos **ninguna** semilla los ubica — un test consumió **1181 segundos de CPU** antes de que se notara. El arreglo es acotar los intentos y **agrandar la tabla** si no cierra, que es exactamente para lo que `table_length` y `resource_count` son campos separados en la cabecera. El test que colgaba ahora resuelve en 0.01 s.

✓ **Éxito: logrado** — `jlink --jimage` escribe un `lib/modules` de verdad y el `jimage` de Temurin lo lee: `info` da los 10 campos idénticos, `list` las 428 entradas, y `extract` devuelve los 428 archivos **byte a byte idénticos al origen**.

J5 Imagen ejecutable — JPMS en runtime **logrado**

Cumbre `ClassFile::from_bytes` separa el parseo de la lectura del archivo — el requisito de fondo: una clase que **no viene de un archivo**.

Cumbre `BootImage` indexa la imagen por **nombre binario**: la imagen se indexa por nombre completo (`/módulo/ruta`) pero un loader pide `java/lang/Object` y **no sabe en qué módulo está**, así que el mapa se arma una vez al abrir y cargar cuesta un lookup.

Cumbre La imagen es parte del loader **bootstrap**, consultada *después* de sus directorios (uno todavía puede sombrearla). Que la defina el mismo loader mantiene la identidad de runtime `(nombre, loader)` igual, se haya booteado de directorios o de imagen.

Cumbre **La accesibilidad, aplicada de verdad.** La regla pide **dos** condiciones y hacen falta las dos —el par que se confunde—: el módulo que llama tiene que **leer** al otro y el otro tiene que **exportar** el paquete. Leer no alcanza: un módulo puede leer a otro y seguir sin poder tocar sus paquetes internos.

Cumbre Para que sea chequeable la imagen **recuerda el dueño de cada clase**; sin eso una clase es sólo un nombre sin dueño. Los `module-info` van en un mapa aparte porque **todos comparten el mismo nombre binario** y en el de clases se pisaban entre sí — con dos módulos sobrevivía uno solo (bug encontrado con un test).

Cumbre El chequeo se aplica en los **9 sitios de resolución**: `new`, `getstatic/putstatic`, los cuatro accesos a campos de instancia, los tres `invoke` y *el literal de clase de `Ldc`*. Aplicarlo en uno solo se leería como encapsulación dejando el resto abierto. En `invokevirtual/invokeinterface` se chequea la **referencia simbólica** —la clase que nombra el pool—, no la del receptor en runtime: es la resolución*, no el despacho.

Cumbre **Verificado ejecutando**, con dos clases en la misma imagen y `kaji.app` requiriendo a `kaji.api` en ambos casos: `UsesApi` da **7** y `UsesSecret` muere con `IllegalAccessError`. La única diferencia es que un paquete está exportado y el otro no — y **las dos compilan sin queja**, porque nuestro javac no aplica JPMS: la regla es de runtime.

Cumbre Sin grafo de módulos **todo se permite**, con test propio: una VM booteada de directorios no tiene módulos, y negar accesos ahí rompería todo el uso normal.

✓ **Éxito: La VM bootea desde un `jimage`**. Verificado con los directorios de boot **vacíos**, para que no queden dudas del origen: `ImgSmoke` (que usa `ArrayList`, `StringBuilder` y `String`) da **49** cargando sólo de nuestra imagen de 427 clases, y `Add.add(3,4)` da **7** cargando sólo de `lib/modules` de Temurin — 27084 clases, o sea nuestra VM leyendo la imagen real de un JDK de 145 MB con el lector de J3.

J6 Plugins del pipeline **logrado** **Cumbre**

Cumbre `--strip-debug` es el que tiene sustancia: cirugía a nivel de bytes que saca cuatro atributos (`LineNumberTable`, `LocalVariableTable`, `LocalVariableTypeTable`, `SourceFile`) recalculando **cada longitud envolvente, incluida la de `Code`**, porque los de debug van anidados adentro. Verificado con el `javap` real sobre una clase extraída de la imagen: **0** atributos de debug, `Code` y `StackMapTable` intactos, y `ImgSmoke` **sigue dando 49** — quitar debug no cambia la semántica.

Cumbre Decisión documentada: **el constant pool se copia tal cual**, así que los `Utf8` de los atributos borrados quedan sin referencias (`#16 = Utf8 LineNumberTable`). Renumerar el pool costaría mucho más que los bytes que ahorra, y el tool real hace lo mismo; hay un test que lo fija como intencional.

Cumbre `--compress`: el formato del *recurso comprimido* se descifró contra la imagen real — `0xCAFEFAFA`, `content_size`, `uncompressed_size`, offset al nombre (`"zip"`), flags, `is_terminal`, y después un stream **zlib**—; la cuenta cerraba sola, $29 + 1032 = 1061$, el `compressed` que reporta el índice. Se emite zlib con bloques DEFLATE **stored**, que es exactamente el `zip-0` que el tool real documenta como «No compression»: un stream válido que cualquier inflater acepta, y que simplemente no encoge. El crecimiento da la cuenta exacta: $+18410 = 428 \times 43$.

Cumbre Y cierra la limitación que J3 había anotado: `BootImage` **descomprime al servir una clase**, que es lo que hace booteable una imagen comprimida — sin eso la VM fallaba con `invokeinterface: could not resolve the receiver's class`, el mismo síntoma engañoso de un module path incompleto.

Cumbre **Y se escribió el inflate completo** (RFC 1951 + el envoltorio zlib): bloques *stored*, Huffman **fijo** y Huffman **dinámico**, con las tablas canónicas y las referencias LZ77. Con eso la VM lee imágenes comprimidas **de verdad**: extraer la imagen `zip-6` real del JDK da **7440 de 7440** recursos, 7438 byte a byte idénticos a la referencia. Los 2 restantes no son un fallo — `SystemModules$all` y `SystemModulesMap` son clases que **jlink genera por imagen** para codificar su grafo, y la nuestra tiene 2 módulos donde la de referencia tiene 70.

Cumbre Dos cosas del formato se equivocan fácil y fallan **en silencio**: el stream se lee **LSB-first** pero un código Huffman va empaquetado **MSB-first** — los dos órdenes conviven—, y una *back-reference* puede **solaparse** con lo que todavía está escribiendo (distancia menor que longitud), así que hay que copiar byte a byte; copiar el rango de una vez da otra cosa. Es justo lo que hace que una tirada `aaaa...` cueste tres bytes.

Cumbre **La consecuencia cruzada se cumplió**: `--strip-debug` borra justo lo que lee el `locals` de nuestro `jdb` (Fase I). Un hito que rompe a otro, igual que en el JDK real.

✓ Éxito: logrado — los cuatro plugins. `--strip-debug` baja la imagen de **828588 a 693048 bytes** (−16 %), y `--compress` escribe recursos comprimidos que **el `jimage` de Temurin descomprime**, devolviendo los 428 archivos byte a byte idénticos al origen.

Estado actual

Fase A / Hitos A0–A5 logrados; A6 en progreso. Fase B / front-end + semántica (pasadas 1 y 2) + chequeo de declaraciones + análisis de flujo + desugar + Codegen (núcleo completo, con StackMapTable e `invokedynamic`). El proyecto compila **sin warnings**, pasa **838 tests** (JVM + el compilador propio), produce **fixtures byte-idénticos** a `javap` y ya **ejecuta bytecode real con objetos, GC generacional, green threads, monitores y un sistema de tipos completo**: aritmética, control de flujo, recursión, `switch` (`tableswitch`/`lookupswitch`), un **heap** con objetos/herencia/campos/arrays, **dispatch dinámico**, **excepciones**, **inicialización de clases**, **class loaders**, un **recolector generacional** (young por copia + Old mark-sweep-compact, con **referencias débiles** de `java.lang.ref`), **hilos** sobre tres sustratos (verde cooperativo, hilos de SO con **GIL**, y hilos de SO en **paralelismo real**) con **monitores** (`synchronized` de bloques y métodos, `wait`/`notify`), y un **verificador de bytecode JVMs-estricto**.

Al día (2026-08-13). El motor se unificó con la línea avanzada del repositorio remoto: la concurrencia dejó de ser sólo *green threads* y hoy corre sobre **tres sustratos** (verde cooperativo · hilos de SO serializados por un **GIL** · hilos de SO en **paralelismo real** con *safe-points* stop-the-world), con `invokedynamic` completo y las arenas `eden`/`atomic_region`. El **depurador** (JVMTI/JDWP/JDI) se reapió sobre esa arquitectura: el `jdb` real de Temurin lo maneja end-to-end — breakpoints por línea, `where`, `locals` y `threads` multi-hilo enumerando el registro real de hilos. La **fusión de `java.lang`** —que KajiLibrary sea la biblioteca única y `boot/` desaparezca— está **parcial**: el hueco bajó de 22 miembros a 7, y 6 de esos son divergencias deliberadas (ctors privados donde el otro los tenía públicos, sintéticos del enum, `Enum.valueOf` ausente por diseño); el único real es el flag `ACC_VOLATILE` de `Thread.interrupted`. Los dos bloqueos que restan son del **compilador**, no de la biblioteca: el desugaring del `for-each` era el retorno genérico a `Object` (finding #113) y `volatile` se pierde al emitir (#115).

Arrancó la Fase J — `jlink`. Como esos dos bloqueos son del track del compilador, el trabajo se movió a lo que no depende de ellos: la imagen de runtime. **J0** (leer un `module-info.class` a `ModuleDescriptor`) y **J1** (resolver el grafo JPMS) están **logrados y verificados diferencialmente** contra el JDK real: el descriptor de `java.base` sale idéntico a `java --describe-module` (233 líneas), y el conjunto que resolvemos coincide con el del `jlink` real en las **68 raíces comparables** del JDK. La clave conceptual: `jlink` no necesita un `java.base` completo —le importa el **grafo**, no cuántos métodos tenga la biblioteca —, así que es trabajo desbloqueado. Con **J2** el círculo se cierra: `jlink` ya **produce una imagen** de 427 clases con nuestro `java.base`, y un programa que usa `ArrayList`/`StringBuilder` **corre desde ella**. Con **J3** sabemos *leer* ese contenedor: los tres comandos de `jimage` salen idénticos a los del tool real sobre la imagen de 145 MB del JDK. Y con **J4** también lo escribimos: el `jimage` de Temurin lee las imágenes que produce nuestro `jlink`, y sus `extract` salen byte a byte idénticos al origen. **J5** hace que la VM **bootee desde una imagen** —incluso desde la de un JDK real—, y **J6** agrega el pipeline de plugins. La **Fase J queda cerrada**: la VM bootea desde una imagen y hace valer la encapsulación de JPMS (un paquete no exportado da `IllegalAccessError` al ejecutar, aunque compile), y los cuatro plugins andan. Y el **inflaté propio** (DEFLATE completo, escrito de cero) cierra la última pieza: la VM lee imágenes comprimidas reales — los 7440 recursos de una imagen `zip-6` del JDK salen byte a byte. Lo único que queda afuera es *comprimir* de verdad, que pide un encoder LZ77 + Huffman; escribir sigue siendo `zip-0`.

Al día (2026-08-18) — la biblioteca pasó de «forma correcta» a «corre de verdad». Tres hitos nuevos cerraron en KajiLibrary. **H9** — `java.util.concurrent` entero **en Java puro**, con sólo dos natives agregados (`Thread.currentThread()` y `Object.wait(long)`): la observación que lo habilita es que, en una VM cuyos hilos se intercalan **entre opcodes**, un atómico hecho con `synchronized` es *indistinguible* de uno lock-free con CAS. **H10** — `jakarta.persistence` (JPA 3.2) quedó en **990 match / 0 mismatch / 0 extra** sobre 204 clases. Y **H11** llenó los huecos de `java.base` con una **flota de agentes en paralelo** (~200 clases): `java.util.function` quedó **43/43**, y `java.util`/`java.io`/`java.lang` subieron fuerte. **La biblioteca entera pasa el gate: 3074 match / 0 mismatch / 0 extra, 47 allowed, PASS sobre 681 clases públicas.**

Pero el cambio importante del período es de método, no de volumen. Hasta ahora la biblioteca se validaba comparando **forma de API** contra el JDK y corriendo el algoritmo portado en **Java real**. Las dos cosas resultaron insuficientes de una forma que conviene dejar escrita: nuestro `TimeUnit.cvt` estaba **validado 112/112 contra el JDK** y sin embargo **toda** conversión de grueso a fino *trapeaba* en nuestra VM (leía `Long.MAX_VALUE` con `getField`, finding #110) — sólo la forma `src == dst` esquivaba el bug, que es justo la que cubría la validación. Por eso H9 se apoya en **24 tests de comportamiento que corren la biblioteca sobre el intérprete real**, y por eso `java.util.concurrent` es hoy el **único paquete verificado end-to-end**: `java.time` figura 100% en forma y **nunca se ejecutó**. La regla que queda: **validar en Java real no prueba nada sobre nuestra VM**.

Y el dogfooding devolvió un diagnóstico, no sólo una lista de bugs. Seis findings del compilador (**#108, #111, #114, #118, #119, #120**) comparten una única raíz: **cundo la resolución falla, el compilador emite silencio en vez de un error** — la llamada desaparece del bytecode y deja operandos colgados o la pila vacía, así que el `.class` pasa el gate de forma y explota (o peor, calla) en ejecución. Cuatro de ellos son puntos ciegos del **lector de classpath** (`Exceptions` #104, `ACC_STATIC` #110, `ACC_VARARGS` #118, la tabla de métodos del supertipo #120): lo que *escribimos* está bien, lo que *leemos* se interpreta mal. La recomendación para el track del compilador cabe en una línea — **una llamada que no resuelve tiene que ser error de compilación**—: eso convierte seis bugs mudos en seis mensajes. Caso testigo: `WeakHashMap` **gatea limpio y no puede correr** (#119), sin workaround posible desde el fuente. Y el #101 resultó tener **dos mitades**, de las que solo una estaba documentada: como *tipo*, un anidado importado funciona —aunque erase a `Object` en el descriptor—, pero como **calificador de un miembro estático es inalcanzable desde fuera de su archivo por cualquier vía**. Las tres formas fallan igual (`Outer.Nested.miembro`, el anidado importado, y el `import static` de la constante), lo que costó omitir `MethodHandleDesc.ofConstructor` y los diez `BSM_*` de `ConstantDescs`: todos necesitan el *valor* `Kind.STATIC`, y no hay forma de escribirlo. Del lado del runtime queda un bug de una línea con impacto real: un `wait(ms)` que **expira** no se saca del *wait-set* del monitor, y un `notify` posterior termina bloqueando al hilo que corre.

Dos defectos más del compilador, y los dos aparecieron por escribir código normal. El **#125** es que el emisor **no soporta `super.metodo(...)`**: el `super(...)` de constructor anda, pero el acceso calificado por `super` a un método —un *invokespecial* sobre `this` con el método del supertipo— no. Saltó apenas se escribió GZIP, y no por casualidad: **es la forma canónica de «extender sin reemplazar»**, así que aparece en cuanto hay una jerarquía de decoradores. Lo incómodo es que **invalida un workaround que el propio archivo de findings recomendaba** («evitarlos o usar `super.m()`»). El **#126** es que el **retorno de un override contra un supertipo del classpath se erasa a `Object`: `CallSite` declara `abstract MethodHandle getTarget()` y compilada sola emite el descriptor correcto, pero el override en `MutableCallSite` emite `()Ljava/lang/Object;`. Lo revelador es que **el parámetro del mismo tipo sale bien** —`setTarget(MethodHandle)` es correcto en esa misma clase—, así que el tipo resuelve perfecto y lo que se pierde es específicamente el retorno *cundo el método es un override*. Es familia del #123, pero más amplio: ahí había covarianza y acá el tipo de retorno es idéntico al del supertipo.**

De la anécdota a la medición. «El gate no prueba que funcione» era una intuición repetida; ahora es un número. Dos scripts (`tools/scan110.py` y `scan112.py`, versionados como chequeo de regresión) recorren el **bytecode ya emitido** y responden dos preguntas concretas: qué `getField` apunta a un

campo que en su clase dueña es `static` (#110), y qué constante `static final` viaja solo en el atributo `ConstantValue` sin `<clinit>` que la asigne (#112). El resultado: **26 clases** tocadas por el #110 —de 94 accesos cross-class, 36 pares (clase, campo) mal emitidos— y **14** por el #112, con 65 constantes que **valen 0 en runtime**. Unidas, y sumando `WeakHashMap`: **35 clases que gatean limpio y están muertas al ejecutar**. El #110 está brutalmente concentrado —**21 de las 26 son `java.time`**, leyendo constantes de `ChronoField` / `ChronoUnit` —, lo que confirma con evidencia lo que el hito H4 no podía ver: el paquete figura 100% en forma y nunca corrió. El #112 es más chico y de peor carácter: **no trapea, devuelve 0 en silencio** — las 12 constantes de `Modifier` hacen que `isPublic` / `isStatic` mientan sin avisar.

Los dos arreglados, en el compilador. El #110 era una línea que tiraba información: el lector de `.class` leía los *access flags* de cada campo y los descartaba (`r.u2()?; // access`), así que un campo del classpath nacía sin `ACC_STATIC` y el emisor —que decide bien— elegía `getField`. Ahora el flag se conserva y viaja al símbolo. El #112 se arregló del lado que hace javac: **plegar la constante en el punto de uso** (JLS §13.1), tanto para las declaradas en la unidad como para las que vienen del classpath, para lo cual el lector también pasó a leer el atributo `ConstantValue`. Ambos cambios llevan un comentario que explica el defecto, porque los mergea otra sesión. **Consecuencia operativa:** lo compilado antes del fix sigue roto — hay que **recompilar `java.time` entero**, la misma nota que ya existía para el #21.

- **ClassReader** (cursor big-endian) y **constant pool** completo (`ConstantPoolEntry`, 17 tags + `Tombstone`), con árbol de referencias en `pretty_class_visualizer.rs`.
- **Header:** versiones, `access_flags` (+ métodos `is_*`), `this_class` / `super_class` con `class_name().from_path() → Result` (sin panics).
- **Code** con **desensamblado completo** (opcodes + `tableswitch` / `lookupswitch` / `wide`) y comentarios `// ...` resueltos.
- **StackMapTable** (los 7 frame types), cada frame en su clase bajo `parser/stack_map_table/`, con `verification_type_info`.
- **javap brief y -v byte-identicos** (incl. cabecera Classfile / Last modified / SHA-256). Flags de visibilidad (`-public` / `-protected` / `-package` / `-p`).
- Renderers factorizados en `parser/printers/`: `verbose` (orquestación) + `file_header` + `pool_comments` + `member_dump` + `brief` + `dump_common` + `visibility`.

Pendiente de A0 (no esencial, no bloquea avanzar): atributos `Signature` (reescribe la declaración → parser de firmas genéricas), `BootstrapMethods`, `InnerClasses` / `EnclosingMethod`, `Exceptions`, `ConstantValue`, anotaciones, `Record`, `PermittedSubclasses`, `NestHost` / `NestMembers`, `LocalVariableTable` / `Type`; y flags de contenido (`-c` / `-l` / `-s`).

A1–A2 — el intérprete. Loop de despacho sobre una **pila de frames**; cada frame referencia su bytecode por índice (`MethodId`) en el *Method Area* (`metaspace`, con resolución de llamadas por el código del constant pool). Opcodes: `iconst` / `iload` / `istore`, `iadd` / `isub` / `imul`, `goto` / `if_icmpgt`, `invokestatic` / `ireturn`. Corre **factorial** y **fibonacci recursivos**. Visualizador `jvm-step` paso a paso con vista de dos paneles de la call stack.

A3 — objetos y heap. El **heap** es una arena de bytes (`Vec<u8>`) con `malloc` bump (sin liberar; el GC llega en A5). Los objetos se disponen como `[class_id | mark | campos]` con *layout* que respeta la herencia; cada clase recibe un **Class ID** (UUID) y un *mirror* en el heap para sus estáticos (estilo HotSpot). Opcodes: `new`, `dup`, `getField` / `putfield`, `aload` / `astore`, los cuatro `invoke*` y la familia de arrays (`newarray` / `anewarray`, `arraylength`, `iaload` / `iastore` / `aaload` / `aastore` + `byte/char/short`). El **dispatch dinámico** usa *vtables* por clase (slot del tipo estático, método del tipo real) e *itable* para interfaces — verificado: `Animal a = new Dog(); a.sound() → Dog.sound`.

A4 — robustez del runtime. Excepciones: `athrow` + tabla de excepciones + *stack unwinding* por la pila de frames, con `finally` (catch-all + re-throw) y las **implícitas** que la VM sintetiza (NPE, índice fuera de rango, cast inválido, tamaño negativo). **Verificador** de bytecode (type-checking en una pasada con el `StackMapTable`) que corre antes de ejecutar. **Inicialización** de clases `<clinit>` perezosa y super-primero. **Class loaders** bootstrap/app con delegación parent-first (las `java.lang.*` propias viven en `boot/`). Resolución que falla con **errores de linkage** (`NoClassDefFoundError` / `NoSuchMethodError`) en vez de paniquear. **Robustez fina cerrada (2026-08):** `Throwable` con `getMessage` / `toString` y **stack traces** capturados por la VM en el throw; `ExceptionInInitializerError` (JVMS §5.5: un `<clinit>` que lanza propaga, se envuelve, y deja la clase *erroneous* → `NoClassDefFoundError` al reuso); **default methods** de interfaces (JSR 335 — la vtable fusiona superinterfaces, regla *maximally-specific*). Pendiente fino: chequeos de acceso e identidad (`nombre`, `loader`) (no producibles con javac — ver `holdon.md`).

A5 — nativos + GC. Puente nativo (`System.out.println` imprime de verdad) e *intrinsic*s (`getClass`, `hashCode`, `Math`, `arraycopy`, `String` / `Class`). **Recolector de basura** que arrancó en mark & sweep y creció a un colector con **compactación** (mover objetos + reescribir punteros), **free list** con *coalescing*, **política de fragmentación** configurable, **disparadores** (out-of-space / occupancy / allocation-rate / explícito) sobre un *safe point*, y **marcado transitivo** correcto (sigue campos, arrays y estáticos).

A6 (en progreso) — cumbre del runtime. Verificador de bytecode JVMS-estricto: además de la cobertura completa de opcodes (incluido `switch`), verifica **con o sin `StackMapTable`** — inferencia por punto fijo como fallback, con *cross-check* de la tabla contra la inferencia — sobre un **gate estructural** (§4.9), con **tipos de locales** estrictos + `max_stack`, reglas de **objetos sin inicializar**, **handlers de excepción** tipados y **acceso/linkage** (regla `protected`, `count` de `invokeinterface`). **Sistema de tipos completo:** los cuatro tipos numéricos (`int` / `long` / `double` / `float`) **ejecutados y verificados** — cómputo, conversiones, comparaciones, división con `ArithmeticException`, **categoría-2** (params, campos, estáticos, arrays, frames del `StackMapTable`) y el **lattice de referencias** (covarianza de arrays + *join*/LUB). **GC generacional:** arena dividida en `Eden` | `S0` | `S1` | `Old`; los objetos nacen en Eden, un **minor** por **copia** evacúa los pocos sobrevivientes a un *survivor* (o los **promueve** a Old por edad) reescribiendo referencias con *forwarding*, un **write barrier** mantiene un **remembered set** de punteros *old→young* (las raíces que el minor escanea, sin recorrer todo Old), y el **major** hace mark-sweep-compact sobre Old. **Referencias débiles** (`java.lang.ref`): `WeakReference` + `ReferenceQueue` — el major trata el *referent* como débil y, al morir, lo limpia (`get()` → `null`) y **encola** la referencia; cimiento de `PhantomReference` / `Cleaner`.

Cobertura del set de opcodes — 199/199 alcanzables. El intérprete despacha todos los opcodes del JVMS que un class file moderno puede contener. Los últimos en cerrarse fueron `nop` (0x00), `goto_w` (0xc8) — el `goto` de offset de 4 bytes, para targets a más de ±32 KB, implementado ensanchando `jump_to` hacia un `jump_to_wide` común en vez de duplicar la aritmética — y el prefijo `wide` (0xc4), que reejecuta el opcode siguiente con un índice de local de **16 bits**: la única manera de direccionar los slots pasados el 255 en un método con más de 256 locales (`wide iinc` mide 6 bytes, el resto 4). Ese último salió barato por una propiedad que el diseño ya tenía: los handlers de `variable_operations` reciben `slot: usize` y **nunca se enteran del ancho**, así que ensanchar es puro *decoding* y el módulo no cambió de comportamiento. Verificado por *differential testing*: `WideLocals.java` declara 300 locales para forzar a `javac` a emitir el prefijo (`istore_w 299`, `iinc_w 299, 35`, `iload_w 299`), y el mismo `.class` da 42 tanto en el `java` de JDK 25 como en nuestra VM. **De los 4 restantes, 3 son una decisión y no deuda:** `jsr` / `ret` / `jsr_w` están prohibidos por **JVMS §4.9.1** en class files de versión 50.0+ (Java 6 en adelante) — desde que existe el `StackMapTable` y el verificador por *type-checking*, las subrutinas quedaron fuera del modelo, y `javac` compila *finally* duplicando el bloque. La postura del proyecto es **leer sí, ejecutar no**: el desensamblador soporta `jsr` / `ret` / `jsr_w` / `ret_w` porque un `javap` debe renderizar *cualquier*

class file legal (lo exige A0, medido sobre `java.base`), mientras el gate estructural del verificador los rechaza y el intérprete no los despacha. Curiosamente ejecutarlos sería lo más fácil que queda —unas 10 líneas—; el costo está en una variante `ReturnAddress` que toca todos los `match` sobre `Value` y, sobre todo, en verificar subrutinas polimórficas, la parte más difícil del type-checker del JVM. Razonamiento completo en la nota de diseño `subrutinas-jsr-ret.md`. Contarlos como faltantes mezclaría una decisión con una tarea: el número honesto es **199 de 199 alcanzables — completo**.

invokedynamic (0xba) — el subsistema. El opcode solo no hace nada: no nombra ningún método, sino un *bootstrap* que produce el destino la primera vez. Esa indirección es lo que le permitió a Java agregar lambdas, concatenación de strings y records sin inventar un opcode por cada una — el lenguaje pone la política en el bootstrap y la VM queda fija. Corren **5 de las 6 fábricas** que emite `javac`: `StringConcatFactory`, `SwitchBootstraps.typeSwitch` (patrones de tipo y de enum), `ObjectMethods` (los tres métodos de un `record`, desde una sola entrada de `BootstrapMethods` y distinguidos por el *nombre* del call site), `LambdaMetafactory.metafactory` y `ConstantBootstraps.invoke`. La sexta, `altMetafactory`, necesita serialización (`java.io`) y queda fuera de alcance. Dos decisiones de diseño: los *bootstrap methods* se modelan como **intrínsecos** —el mismo criterio de `intrinsicsec.md`—, con `java.lang.constant` escrito en **Java**; y `LambdaMetafactory` genera una clase real al vuelo (con el escritor de `.class`): implementa la interfaz funcional, con campos para las capturas y un método SAM que reenvía al handle — igual que el JDK, que la spinnea con ASM. El objeto lambda es entonces una instancia común, despachada por el itable normal, sin atajo. Ruta, correcciones y mediciones: `invokedynamic-ruta.md`.

La VM puede invocar Java (call_java). Empuja un frame propio y lo corre con un bucle de `run_one` anidado, devolviendo el resultado. Resultó ser el caso general de lo que la inicialización de clases venía haciendo: `<clinit>` era la variante sin argumentos ni resultado, así que el mecanismo ya existía entero y sólo estaba especializado. Importa porque **los intrínsecos dejan de ser terminales**: un nativo que sólo puede calcular y devolver está obligado a reimplementar lo que necesite de la biblioteca — así fue como `String.valueOf` terminó a medias en Rust. Con esto, `String.valueOf(Object)` llama al `toString()` del objeto, un `record` pregunta el `equals / hashCode` de sus componentes en vez de comparar referencias, y una constante dinámica ejecuta su bootstrap. Eso **cerró 0xba**: el **modelo de objetos de `java.lang.invoke`** (`MethodHandle / MethodType / Lookup`) vive en Java, con `MethodHandle.invoke / invokeWithArguments` nativos —polimorfía de firma, interceptados antes de la resolución—, el único intrínseco que corresponde. El premio se cobró: `ConstantBootstraps.invoke` es **ahora Java** (`handle.invokeWithArguments(args)`), lo que obligó a hacer el **cache de condy raíz de GC**. El ldc de `MethodHandle / MethodType` —que `javac` nunca emite— se probó con **class files hechos a mano por el escritor de `.class`** (su estreno). Con mirrors de primitivos (`int.class` → `Integer.TYPE`) y los kinds static/virtual/special/constructor, y `LambdaMetafactory` generando clases reales, `invokedynamic` queda completo.

multianewarray (0xc5). El último en cerrarse, y el que más enseña sobre el modelo de datos: **Java no tiene arrays rectangulares**. `new int[2][3]` no es un bloque plano sino dos `[I` colgando de un `[[I`, cada nivel un objeto real — por eso las filas se reemplazan por separado y `a[0].length` no tiene por qué igualar `a[1].length`. La instrucción lleva el descriptor del array *mismo* (`[[I`, no el elemento como `anewarray`) más un contador de dimensiones, porque sólo se materializan **esos** niveles: `new int[3][ ]` deja los internos en `null`. Los conteos se validan *todos* antes de alocar nada, para que un largo negativo en una dimensión posterior no deje un array a medio construir; los niveles superiores guardan referencias y sólo el más interno usa el ancho real del elemento (una fila `byte[ ]` ocupa un byte por elemento, no cuatro); y las referencias hijo se escriben por `store_reference`, nunca por un `write_u32` crudo, porque son exactamente los punteros *old→young* que el *remembered set* debe registrar. **Ejecutarlo era la mitad del trabajo**: hubo que modelarlo en el `transfer` del verificador, ya que un opcode sin modelar produce un `VerifyError` marcado `unsupported` — que por diseño hace que el llamador avise y siga, o sea que la clase habría corrido con el verificador claudicando en silencio.

Green threads (Fase 0). `java.lang.Thread + Thread.start()` nativo, un scheduler **cooperativo** round-robin que conmuta en el safepoint (entre opcodes), un call stack por-thread, y el GC tomando las raíces de **todos** los threads. Verificado: dos workers intercalándose con `main` (que los espera por spin-wait) dan `100`, y el step-visualizer muestra los cambios de contexto, el spawn y la terminación de cada thread. Es el modelo de los *virtual threads* de Java 21 (scheduling en espacio de usuario), con un solo carrier.

Monitores. Cada objeto tiene su monitor intrínseco (lock reentrante con dueño + *blocked-set* + *wait-set*). `synchronized` funciona en sus dos formas: **bloques** (opcodes `monitorenter / monitorexit`) y **métodos** (`ACC_SYNCHRONIZED`, sin opcode — el VM toma el monitor del receptor o del `Class` al empujar el frame y lo suelta al poparlo, por `return` o por *unwind* de excepción, vía un único punto de release imposible de saltar). La señalización `wait / notify / notifyAll` libera el monitor (guardando el conteo de reentrada), parquea al hilo en el *wait-set* y lo re-adquiere al despertar. Verificado con exclusión mutua real (dos hilos × 100 incrementos → 200, contra el control sin lock que pierde updates) y el *handshake* productor/consumidor (`wait` → `notify` → 42). **Pendiente del monitor:** `IllegalMonitorStateException`, `wait(timeout)`, interrupciones y la interacción monitor→compactación del GC (hoy se keyea por offset).

Arquitectura en capas (estilo service). El heap se encapsuló tras un `HeapService` que es su **único dueño**: toda escritura de referencia pasa por un portón (`store_reference`) que aplica el **write barrier** de forma no-bypassable — el seam donde irá la sincronización cuando los threads pasen a ser de SO. El runtime es la **JVM** (composition root) sobre `HeapService + MetaspaceService` + el sistema de threads.

Fase B — el compilador (front-end + semántica + chequeo + flujo + desugar). El `javac` propio (crate **desacoplado** en `src/javac/`, cuyo único contrato es el formato `.class`) tiene el front-end entero, las **dos pasadas** semánticas, el análisis de flujo y el desugar. **Front-end: lexer** (las 115 `TokenKind` de JDK 25, *maximal munch*, BOM), **parser** por *recursive descent* con **todas las sentencias** (incl. `switch` con patterns/guards, `try`-with-resources, `synchronized`), `enum / record / @interface` y **genéricos completos** — argumentos de tipo, *wildcards*, cotas y el diamante, partiendo `>> / >>>` con un *undo log* para no corromper el *backtracking*. El **lenguaje moderno** también entró: **lambdas** (con el desambiguador `(-cast-vs-paréntesis-vs-parámetros` por *lookahead* de la flecha), **referencias a método** (incl. `T[ ]::new`), **clases anónimas y locales**, el **type witness** (`Collections.<String>emptyList()`), que además se **aplica** —fija los parámetros de tipo en vez de inferirlos— y las **anotaciones** (§9.7), que antes se **descartaban** y ahora se cuelgan de la declaración. Cada forma se parsea enteramente y se guarda fiel en el AST; compilarlas es de fases posteriores, y hasta entonces la barrera del emisor las corta. **Pasada 1 (Enter/MemberEnter):** tabla de símbolos con paquetes y tipos **anidados**, detección de duplicados y **ciclos de herencia**, un **class finder real** que lee `.class` del classpath (lector propio) **incluido el atributo `Signature`** — así `java.util.List` carga con su `<E>` y `List.get` devuelve `E`, no `Object` — resolución con **errores posicionados**, y el **grafo tipado persistido** como salida. **Pasada 2 (Attribute):** resuelve nombres y tipa los cuerpos **decorando el AST in situ** (tipo + *binding* por nodo, slot por local con categoría-2, al estilo `JCTree.type / sym`); con **overload resolution en 3 fases** (estricta → boxing → varargs, con *most specific* y ambigüedad) y **genéricos reales** — subtipado con *containment* de *wildcards* (invariancia), sustitución del receptor, `lub / glb` e **inferencia** de los argumentos de tipo de una llamada (Cap. 18). El álgebra de tipos vive en un módulo `types` (el `Types` de `javac`) y la inferencia en `infer` (el `Infer`). **Pasada Flow (Flow):** sobre el AST decorado, la **asignación definitiva** del Cap. 16 — con los dos conjuntos duales *definitely assigned / definitely unassigned*, los flujos *when-true/when-false* de `&& / || / !`, las reglas de variables `final` (no reasignar, asignación única, multi-catch implícito), la **alcanzabilidad** (§14.21) y el `break / continue` **etiquetados** (§14.15/§14.16: la pila de contextos de `break` lleva la etiqueta a la que responde cada uno).

Pasada Desugar (desugar): baja el azúcar de **sentencias** (`for-each` → `for` indexado o `while` + `Iterator`, `try-with-resources` → `try/finally` + `close()`, `assert` → `if(!cond) throw`) y de **expresiones** (concatenación de `String` → cadena de `StringBuilder`, asignación compuesta `x op= y` → `x = (T)(x op y)` con el cast de reducción, y **varargs** → array en el *call site*) — verificado re-tipando el árbol bajado. En esta iteración se sumaron cuatro *rewrites* más: (1) **++ / -- en posición de descarte** (§15.14.2/§15.15.2) — como sentencia (`x++;`, `--x;`) o en el `update` de un `for` — se reescriben a `x += 1` reusando el *lowering* de la asignación compuesta (mismo cast de reducción), mientras que en posición de *valor* (`y = x++`) quedan para el codegen (`iinc / dup`); (2) la **switch-expresión** en posición de cola (§15.28) — `T v = switch..., return switch..., x = switch..., yield switch...` — se convierte en una *switch-sentencia* cuyos brazos **asignan** el valor (con temporal cuando hace falta); requiere `default` y etiquetas constantes, y los brazos con **bloque** o de **dos puntos** con `yield` (incluso adentro de un bucle o un `if`) se bajan reescribiendo cada `yield e` a `{ v = e; break $sw; }` con un **break etiquetado** sobre el `switch` generado — un `break` pelado solo saldría del bucle — mientras que el exhaustivo sin `default` y las *switch-expresiones* embebidas quedan como expresión sobre la pila; (3) **try-with-resources con excepción suprimida** (§14.20.3.1) — corrección de un bug: antes la excepción del `close()` *tapaba* la del cuerpo; ahora se preserva la primaria y la del `close()` queda **suprimida** (`Throwable.addSuppressed`), con la traducción completa del JLS (`$primary=null; try{...}catch($t){$primary=$t;throw;}finally{ if(r!=null){ if($primary!=null) try{r.close();}catch($x){$primary.addSuppressed($x);} else r.close(); }`); (4) **switch sobre String** (§14.11) → **dos switches sobre int**: el primero mapea `s.hashCode()` a un índice sintético vía `equals()` (cadena *if/else* para las colisiones de hash) y el segundo replica los brazos indexados (preservando *fall-through* y `default`), con el `String.hashCode()` calculado en tiempo de compilación (algoritmo del JDK: `31·h + c` sobre unidades UTF-16, aritmética `i32` que envuelve); y (5) **switch sobre enum** (§14.11) con el `$SwitchMap` **fiel a javac**: `switch(color) → switch(C$1.$SwitchMap$Color[color.ordinal()])`, donde `$SwitchMap$Color` es un `int[]` sintético alojado en una **clase anidada C\$1** y poblado en un `<clinit>` con un `try { map[Color.C.ordinal()] = k; } catch (NoSuchFieldError e) {}` por constante (robusto ante recompilación separada del enum). Esto requirió una **máquina de síntesis a nivel clase**: el miembro `Member::StaticInit` (con lo que los bloques `static { }` del usuario, antes **descartados**, ahora se conservan y analizan), la tabla de símbolos **mutable** en el *desugar* (para registrar la clase y el campo sintéticos, que la re-atribución tiene que ver) y el `enum extends Enum` implícito (§8.9, de donde sale `ordinal()`). y (6) **switch con type patterns** (§14.11.1) → una cadena de `instanceof` dentro de un bloque **etiquetado**: cada brazo es un `if` **suelto** (no `else if`) para que una **guarda que falla** caiga al `case` siguiente, el binding se materializa con un cast (`T v = (T) $t`, porque el `instanceof` del AST no lleva variable), se sale con `break` etiquetado —omitido cuando el brazo ya retorna, para no generar código inalcanzable— y un selector `null` sin `case null` lanza **NPE**. Esto exigió además que **Attribute bindee** la variable de patrón (visible en la guarda y en el cuerpo) y que **Flow** la vea siempre asignada dentro del brazo, como la del `catch`. y (7) el **guard \$assertionsDisabled** del `assert` (§14.10): la aserción pasa a `if (!$assertionsDisabled && !cond) throw ...`, con un campo sintético `static final boolean` y el `<clinit>` que lo calcula de `C.class.desiredAssertionStatus()` — es lo que hace que una aserción **no cueste nada** con las aserciones apagadas (el default), porque al ser `static final` el JIT lo pliega. Con eso la pasada **no tiene reescrituras pendientes**, y además **materializa los miembros implícitos de un record** (campos, constructor canónico y *accessors*): sus símbolos ya venían de `enter` —por eso `p.x()` resolvía— pero sin cuerpo en el AST el `.class` los referenciaba **sin declararlos**, y un `record` compilaba a una clase **vacía**. Y ya está la **síntesis equivalente del enum** (§8.9.3): las constantes como campos, el `$VALUES`, el constructor privado (`String, int`) —lo único que puede darle a `java.lang.Enum` su nombre y su ordinal— y `values()` / `valueOf()`. Destruirla obligó a cerrar antes la **invocación explícita de constructor** (`super(...)/this(...)`, §8.8.7.1), que **no existía** —el parser la producía y la atribución la rechazaba, así que no se podía heredar de ninguna clase con constructor parametrizado— y a sumar `java.lang.Enum` a `bootstrap/ + boot/`, sin la cual el `.class` emitido no se verificaba ni corría. En el camino salieron a la luz cuatro fugas **silenciosas** del back-end, ya cerradas: el emisor **nunca generaba <clinit>** (el `$VALUES`, el `$SwitchMap` y el `guard` del `assert` quedaban declarados y en cero), los **inicializadores de campo** (`int v = 7;`) **no llegaban al .class**, `a.length` se tipaba `Unresolved` —y `i < a.length` salía comparando con `if_acmpne`— y un `new` de un tipo sin resolver no emitía nada. Falta el `enum` con constantes **parametrizadas** (`ROJO("rojo")`) o constructor propio; el boxing/erasure queda para `TransTypes`. Desde entonces el *desugar* ganó tres reescrituras grandes: **LambdaToMethod** (lambdas → método sintético `lambda$...` + `invokedynamic` con `LambdaMetafactory`, capturando locales y `this`), las **referencias a método** y el `record` por `ObjectMethods` (ambos por un nodo `Indy` genérico), y la **captura de this\$0** de las clases internas de instancia. Un binario **javac-step** recorre las cuatro fases **paso a paso** (tokens → AST → tabla de símbolos → chequeo), el modo `--desugar` muestra el árbol bajado, y la semántica está citada en la JLS 25 en `docs/pasada2-attribute.md` y `docs/semantica-jdk25.md`.

Fase B — Codegen (B3): objetos y flujo de control. El `compile()` corre ya las **pasadas** en el orden de `javac` (`Enter` → `Attribute` → `Check` → `Flow` → `Desugar` → `re-Attribute` → `Codegen`): **Flow antes que Desugar**, para que sus errores apunten al código fuente y no al bajado, y una **re-atribución** después, porque las reescrituras dejan nodos frescos sin `ty / binding / slot` y el emisor los necesita para elegir el opcode. Un programa con errores semánticos **no emite .class**. El **escriptor** (`class_writer`) es propio y desacoplado: constant pool con deduplicación —incluidas las entradas `String`, `Long`, `Double` y `Float`, con el **hueco** que la JVMs exige tras cada `Long / Double` (ocupan dos entradas, y si no se reserva se corren todos los índices)— más la sección de **campos** y los atributos `Code / SourceFile`. El **emisor** cubre hoy: literales de todos los tipos (`ldc_w` para los anchos), la **promoción numérica binaria** (§5.6.2) con los doce opcodes `x2y` —`longA + 1` inserta el `i2l`, y los desplazamientos quedan exceptuados (§15.19)—, **objetos** (`new + dup + invokespecial`, `getField / putField`, `getStatic / putStatic`, `invokeVirtual`, `checkCast`), **asignación** a local y a campo, y **flujo de control** completo (`if / else`, `while`, `do-while`, `for`, `break`, `continue`) con etiquetas y parcheo de saltos, comparaciones por categoría y `&& / ||` compilados como **saltos** —cortocircuito real, no un booleano calculado—. Verificado **ejecutando en la JVM propia**: `fib(10)=55` recursivo con rama, `fact(5)=120`, `break / continue`, aritmética `long / double`, y un objeto completo (`new M(10); m.add(5); m.get()` → 15). Tres bugs de corrección salieron de ahí: los constructores no emitían el `super()` implícito (§8.8.7, dejando el `this` sin inicializar), el `.class` no declaraba sus **campos** (referenciaba un `fieldref` inexistente) y faltaba el `Unary` de negación. También emite **excepciones**: `try / catch` con su tabla, `throw`, y el `finally` **duplicado** en la salida normal y en un handler `catch-all` que re-lanza (§14.20.2) —la v69 ya no acepta `jsr / ret`, así que duplicar es la única vía—. Eso, de paso, **reubicó** el *inlining* del `finally`, que teníamos anotado como tarea pendiente del *desugar*: no es una reescritura de AST sino trabajo del emisor. Y está la **StackMapTable** (§4.7.4), la pieza que separa «corre en mi JVM» de «corre en cualquier JVM»: el frame de cada destino de salto se calcula como el **merge** de los estados que llegan ahí (un slot que no coincide en todos los caminos pasa a `Top`), siempre en forma `full_frame`, y los *handlers* llevan `stack = [E]` con el frame **forzado**, porque no los alcanza ningún salto. La validación es el **cross-check del verificador propio** contra su propia inferencia por punto fijo — que ya cazó un bug real: decidir «¿hace falta frame acá?» al fijar la etiqueta se saltaba el tope de los bucles, porque el `goto` hacia atrás se emite *después*. Y tiene **barrera de seguridad**: `generate()` devuelve `Result`, así que una construcción no soportada **falla con posición** en vez de emitir un `.class` a medias. Eso destapó al toque dos agujeros que el propio *desugar* venía alimentando en silencio — `instanceof` (todo *pattern switch*) y la creación de **arrays** (todo *varargs*)—, ya cerrados: el emisor cubre hoy `newarray / anewarray`, `arraylength`, las familias `Xaload / Xastore`, los inicializadores y el `instanceof`, de modo que un *pattern switch* y una llamada *varargs* **compilan y corren**. Y se cerró el resto del flujo de control: el **ternario** con sus dos ramas **promovidas al tipo del todo** (§15.25), el `switch` como `tableswitch` o `lookupswitch` según la **densidad** de sus etiquetas —con el relleno de alineación a 4 bytes y los desplazamientos de 4, preservando el *fall-through* de la forma de dos puntos y cortándolo en la de flecha—, los `break / continue` **con y sin etiqueta** sobre una pila de sentencias «de las que se puede salir» —un `switch` interpuesto captura un `break` pero no un `continue` (§14.16)— más el **bloque etiquetado**, y **synchronized** con `monitorexit` en

las dos salidas, la normal y un handler *catch-all* que re-lanza. De yapa, el **switch sobre String** pasó a compilar de punta a punta: el desugar ya lo bajaba a dos switches sobre **int**, pero ninguno de los dos se podía emitir. Cerrar eso destapó el último agujero **silencioso** del emisor, y en la pieza más delicada: la pila de operandos se llevaba como un **simple contador de altura**, pero un frame del **StackMapTable** tiene que declarar **los tipos** de lo que hay en ella. Mientras todos los destinos de salto caían en bordes de sentencia —con la pila vacía— no se notaba; el ternario los mete en **medio de una expresión**: en `new M(a > 0 ? 1 : 2)` el `new` y su `dup` siguen vivos en la pila, y el frame los declaraba como pila vacía. El `.class` salía igual —la barrera no ve esto, porque el emisor cree estar emitiendo bien— y lo rechazaba recién el verificador. La pila pasó a llevarse **tipada** (48 puntos de ajuste), lo que de paso hizo representable el tipo **uninitialized** (tag 8), que identifica un objeto recién creado por el **offset de su new** y, corrido su `<init>`, pasa a definitivo en todas sus apariciones —pila y locales— (§4.10.2.4). **Después** se sumó el **invokedynamic** (opcode `0xba`): el pool ganó `MethodHandle` / `MethodType` / `InvokeDynamic` y el atributo `BootstrapMethods`, y un nodo **Indy genérico** (bootstrap + argumentos estáticos tipados) sirve tanto al `LambdaMetafactory` de las lambdas/*method refs* como al `ObjectMethods` de un **record** — con el emisor empujando las capturas y el verificador propio validando el call site. Y se cerró el frente de **clases internas, locales y anónimas**: la **captura del entorno** — `this$0` de la instancia envolvente y `val$x` de los locales *effectively-final*, con un pase `hoist_anonymous` que baja cada `new T(){...}` a una local sintética `Outer$N` —, las tres formas **corren y verifican**. Por último, la **emisión de interfaces propias**: el `.class` de una *interface* sale con `ACC_INTERFACE` y sus métodos **abstractos sin Code**, y se sumó `invokeinterface` (con `InterfaceMethodref`) para las llamadas sobre un receptor de interfaz —lo que desbloqueó las **anónimas sobre una interfaz** (`new Runnable(){...}`), el caso más común) y las lambdas contra interfaces funcionales propias—.

✓ **Siguiente (compilador)**: el **frente grande de invokedynamic quedó cerrado** —lambdas (con un `LambdaToMethod` propio), **referencias a método** (las cuatro formas del §15.13.1) y el `equals` / `hashCode` / `toString` de un **record** (por `ObjectMethods`) se tipan, emiten y **verifican**; la ejecución del call site vive en la JVM (KajiJDK)—. Se cerraron además las **clases internas, locales y anónimas** —captura del entorno completa (`this$0` + `val$`, con un pase `hoist_anonymous` que baja la anónima a una local sintética `Outer$N`), las tres **corren y verifican**, incluidas las formas **cualescadas** (`Outer.this` → cadena de `this$0`, con desambiguación de campo sombreado y multinivel; y `outer.new Inner()`, que empuja el calificador como `this$0`) y los **argumentos de super** (`new ArrayList(10){...}`), la **colisión de nombres** (dos locales homónimas se renombra a únicas `1L` / `2L` con alcance léxico por bloque), la **local-en-local** (anidada como tipo de su local envolvente) y el **multinivel implícito** (acceso a un miembro del abuelo vía `this$0` encadenado) —las clases internas quedan **sin colas**—; el parser (B1) quedó **sin bordes** (anotaciones de tipo/uso `List<@NonNull String>`, `<@Foo T>`, `new <T>Foo()`), y las de **nivel de array** `String @A []`); y la **emisión de interfaces propias** cerró el caso más común de anónima (`new Runnable(){...}`) sumando `invokeinterface`. Y se cerró el **boxing dirigido por tipo** (*TransTypes*): una pasada propia inserta `Integer.valueOf / x.intValue` en cada contexto de conversión (asignación, `return`, argumentos, operandos, cuerpo de lambda), cerrando el caso insignia de las *poly expressions* — `Function<Integer,Integer> f = x -> x + 1` de punta a punta, validado por el verificador (ejecutar `valueOf` es de la JVM)—. Y se cerró la **inferencia en posición de argumento** (dos fases §15.12.2.6): un argumento **poly** —lambda, *method ref*, diamante— se re-atribuye con el tipo de su parámetro como *target* una vez resuelta la sobrecarga, lo que por fin baja una **lambda-argumento** (`call(x -> x + 1)`) a `invokedynamic`. Y se cerró el **enum con constantes parametrizadas** (`ROJO("rojo")`): al ctor propio se le antepone (`String, int`) + `super(...)` y cada constante se construye con sus argumentos. Y se *liquidaron* los sueltos** que quedaban: `@Override` / `throws` en el chequeo de override (§8.4.8.3/§9.6.4.4), `RuntimeVisibleAnnotations` (§4.7.16), el **plegado de constantes** en un `case` (§15.28), las **formas comprimidas del StackMapTable** (§4.7.4), los **inicializadores de campo atribuidos** (§8.3.2), la **desambiguación de sobrecargas por el arg lambda** (§15.12.2.5) y el **boxing wrapper→primitivo más ancho** (`long l = anInteger`). Y se cerró el primer **frente grande de Java SE 25: sealed / non-sealed + exhaustividad de switch** (§8.1.1.2/§9.1.1.4/§8.1.6/§14.11.1) — el parser toma las keywords restringidas y el `permits`, el grafo gana los **subtipos autorizados** (con el `permits` implícito de la unidad), **Check** valida las reglas de sellado y exige **exhaustividad** a la `switch`-expresión y a la sentencia con *patterns* (jerarquía `sealed` recursiva, todas las constantes de un `enum`, `true` / `false` de un `boolean`, una guarda no cubre), y el `.class` lleva el atributo `PermittedSubClasses` (§4.7.31) — de yapa se corrigió que `case RED -> 1` se leía como lambda. Con eso la lista **tracked** del compilador queda vacía. Y se cerró el segundo frente de **Java SE 25**: los **text blocks** (§3.10.6) — el parser decodifica el `"""..."""` con el algoritmo del JLS (normalización de terminadores, descarte de la línea de apertura, **indentación incidental** mínima + blancos finales, y luego los escapes con `s` y la **continuación de línea** `<LF>`), dando un `String` común que se emite como cualquier literal. Y se cerró el tercer frente: la **capture conversion** (§5.1.10) con **variables de captura frescas** (un `RType::Capture{id,upper,lower}` con cotas inline e `id` fresco para la *freshness*): se aplica en acceso a miembros y en argumentos, el subtipado gana `CAP <: upper` y `s <: CAP` por la cota inferior, y el caso insignia `swap(List<T>) → swapHelper(List<T>)` ahora **infiere T**. Y se cerró el cuarto frente: los **bridge methods** (§8.4.8.3/§15.12.4.5) — el emisor sintetiza los puentes (`ACC_BRIDGE` + `ACC_SYNTHETIC`) que hacen que la sobrescritura funcione en bytecode cuando la *erasure* difiere, por un **parámetro genérico borrado** (`Comparable<Foo> → compareTo(Object)`, `Node<Integer>.set(Integer) → set(Object)`) o por un **retorno covariante** (`A.get():Object` / `B.get():String`), reenviando al método real con `checkcast` + `invokevirtual`; verifica y **despacha** de punta a punta. Y se cerró el último frente grande: los **módulos** (§7.7/§4.7.25) — el parser toma el `module-info.java` con las cinco directivas (keywords restringidas), el emisor produce un `module-info.class` fiel (`ACC_MODULE`, atributo `Module` con `CONSTANT_Module` / `Package` y el `requires java.base` mandated), y **Check** valida la buena formación (directivas duplicadas, auto- `requires`); la resolución del grafo es de runtime (JPMS, track aparte). **Con esto, los cinco frentes grandes de Java SE 25 quedan cerrados del lado del compilador** —sealed + exhaustividad, text blocks, captura de wildcards, *bridge methods* y módulos (B5)—. Lo que le queda al compilador ***no*** son features de lenguaje sino ***fidelidad*** (B6). Ya se cerraron los cuatro primeros atributos: ✓ **Signature** (§4.7.9) —los genéricos ya ***no* se pierden** en el binario—, ✓ **InnerClasses** (§4.7.6), ✓ **EnclosingMethod** (§4.7.7) —anidamiento y método envolvente de local/anónimas— ✓ **Record** (§4.7.30) —un **record** ahora extiende `java.lang.Record`, es **final** y lleva el atributo con sus componentes, así `isRecord()` / `getRecordComponents()` andan (y se cerraron dos bugs latentes: el parseo de un record genérico y la *erasure* del descriptor de una variable de tipo)—y ✓ **NestHost / NestMembers** (§4.7.28/§4.7.29) —el *nest* plano (host + nestmates transitivos) que da acceso privado entre anidadas—, ✓ **NestHost / NestMembers** (el *nest*) y ✓ **MethodParameters** (§4.7.24, los nombres de parámetros para la reflexión). De la ***inferencia*** ya se cerraron el lado escritura de la captura por llamada y el **lub con intersección** (`Number & Comparable`); queda solo el *solver* transitivo de §18 (bounds cruzados). Del lado de los **atributos ya no queda fidelidad reflexiva pendiente**: se cerraron `Exceptions`, `LineNumberTable`, `LocalVariableTable`, `ConstantValue` y **todas** las type annotations (`RuntimeVisibleTypeAnnotations`, §4.7.20 — parámetros de tipo, cotas, extends/implements, throws, field/return/param y los targets de `Code` : `cast` / `instanceof` / `new` / local), todo verificado byte-fiel contra el `javap` de Temurin.

Fase H — KajiLibrary (la biblioteca, dogfooded). La biblioteca propia vive en `KajiLibrary/`, se compila con el `javac` propio y corre en KajiJDK. **H1** cerró los *tiers* 0–5 (el esqueleto: `Object` / `String` / `StringBuilder`, excepciones, interfaces funcionales, colecciones); **H2** (utilidades y conveniencia de `java.util`) está **en curso**; **H3** (`java.util.stream`), **H4** (`java.time`) y **H5** (`java.util.regex`) tienen su **núcleo cerrado**, y **H6** (`java.util.Formatter` / `String.format`) está **diseñado**. Lo acoplado al runtime — `java.nio`, `java.util.concurrent` y la reflexión completa— queda en el backlog (**H7**): esa parte avanza por el track de la JVM, no por el de la biblioteca.

✓ **Siguiente**: con **H4 (JMM relajado)**, **H5 (CAS — compareAndSwap + los Atomic*)** y el **núcleo + primeras utilidades de H6 (AQS exclusivo+compartido** → `ReentrantLock` / `Condition`, `ReentrantReadWriteLock`, `Semaphore`, `CountDownLatch`, `CyclicBarrier`, `ArrayBlockingQueue` / `LinkedBlockingQueue` / `PriorityBlockingQueue` / `DelayQueue`, el framework `Executor` (`ThreadPoolExecutor` + `ScheduledThreadPoolExecutor`), `ConcurrentHashMap` y `CompletableFuture`) ya hechos y validados por el oráculo, sigue el **volumen restante de java.util.concurrent** (deques, *resize*/lecturas lock-free del `ConcurrentHashMap`) y los **locks finos por estructura** (ver la sección «Concurrencia»). Los green

threads quedan como substrato elegible y **oráculo determinista** de corrección (`green=0s-gil=0s`) — clave para cazar el **bug residual** del modo paralelo (referencia *stale* bajo GC intenso; guard parcial landeado, próximo paso ThreadSanitizer). El paralelismo de cómputo del LLM vive en **kernels off-heap** (rayon/CUDA).

Fase I — el depurador (JPDA: JVMTI · JDWP · jdb). La VM propia se depura, y no solo desde adentro. El back-end JVMTI (módulo `jvmti.rs`) es un modelo *push*: un `JvmtiAgent` registra *callbacks* (`breakpoint / single_step / method_entry / method_exit / exception`) que la VM dispara en medio de `step()`, pasándole un `JvmtiEnv` **acotado** para re-inspeccionar/controlar la ejecución — con el *fast-path* por `Capabilities` («pagás por lo que activás») y, la pieza fina, **ganarle al borrow checker sin unsafe**: la VM **posee** al agente pero al disparar un evento **destruttura `self` en campos disjuntos** (`agent` ⊥ los del `env`), y acotar el `env` es lo que lo hace compilar. Sobre eso corren **dos front-ends**: el **jdb pragmático** (I1, *in-process*) — un binario cuyo prompt (`step / cont / break / where / locals / print`) corre *dentro* del callback — y el **camino fiel por protocolo** (I3): el JDWP completo —codec puro (handshake, framing, serialización tipada big-endian), *command handlers* (`VirtualMachine / EventRequest`) sobre una `JdwpSession`, transporte genérico sobre `Read + Write`, y el **bridge JVMTI↔JDWP** que **empuja** los eventos como *Composite packets* y **aplica** los `EventRequest` del cliente a la VM real (un breakpoint pedido por cable **frena de verdad**)—. El binario `jvm-jdwp` expone la VM como servidor `dt_socket` (equivale a `-agentlib:jdwp=...,server=y,suspend=y`): **probado end-to-end** con un cliente JDWP **externo** (otro proceso) que *attachea*, *handshakea*, pide *single-step* y recibe los eventos frenando/reanudando por el socket hasta el retorno. Ambos caminos son **dos front-ends sobre el mismo JVMTI**. Encima del cable está la **JDI** (I4, `jdi.rs`): la API cliente de alto nivel que habla por *mirrors* tipados (`Vm / Location / Event`) en vez de bytes —contraparte del *bridge* del lado del debugger, depende solo del codec—, con el binario `jdi-attach` que maneja la ejecución por socket **dos procesos, todo propio** (reemplazó al cliente de Python). Y la **fidelidad** (I5) trajo las dos capacidades que hacen a un depurador **de verdad**: **(a) inspección de pila y variables** con la VM parada (`ThreadReference.Frames`, `StackFrame.GetValues`) leyendo el mismo `JvmtiEnv`; y **(b) breakpoints por línea de fuente** (`ClassesBySignature`, `ReferenceType.Methods`, `Method.LineTable`) — el cliente traduce `Add.java:3` a (`methodID`, `indice`). El muro acá fue que listar métodos con su `MethodID` real exige resolverlos (`&mut metaspace`) pero en un callback el metaspace es inmutable; la salida es un **snapshot** capturado al *attachear*, y como **resolve_method** **cachea**, el `methodID` del snapshot es *el mismo* que la VM usa al correr, así el breakpoint frena. **Probado en vivo** con cliente propio: `jdi-attach --break 3` contra `jvm-jdwp Add.add 3 4` resuelve `LAdd;.add:3`, la VM **frena en el breakpoint**, se leen los locales (`slot0=3`, `slot1=4`) y reanuda a `Some(Int(7))`. Y —la vara de la cumbre— **validado contra el jdb real de Temurin** (el mismo `com.sun.jdi` que usa IntelliJ Platform): *attachea* por `dt_socket` **sin saber que la VM es propia**, pone un breakpoint por línea y lo frena — `VM Started: Add.add(), line=3 bci=0 · Breakpoint hit: Add.add(), line=3 bci=0 · where → [1] Add.add (Add.java:3)`. Destrabar ese attach pidió cerrar la superficie que un JDI real emite —las variantes `WithGeneric`, la negociación (`Capabilities`, `ClassPaths`, grupos de hilos), el evento automático `VMStart`, y los **ids no-cero** (JDWP reserva el 0 para *null*: `classID`, `threadID` y `methodID = MethodId + 1`)—, todo hallado iterando contra `jdb` como oráculo. Y se cerró el resto de la superficie interactiva, cada gap hallado contra `jdb`: **locals** por nombre y valor (`Method.VariableTable` desde la `LocalVariableTable` del `.class` compilado con `-g`); los **field watchpoints** (I2) por protocolo —`watch Watched.value → Field (Watched.value) is 0, will be 7`—, que necesitaron el gancho central `fire_field_watchpoint` (decodifica el opcode de campo read-only, sin tocar los handlers), el modificador `FieldOnly`, y las caps `canWatchField*`; y la **inspección de objetos** que ese display exige —el `JvmtiEnv` ganó acceso *read-only* al **heap** y `ObjectReference.ReferenceType/GetValues` leen la clase real de un objeto (por el *mirror* de su header) y el valor de sus campos (por su offset precomputado)—. Con eso la sesión interactiva de `jdb` queda **100% limpia** —`threads`, `classes`, `stop at / cont`, `where`, `locals`, `watch`, todos sin errores—. Lo único fuera de alcance es correr el IntelliJ **gráfico** (no headless acá); `jdb` es el mismo `com.sun.jdi` que hay debajo. **El hito I queda completo**: JVMTI (I0–I2, con field watchpoints), JDWP (I3), JDI (I4) y fidelidad validada contra el debugger real (I5).

Fase B — el compilador (front-end + semántica + chequeo + flujo + desugar). El `javac` propio (crate **desacoplado** en `src/javac/`, cuyo único contrato es el formato `.class`) tiene el front-end entero, las **dos pasadas** semánticas, el análisis de flujo y el desugar. **Front-end: lexer** (las 115 `TokenKind` de JDK 25, *maximal munch*, BOM), **parser** por *recursive descent* con **todas las sentencias** (incl. `switch` con patterns/guards, `try-with-resources`, `synchronized`), `enum / record / @interface` y **genéricos completos** —argumentos de tipo, *wildcards*, cotas y el diamante, partiendo `>> / >>>` con un *undo log* para no corromper el *backtracking*. El **lenguaje moderno** también entró: **lambdas** (con el desambiguador `(-cast-vs-paréntesis-vs-parámetros` por *lookahead* de la flecha), **referencias a método** (incl. `T[]::new`), **clases anónimas y locales**, el **type witness** (`Collections.<String>emptyList()`), que además se **aplica** —fija los parámetros de tipo en vez de inferirlos— y las **anotaciones** (§9.7), que antes se **descartaban** y ahora se cuelgan de la declaración. Cada forma se parsea entera y se guarda fiel en el AST; compilarlas es de fases posteriores, y hasta entonces la barrera del emisor las corta. **Pasada 1 (Enter/MemberEnter)**: tabla de símbolos con paquetes y tipos **anidados**, detección de duplicados y **ciclos de herencia**, un **class finder real** que lee `.class` del classpath (lector propio) **incluido el atributo `Signature`** — así `java.util.List` carga con su `<E>` y `List.get` devuelve `E`, no `Object` — resolución con **errores posicionados**, y el **grafo tipado persistido** como salida. **Pasada 2 (Attribute)**: resuelve nombres y tipa los cuerpos **decorando el AST in situ** (tipo + *binding* por nodo, slot por local con categoría-2, al estilo `JCTree.type / sym`); con **overload resolution en 3 fases** (estricta → boxing → varargs, con *most specific* y ambigüedad) y **genéricos reales** — subtipado con *containment* de *wildcards* (invariancia), sustitución del receptor, `lub / glb` e **inferencia** de los argumentos de tipo de una llamada (Cap. 18). El álgebra de tipos vive en un módulo `types` (el `Types` de `javac`) y la inferencia en `infer` (el `Infer`). **Pasada Flow (flow)**: sobre el AST decorado, la **asignación definitiva** del Cap. 16 — con los dos conjuntos duales *definitely assigned / definitely unassigned*, los flujos *when-true/when-false* de `&& / || / !`, las reglas de variables **final** (no reasignar, asignación única, multi-catch implícito), la **alcanzabilidad** (§14.21) y el `break / continue` **etiquetados** (§14.15/§14.16: la pila de contextos de `break` lleva la etiqueta a la que responde cada uno). **Pasada Desugar (desugar)**: baja el azúcar de **sentencias** (`for-each` → `for` indexado o `while + Iterator`, `try-with-resources` → `try/finally + close()`, `assert` → `if(!cond) throw`) y de **expresiones** (concatenación de `String` → cadena de `StringBuilder`, asignación compuesta `x op= y` → `x = (T)(x op y)`) con el cast de reducción, y **varargs** → array en el *call site* — verificado re-tipando el árbol bajado. En esta iteración se sumaron cuatro *rewrites* más: (1) `++ / --` **en posición de descarte** (§15.14.2/§15.15.2) — como sentencia (`x++`; `--x`;) o en el `update` de un `for` — se reescriben a `x += 1` reusando el *lowering* de la asignación compuesta (mismo cast de reducción), mientras que en posición de *valor* (`y = x++`) quedan para el codegen (`iinc / dup`); (2) la **switch-expresión** en posición de cola (§15.28) — `T v = switch..., return switch..., x = switch..., yield switch...` — se convierte en una **switch-sentencia** cuyos brazos **asignan** el valor (con temporal cuando hace falta); requiere `default` y etiquetas constantes, y los brazos con **bloque** o de **dos puntos** con `yield` (incluso adentro de un bucle o un `if`) se bajan reescribiendo cada `yield e` a `{ v = e; break $sw; }` con un **break etiquetado** sobre el `switch` generado —un `break` pelado solo saldría del bucle— mientras que el exhaustivo sin `default` y las switch-expresiones embebidas quedan como expresión sobre la pila; (3) **try-with-resources con excepción suprimida** (§14.20.3.1) — corrección de un bug: antes la excepción del `close()` *tapaba* la del cuerpo; ahora se preserva la primaria y la del `close()` queda **suprimida** (`Throwable.addSuppressed`), con la traducción completa del JLS (`$primary=null; try{...}catch($t){$primary=$t;throw;}finally{ if(r!=null){ if($primary!=null) try{r.close();}catch($x){$primary.addSuppressed($x);} else r.close(); } }`); (4) **switch sobre String** (§14.11) → **dos switches** sobre `int`: el primero mapea `s.hashCode()` a un índice sintético vía `equals()` (cadena *if/else* para las colisiones de hash) y el segundo replica los brazos indexados (preservando *fall-through* y `default`), con el `String.hashCode()` calculado en tiempo de compilación (algoritmo del JDK: `31·h + c` sobre unidades UTF-16, aritmética `i32` que envuelve); y (5) **switch sobre enum** (§14.11) con el **\$SwitchMap fiel a javac**: `switch(color) → switch(C$1.$SwitchMap$Color[color.ordinal()])`, donde `$SwitchMap$Color` es un `int[]` sintético alojado en una **clase anidada**

`C$1` y poblado en un `<clinit>` con un `try { map[Color.C.ordinal()] = k; } catch (NoSuchFieldError e) {}` por constante (robusto ante recompilación separada del `enum`). Esto requirió una **máquina de síntesis a nivel clase**: el miembro `Member::StaticInit` (con lo que los bloques `static { }` del usuario, antes **descartados**, ahora se conservan y analizan), la tabla de símbolos **mutable** en el desugar (para registrar la clase y el campo sintéticos, que la re-atribución tiene que ver) y el `enum extends Enum` implícito (§8.9, de donde sale `ordinal()`). y (6) **switch con type patterns** (§14.11.1) → una cadena de `instanceof` dentro de un bloque **etiquetado**: cada brazo es un `if` **suelto** (no `else if`) para que una **guarda que falla** caiga al `case` siguiente, el binding se materializa con un cast (`T v = (T) $t`, porque el `instanceof` del AST no lleva variable), se sale con `break` etiquetado —omitido cuando el brazo ya retorna, para no generar código inalcanzable— y un selector `null` sin `case null` lanza **NPE**. Esto exigió además que **Attribute** *bindee* la variable de patrón (visible en la guarda y en el cuerpo) y que **Flow** la vea siempre asignada dentro del brazo, como la del `catch`. y (7) el **guard \$assertionsDisabled** del `assert` (§14.10): la aserción pasa a `if (!$assertionsDisabled && !cond) throw ...`, con un campo sintético `static final boolean` y el `<clinit>` que lo calcula de `C.class.desiredAssertionStatus()` — es lo que hace que una aserción **no cueste nada** con las aserciones apagadas (el default), porque al ser `static final` el JIT lo pliega. Con eso la pasada **no tiene reescrituras pendientes**, y además **materializa los miembros implícitos de un record** (campos, constructor canónico y *accessors*): sus símbolos ya venían de `enter` —por eso `p.x()` resolvía— pero sin cuerpo en el AST el `.class` los referenciaba **sin declararlos**, y un `record` compilaba a una clase **vacía**. Y ya está la **síntesis equivalente del enum** (§8.9.3): las constantes como campos, el `$VALUES`, el constructor privado `(String, int)` —lo único que puede darle a `java.lang.Enum` su nombre y su ordinal— y `values()` / `valueOf()`. Destruirla obligó a cerrar antes la **invocación explícita de constructor** (`super(...)/this(...)`, §8.7.1), que **no existía** —el parser la producía y la atribución la rechazaba, así que no se podía heredar de ninguna clase con constructor parametrizado— y a sumar `java.lang.Enum` a `bootstrap/ + boot/`, sin la cual el `.class` emitido no se verificaba ni corría. En el camino salieron a la luz cuatro fugas **silenciosas** del back-end, ya cerradas: el emisor **nunca generaba <clinit>** (el `$VALUES`, el `$SwitchMap` y el guard del `assert` quedaban declarados y en cero), los **inicializadores de campo** (`int v = 7;`) **no llegaban al .class**, `a.length` se tipaba `Unresolved` —y `i < a.length` salía comparando con `if_acmpne`— y un `new` de un tipo sin resolver no emitía nada. Falta el `enum` con constantes **parametrizadas** (`ROJO("rojo")`) o constructor propio; el boxing/erasure queda para `TransTypes`. Desde entonces el desugar ganó tres reescrituras grandes: **LambdaToMethod** (lambdas → método sintético `lambda$...` + `invokedynamic` con `LambdaMetafactory`, capturando locales y `this`), las **referencias a método** y el `record` por `ObjectMethods` (ambos por un nodo `Indy` genérico), y la **captura de this\$0** de las clases internas de instancia. Un binario `javac-step` recorre las cuatro fases **paso a paso** (tokens → AST → tabla de símbolos → chequeo), el modo `--desugar` muestra el árbol bajado, y la semántica está citada a la JLS 25 en `docs/pasada2-attribute.md` y `docs/semantica-jdk25.md`.

Fase B — Codegen (B3): objetos y flujo de control. El `compile()` corre ya las **pasadas** en el orden de `javac` (`Enter` → `Attribute` → `Check` → `Flow` → `Desugar` → `re-Attribute` → `Codegen`): **Flow antes que Desugar**, para que sus errores apunten al código fuente y no al bajado, y una **re-atribución** después, porque las reescrituras dejan nodos frescos sin `ty / binding / slot` y el emisor los necesita para elegir el opcode. Un programa con errores semánticos **no emite .class**. El **escritor** (`class_writer`) es propio y desacoplado: constant pool con deduplicación —incluidas las entradas `String`, `Long`, `Double` y `Float`, con el **hueco** que la JVMs exige tras cada `Long / Double` (ocupan dos entradas, y si no se reserva se corren todos los índices)— más la sección de **campos** y los atributos `Code / SourceFile`. El **emisor** cubre hoy: literales de todos los tipos (`ldc_w` para los anchos), la **promoción numérica binaria** (§5.6.2) con los doce opcodes `x2y` —`longA + 1` inserta el `i2l`, y los desplazamientos quedan exceptuados (§15.19)—, **objetos** (`new + dup + invokespecial`, `getfield / putfield`, `getstatic / putstatic`, `invokevirtual`, `checkcast`), **asignación** a local y a campo, y **flujo de control** completo (`if / else`, `while`, `do-while`, `for`, `break`, `continue`) con etiquetas y parcheo de saltos, comparaciones por categoría y `&& / ||` compilados como **saltos** —cortocircuito real, no un booleano calculado—. Verificado **ejecutando en la JVM propia**: `fib(10)=55` recursivo con rama, `fact(5)=120`, `break / continue`, aritmética `long / double`, y un objeto completo (`new M(10); m.add(5); m.get()` → 15). Tres bugs de corrección salieron de ahí: los constructores no emitían el `super()` implícito (§8.8.7, dejando el `this` sin inicializar), el `.class` no declaraba sus **campos** (referenciaba un `fieldref` inexistente) y faltaba el `Unary` de negación. También emite **excepciones**: `try / catch` con su tabla, `throw`, y el **finally duplicado** en la salida normal y en un handler `catch-all` que re-lanza (§14.20.2) —la v69 ya no acepta `jsr / ret`, así que duplicar es la única vía—. Eso, de paso, **reubicó** el `inlining` del `finally`, que teníamos anotado como tarea pendiente del *desugar*: no es una reescritura de AST sino trabajo del emisor. Y está la `StackMapTable` (§4.7.4), la pieza que separa «corre en mi JVM» de «corre en cualquier JVM»: el frame de cada destino de salto se calcula como el **merge** de los estados que llegan ahí (un slot que no coincide en todos los caminos pasa a `Top`), siempre en forma `full_frame`, y los `handlers` llevan `stack = [E]` con el frame **forzado**, porque no los alcanza ningún salto. La validación es el **cross-check del verificador propio** contra su propia inferencia por punto fijo — que ya cazó un bug real: decidir «¿hace falta frame acá?» al fijar la etiqueta se saltaba el tope de los bucles, porque el `goto` hacia atrás se emite *después*. Y tiene **barrera de seguridad**: `generate()` devuelve `Result`, así que una construcción no soportada **falla con posición** en vez de emitir un `.class` a medias. Eso destapó al toque dos agujeros que el propio desugar venía alimentando en silencio — `instanceof` (todo *pattern switch*) y la creación de **arrays** (todo *varargs*)—, ya cerrados: el emisor cubre hoy `newarray / anewarray`, `arraylength`, las familias `Xaload / Xastore`, los inicializadores y el `instanceof`, de modo que un *pattern switch* y una llamada *varargs* **compilan y corren**. Y se cerró el resto del flujo de control: el **ternario** con sus dos ramas **promovidas al tipo del todo** (§15.25), el `switch` como `tableswitch` o `lookupswitch` según la **densidad** de sus etiquetas —con el relleno de alineación a 4 bytes y los desplazamientos de 4, preservando el *fall-through* de la forma de dos puntos y cortándolo en la de flecha—, los `break / continue` **con y sin etiqueta** sobre una pila de sentencias «de las que se puede salir» —un `switch` interpuesto captura un `break` pero **no un continue** (§14.16)— más el **bloque etiquetado**, y `synchronized` con `monitorexit` en **las dos salidas**, la normal y un handler `catch-all` que re-lanza. De yapa, el `switch sobre String` pasó a compilar de punta a punta: el desugar ya lo bajaba a dos switches sobre `int`, pero ninguno de los dos se podía emitir. Cerrar eso destapó el último agujero **silencioso** del emisor, y en la pieza más delicada: la pila de operandos se llevaba como un **simple contador de altura**, pero un frame del `StackMapTable` tiene que declarar **los tipos** de lo que hay en ella. Mientras todos los destinos de salto caían en bordes de sentencia —con la pila vacía— no se notaba; el ternario los mete en **medio de una expresión**: en `new M(a > 0 ? 1 : 2)` el `new` y su `dup` siguen vivos en la pila, y el frame los declaraba como pila vacía. El `.class` salía igual —la barrera no ve esto, porque el emisor cree estar emitiendo bien— y lo rechazaba recién el verificador. La pila pasó a llevarse **tipada** (48 puntos de ajuste), lo que de paso hizo representable el tipo `uninitialized` (tag 8), que identifica un objeto recién creado por el **offset de su new** y, corrido su `<init>`, pasa a definitivo en todas sus apariciones —pila y locales— (§4.10.2.4). **Después** se sumó el `invokedynamic` (opcode `0xba`): el pool ganó `MethodHandle / MethodType / InvokeDynamic` y el atributo `BootstrapMethods`, y un nodo `Indy` **genérico** (`bootstrap + argumentos estáticos tipados`) sirve tanto al `LambdaMetafactory` de las `lambdas/method refs` como al `ObjectMethods` de un `record` — con el emisor empujando las capturas y el verificador propio validando el call site. Y se cerró el frente de **clases internas, locales y anónimas**: la **captura del entorno** — `this$0` de la instancia envolvente y `val$x` de los locales *effectively-final*, con un pase `hoist_anonymous` que baja cada `new T(){...}` a una local sintética `Outer$N` —, las tres formas **corren y verifican**. Por último, la **emisión de interfaces propias**: el `.class` de una `interface` sale con `ACC_INTERFACE` y sus métodos **abstractos sin Code**, y se sumó `invokeinterface` (con `InterfaceMethodref`) para las llamadas sobre un receptor de interfaz —lo que desbloqueó las **anónimas sobre una interfaz** (`new Runnable(){...}`), el caso más común) y las lambdas contra interfaces funcionales propias—.

✓ **Siguiente:** del lado del compilador, el **frente grande de invokedynamic** quedó cerrado —lambdas (con un `LambdaToMethod` propio), **referencias a método** (las cuatro formas del §15.13.1) y el `equals / hashCode / toString` de un **record** (por `ObjectMethods`) se tipan, emiten y **verifican**; la ejecución del call site vive en la JVM (KajijDK)—. Se cerraron además las **clases internas, locales y anónimas** —captura del entorno completa (`this$0 + val$`, con un pase `hoist_anonymous` que baja la anónima a una local sintética `Outer$N`), las tres **corren y verifican**, incluidas las formas **cualificadas** (`Outer.this` → cadena de `this$0`, con desambiguación de campo sombreado y multinivel; y `outer.new Inner()`, que empuja el calificador como `this$0`) y los **argumentos de super** (`new ArrayList(10){...}`), la **colisión de nombres** (dos locales homónimas se renomban a únicas `1L / 2L` con alcance léxico por bloque), la **local-en-local** (anidada como tipo de su local envolvente) y el **multinivel implícito** (acceso a un miembro del abuelo vía `this$0` encadenado) —las clases internas quedan **sin colas**—; el parser (B1) quedó **sin bordes** (anotaciones de tipo/uso `List<@NonNull String>`, `<@Foo T>`, `new <T>Foo()`), y las de **nivel de array** `String @A []`; y la **emisión de interfaces propias** cerró el caso más común de anónima (`new Runnable(){...}`) sumando `invokeinterface`. Y se cerró el **boxing dirigido por tipo** (`TransTypes`): una pasada propia inserta `Integer.valueOf / x.intValue` en cada contexto de conversión (asignación, `return`, argumentos, operandos, cuerpo de lambda), cerrando el caso insignia de las *poly expressions* — `Function<Integer,Integer> f = x -> x + 1` de punta a punta, validado por el verificador (ejecutar `valueOf` es de la JVM)—. Y se cerró la **inferencia en posición de argumento** (dos fases §15.12.2.6): un argumento **poly** —lambda, *method ref*, diamante— se re-atribuye con el tipo de su parámetro como *target* una vez resuelta la sobrecarga, lo que por fin baja una **lambda-argumento** (`call(x -> x + 1)`) a `invokedynamic`. Y se cerró el **enum con constantes parametrizadas** (`ROJO("rojo")`): al ctor propio se le antepone (`String, int`) + `super(...)` y cada constante se construye con sus argumentos. Y se **liquidaron los sueltos** que quedaban: `@Override / throws` en el chequeo de override (§8.4.8.3/§9.6.4.4), `RuntimeVisibleAnnotations` (§4.7.16), el **plegado de constantes** en un `case` (§15.28), las **formas comprimidas del StackMapTable** (§4.7.4), los **inicializadores de campo atribuidos** (§8.3.2), la **desambiguación de sobrecargas por el arg lambda** (§15.12.2.5) y el **boxing wrapper—primitivo más ancho** (`long l = anInteger`). Y se cerró el primer **frente grande de Java SE 25**: **sealed / non-sealed + exhaustividad de switch** (§8.1.1.2/§9.1.1.4/§8.1.6/§14.11.1.1) — el parser toma las keywords restringidas y el `permits`, el grafo gana los **subtipos autorizados** (con el `permits` implícito de la unidad), **Check** valida las reglas de sellado y exige **exhaustividad** a la `switch`-expresión y a la sentencia con *patterns* (jerarquía `sealed` recursiva, todas las constantes de un `enum`, `true / false` de un `boolean`; una guarda no cubre), y el `.class` lleva el atributo `PermittedSubclasses` (§4.7.31) — de yapa se corrigió que `case RED -> 1` se leía como lambda. Con eso la lista **tracked** del compilador queda vacía. Y se cerró el segundo frente de **Java SE 25**: los **text blocks** (§3.10.6) — el parser decodifica el `"""..."""` con el algoritmo del JLS (normalización de terminadores, descarte de la línea de apertura, **indentación incidental** mínima + blancos finales, y luego los escapes con `s` y la **continuación de línea** `<LF>`), dando un `String` común que se emite como cualquier literal. Y se cerró el tercer frente: la **capture conversion** (§5.1.10) con **variables de captura frescas** (un `RType::Capture{id,upper,lower}` con cotas inline e `id` fresco para la *freshness*): se aplica en acceso a miembros y en argumentos, el subtipado gana `CAP <: upper` y `s <: CAP` por la cota inferior, y el caso insignia `swap(List<?>) → swapHelper(List<T>)` ahora **infiere T**. Y se cerró el cuarto frente: los **bridge methods** (§8.4.8.3/§15.12.4.5) — el emisor sintetiza los puentes (`ACC_BRIDGE + ACC_SYNTHETIC`) que hacen que la sobrescritura funcione en bytecode cuando la *erasure* difiere, por un **parámetro genérico borrado** (`Comparable<Foo> → compareTo(Object)`, `Node<Integer>.set(Integer) → set(Object)`) o por un **retorno covariante** (`A.get():Object / B.get():String`), reenviando al método real con `checkcast + invokevirtual`; verifica y **despacha** de punta a punta. Y se cerró el último frente grande: los **módulos** (§7.7/§4.7.25) — el parser toma el `module-info.java` con las cinco directivas (keywords restringidas), el emisor produce un `module-info.class` fiel (`ACC_MODULE`, atributo `Module` con `CONSTANT_Module / Package` y el `requires java.base` mandated), y **Check** valida la buena formación (directivas duplicadas, auto-`requires`); la resolución del grafo es de runtime (JPMS, track aparte). **Con esto, los cinco frentes grandes de Java SE 25 quedan cerrados del lado del compilador** —sealed + exhaustividad, text blocks, captura de wildcards, bridge methods y módulos (B5)—. Lo que le queda al compilador ****no**** son features de lenguaje sino ****fidelidad**** (B6). Ya se cerraron los cuatro primeros atributos: ✓ **Signature** (§4.7.9) —los genéricos ya ****no** se pierden** en el binario—, ✓ **InnerClasses** (§4.7.6), ✓ **EnclosingMethod** (§4.7.7) —anidamiento y método envolvente de local/anónimas— ✓ **Record** (§4.7.30) —un `record` ahora extiende `java.lang.Record`, es `final` y lleva el atributo con sus componentes, así `isRecord() / getRecordComponents()` andan (y se cerraron dos bugs latentes: el parseo de un record genérico y la *erasure* del descriptor de una variable de tipo)— y ✓ **NestHost / NestMembers** (§4.7.28/§4.7.29) —el *nest* plano (`host + nestmates` transitivos) que da acceso privado entre anidadas—, ✓ **NestHost / NestMembers** (el *nest*) y ✓ **MethodParameters** (§4.7.24, los nombres de parámetros para la reflexión). De la ****inferencia**** ya se cerraron el lado escritura de la captura por llamada y el **lub con intersección** (`Number & Comparable`); queda solo el *solver* transitivo de §18 (bounds cruzados). Del lado de los **atributos ya no queda fidelidad reflexiva pendiente**: se cerraron `Exceptions`, `LineNumberTable`, `LocalVariableTable`, `ConstantValue` y **todas** las type annotations (`RuntimeVisibleTypeAnnotations`, §4.7.20 — parámetros de tipo, cotas, extends/implements, throws, field/return/param y los targets de `Code`: `cast / instanceof / new / local`), todo verificado byte-fiel contra el `jvarkit` de Temurin. Lo que falta para un reemplazo *drop-in* de `javac` es la **recuperación de errores** a nivel expresión y el *solver* transitivo del Cap. 18. En paralelo, lo demás es del **track de la JVM** (ejecución, resolución de módulos, concurrencia). Del lado de la JVM, seguir la concurrencia hacia el **multithread profesional** (ver «Concurrencia»): cerrar el monitor y dar el salto de substrato — OS threads + GIL (park/unpark real) → sacar el GIL → modelo de memoria (`volatile / fences`) → `java.util.concurrent`. El paralelismo de cómputo del LLM vive en **kernels off-heap** (rayon/CUDA).